Kafka Notes + Interview (Merged)

This file is auto-merged from the Markdown files under `notes/` and `interview/`.

Source files (merge order)

- `notes/01-introduction.md`
- `notes/02-brokers.md`
- `notes/03-cli-topics.md`
- `notes/04-cli-producers.md`
- `notes/05-cli-consumers.md`
- `notes/06-spring-producer.md`
- `notes/07-producer-acks-retries.md`
- `notes/08-idempotent-producer.md`
- `notes/09-spring-consumer.md`
- `notes/10-consumer-deserializer-errors.md`
- `notes/11-consumer-dlt.md`
- `notes/12-consumer-exceptions-retries.md`
- `notes/13-consumer-groups.md`
- `notes/14-idempotent-consumer.md`
- `notes/15-kafka-transactions.md`
- `notes/16-kafka-db-transactions.md`
- `notes/17-integration-testing-producer.md`
- `notes/18-saga-part-1.md`
- `notes/19-saga-part-2-compensation.md`
- `notes/20-kafka-connect.md`
- `notes/spring-kafka-annotations-reference.md`
- `interview/README.md`
- `interview/00-diagrams.md`
- `interview/01-system-design-architecture.md`
- `interview/02-kafka-internals.md`
- `interview/03-reliability-eos.md`
- `interview/04-operations-performance.md`
- `interview/05-spring-kafka.md`
- `interview/06-scenarios-incidents.md`
- `interview/07-coding-takehome.md`
- `interview/08-leadership-tradeoffs.md`

---

notes/01-introduction.md

01. Introduction to Apache Kafka

Scope and assumptions

- Kafka version: **Kafka 4.x**
- Mode: **KRaft** (no ZooKeeper)
- You may run Kafka via:
    - **Local binaries** (downloaded Kafka distribution)
    - **Docker / Docker Compose**

Question bank (covering this section)

Introduction

- What is Kafka, and what problems does it solve?
- Kafka vs traditional message brokers (RabbitMQ/ActiveMQ): what is fundamentally different?
- Kafka vs a database: why not just write events to a table?

What is Microservice

- What qualifies as a microservice (and what doesn't)?
- What are common failure modes of microservices (distributed systems issues)?

Microservice vs Monolithic application

- When should you choose a monolith?
- What kinds of complexity do microservices introduce (and why)?

Microservice communication

- Synchronous vs asynchronous communication—what tradeoffs?
- What is backpressure? How do you handle load spikes?

Event driven architecture with Apache Kafka

- What is EDA? What is a producer/consumer in EDA terms?
- Events vs commands: how do you model them?

Apache Kafka for Microservices

- Why is Kafka a good fit for decoupling microservices?
- How does Kafka support replay and reprocessing?

Messages and Events in Apache Kafka

- Message vs event vs record: what do we mean practically?
- What metadata matters for debugging (offset/partition/timestamp)?

Kafka Topic and Partitions

- What is a topic? What is a partition?
- How do partitions enable parallelism?
- How do partitions affect ordering?

Ordering of events in Apache Kafka

- What ordering guarantees exist?
- What breaks ordering (multi-partition, multiple producers, retries)?

Core concepts (detailed notes)

1) What is Apache Kafka?

Kafka is a **distributed event streaming platform**.

At a practical level, Kafka provides:

- **Durable log**: append-only storage of records, ordered by offset per partition.
- **Publish/subscribe**: producers write to topics; consumers read from topics.
- **Horizontal scalability**: partitions split load.
- **Replayability**: consumers can reread past events by seeking offsets.

Key Kafka "units"

- **Record**: a single event/message with optional `key`, `value`, `headers`, and `timestamp`.
- **Topic**: named stream of records.
- **Partition**: an ordered, append-only log. Ordering is per partition.
- **Offset**: the position of a record within a partition.
- **Consumer group**: a set of consumers sharing work; each partition is assigned to at most one consumer in the group.

Why Kafka (vs REST calls between services)

- REST introduces **tight coupling** (availability + latency coupling)
- Kafka enables:
    - **Loose coupling**: producer doesn't need the consumer online.
    - **Buffering**: Kafka acts like a shock absorber during spikes.
    - **Fan-out**: multiple consumers can react to the same event.
    - **Replay**: recover from bugs by reprocessing.

Pros

- **High throughput** and good scalability
- **Strong durability** via replication
- **Replay** and time travel
- **Decoupling** of services

Cons / pitfalls

- **Eventual consistency**: no single request/response "transaction" across services
- **Operational complexity**: brokers, partitions, replication, tuning
- **Schema evolution**: must manage payload compatibility
- **Observability**: tracing across async boundaries is harder

---

2) Microservices: what they are (and are not)

A microservice is a **small, independently deployable** service aligned to a **bounded business capability**.

Typical properties

- Independent deployment and scaling
- Owns its data (ideally)
- Clear APIs / contracts
- Failure isolation (in theory)

What microservices are NOT

- "Many tiny codebases with chatty synchronous calls" (this is a common anti-pattern)

---

3) Microservice vs Monolith

Monolith strengths

- Simpler to develop, test, and deploy
- Easier local debugging
- Stronger consistency easier (single DB, single process)

Microservice strengths

- Scale parts independently
- Deploy parts independently
- Team autonomy

Rule of thumb

- Start with a monolith unless there's a clear reason not to.
- Move to microservices when:
    - team size and domain complexity demand it
    - scaling needs are uneven
    - deployment independence creates real business value

---

4) Microservice communication

Synchronous

Examples:

- HTTP REST
- gRPC

Pros

- Simple mental model
- Immediate response

Cons

- Availability coupling (if Service B is down, Service A fails)
- Latency coupling
- Harder to handle spikes

Asynchronous

Examples:

- Kafka
- messaging queues

Pros

- Temporal decoupling (producer and consumer don't need to be online at same time)
- Buffering and smoothing spikes
- Natural fan-out

Cons

- Eventual consistency
- More moving parts (consumer retries, DLT, idempotency)

---

5) Event-Driven Architecture (EDA) with Kafka

Events vs Commands

- **Event**: “something happened” (fact). Example: `OrderCreated`.
- **Command**: “please do something” (request). Example: `ReserveInventory`.

A common microservices pattern:

- Publish an **event** for state changes.
- Send **commands** when you need a specific service to take an action.

Example flow

- `Order Service` stores order in DB
- `Order Service` publishes `OrderCreatedEvent`
- `Inventory Service` consumes it and reserves stock, then publishes `ProductReservedEvent` or `ProductReservationFailedEvent`

---

6) Topics, partitions, and ordering

Partitions

Kafka topics are split into partitions to enable:

- parallel writes (across partitions)
- parallel reads (across partitions)

Ordering

Kafka guarantees ordering **within a single partition**.

So:

- If you need strict ordering for an entity (e.g., a specific `orderId`), use a key like `orderId` so all records for that order go to the same partition.

Common ordering mistakes

- Changing partition count later can change key-to-partition mapping (depending on partitioner).
- Having multiple producers for the same key is fine, but you must still reason about ordering based on send order and retries.

Hands-on: minimal local and Docker setup (KRaft)

This section gives quick commands; we'll go deeper in the Brokers + CLI sections.

Local binaries (KRaft) – quickstart

1) Download Kafka 4.x binary distribution.

2) Start Kafka in KRaft mode.

Kafka distributions ship scripts; exact file names can vary by version, but typically:

```
# generate a cluster id
bin/kafka-storage.sh random-uuid

# format the log dirs (uses your server.properties)
bin/kafka-storage.sh format -t <CLUSTER_ID> -c config/kraft/server.properties

# start broker
bin/kafka-server-start.sh config/kraft/server.properties
```

Docker / Compose – quickstart (conceptual)

You can run Kafka in Docker using images that support KRaft.

Key ideas you must configure:

- controller quorum voters
- advertised listeners
- listener security protocol map

(We'll add a ready-to-use `docker-compose.yml` in the Brokers section file once we decide your preferred image: Apache vs Bitnami vs Confluent.)

Cheat sheet

- **Topic** = stream name
- **Partition** = ordered log shard of a topic
- **Offset** = position inside a partition
- **Ordering** = guaranteed only within a partition
- **Scaling consumers** = add partitions + consumers within a group

---

notes/02-brokers.md

02. Apache Kafka Brokers (KRaft)

Question bank (covering this section)

Broker fundamentals

- What is a Kafka broker responsible for?
- What is stored on disk (segments, index files)?
- What is the role of the controller in Kafka (KRaft)?

Leaders and followers

- What is a partition leader?
- What is a follower replica?
- What is ISR (in-sync replicas) and why does it matter?

Leadership balance

- Why is leadership balance important?
- What happens when a broker hosting many leaders becomes overloaded?

Multiple brokers

- What must be unique per broker (`node.id`, listeners/ports, log dirs)?
- How does replication factor interact with number of brokers?

Storage folders

- Why use multiple `log.dirs`?
- What happens when a disk fills up?

Starting/stopping in KRaft

- What is the KRaft cluster ID?
- What is the controller quorum voters setting?
- How do you safely stop brokers?

Core concepts (detailed notes)

1) What is a Kafka broker?

A **broker** is a Kafka server process that:

- accepts produce requests

- serves fetch requests
- stores partitions on disk
- replicates partitions to other brokers

In a cluster, brokers collaborate to provide:

- horizontal scale
- replication and fault tolerance

---

2) Partition leader and followers

Each partition has:

- **Leader replica**: handles all reads/writes for that partition
- **Follower replicas**: replicate data from the leader

ISR (In-Sync Replicas)

ISR is the set of replicas that are:

- fully caught up
- considered safe enough to be used for durability guarantees

If you set `acks=all`, Kafka requires acknowledgements from the leader and enough ISR replicas (subject to `min.insync.replicas`).

---

3) Leadership balance

Kafka tries to distribute leaders across brokers.

Why it matters:

- Leaders do extra work (handle clients)
- If one broker becomes leader-heavy, it can become CPU/network hot spot

Typical operational actions:

- ensure even partition distribution
- use partition reassignment tools during scaling

---

4) Multiple broker configuration: what must differ?

When running multiple brokers on one machine (lab setup), you must make these unique per broker:

- `node.id`
- listeners port(s) (e.g., 9092, 9093, 9094)
- `log.dirs` (e.g., `./data/broker-1`, `./data/broker-2`)

And in KRaft, you must set consistent cluster-wide settings:

- `process.roles` (broker/controller or split roles)
- `controller.quorum.voters` (same list on all nodes)
- `controller.listener.names`

---

5) Storage folders and log segments

Kafka stores each partition as:

- multiple **log segment files** (append-only)
- index files for fast lookup

Why multiple `log.dirs` can help:

- spread partitions across disks
- more throughput

Pitfalls:

- if any disk becomes unavailable, partitions on that disk become unavailable
- disk-full causes broker issues; monitor disk usage

Local binaries: running 3 brokers in KRaft on one machine (lab)

Below is a **repeatable local-lab pattern**. Exact filenames may vary by Kafka distribution, but the concepts are stable.

Step A: Create 3 config files

Copy the base KRaft config 3 times:

- `config/kraft/server-1.properties`
- `config/kraft/server-2.properties`
- `config/kraft/server-3.properties`

Typical differences:

- `node.id=1/2/3`
- `listeners=PLAINTEXT://:9092` (then 9093, 9094)
- `advertised.listeners=PLAINTEXT://localhost:9092` etc.

- `log.dirs=/tmp/kraft-combined-logs-1` etc.

Step B: Generate cluster id and format storage

```
CLUSTER_ID=$(bin/kafka-storage.sh random-uuid)

bin/kafka-storage.sh format -t $CLUSTER_ID -c config/kraft/server-1.properties
bin/kafka-storage.sh format -t $CLUSTER_ID -c config/kraft/server-2.properties
bin/kafka-storage.sh format -t $CLUSTER_ID -c config/kraft/server-3.properties
```

Step C: Start brokers

```
bin/kafka-server-start.sh config/kraft/server-1.properties
bin/kafka-server-start.sh config/kraft/server-2.properties
bin/kafka-server-start.sh config/kraft/server-3.properties
```

Step D: Stop brokers

Prefer a graceful stop:

```
bin/kafka-server-stop.sh
```

If you started each broker in its own terminal, stop each one cleanly to avoid log recovery time.

Docker / Docker Compose (KRaft)

You can run Kafka 4.x in KRaft using the **Apache Kafka official image**.

Files included in this repo:

- `docker/docker-compose.kafka-kraft-1broker.yml`
- `docker/docker-compose.kafka-kraft-3brokers.yml`

How to run (1 broker)

```
docker compose -f docker/docker-compose.kafka-kraft-1broker.yml up
```

Host bootstrap:

- `localhost:9091`

How to run (3 brokers)

```
docker compose -f docker/docker-compose.kafka-kraft-3brokers.yml up
```

Host bootstrap options (any one is enough):

- `localhost:9091`
- `localhost:9092`
- `localhost:9093`

Quick test (topics)

```
bin/kafka-topics.sh --bootstrap-server localhost:9091 --list
```

Listener model used in these compose files

Each broker defines 3 listeners:

- `CONTROLLER` (internal quorum comms)
- `PLAINTEXT` (inter-broker + in-network client access using Docker DNS, e.g. `kafka-1:9192`)
- `EXTERNAL` (host access via `localhost:<port>`)

Key settings:

- `KAFKA_LISTENERS` binds ports in the container
- `KAFKA_ADVERTISED_LISTENERS` is what clients are told to connect to

If `KAFKA_ADVERTISED_LISTENERS` is wrong, you'll see the classic symptom:

- initial connection works
- then producer/consumer fails because broker returns an unreachable address

---

Pros / Cons recap

Pros

- Multi-broker clusters provide durability and availability
- Partition replication allows broker failures without losing data

Cons / pitfalls

- Misconfigured listeners/adverts are the #1 source of "it runs but clients can't connect"
- Too few brokers for the replication factor causes topic creation failures
- Unbalanced leadership can create hot brokers

---

notes/03-cli-topics.md

03. Kafka CLI: Topics (Kafka 4.x / KRaft)

Question bank

- What is the difference between a **topic**, **partition**, and **replica**?
- What does **replication factor** actually replicate (partitions) and why?
- What does topic `--describe` output mean (leader, replicas, ISR)?
- When should you increase partitions? What are the tradeoffs?
- What happens when you delete a topic? Is it immediate?
- What are common topic-level configs (retention, cleanup policy) and what do they do?

Prerequisites

Local binaries

Assumptions:

- Kafka is running locally.
- You have Kafka scripts like `bin/kafka-topics.sh`.
- Your bootstrap server is reachable at something like `localhost:9092`.

Set a helper variable:

```
export BS=localhost:9092
```

Docker / Compose

Two common patterns:

- Run CLI inside the Kafka container
- Run CLI on host and connect to `localhost:<mapped-port>` (depends on `advertised.listeners`)

I'll provide both patterns below; for Compose specifics we'll finalize once you pick the image family.

Core commands (local binaries)

1) Create a new topic

```
bin/kafka-topics.sh --bootstrap-server $BS --create \
  --topic orders \
  --partitions 3 \
  --replication-factor 1
```

Notes:

- `--partitions` controls parallelism.
- `--replication-factor` must be <= number of brokers.

2) List topics

```
bin/kafka-topics.sh --bootstrap-server $BS --list
```

3) Describe a topic

```
bin/kafka-topics.sh --bootstrap-server $BS --describe --topic orders
```

How to interpret:

- **Leader**: broker id currently handling reads/writes for that partition
- **Replicas**: all replicas assigned to the partition
- **ISR**: replicas currently "in-sync" with the leader

4) Delete a topic

```
bin/kafka-topics.sh --bootstrap-server $BS --delete --topic orders
```

Notes:

- Delete is not always immediate; it is typically asynchronous.

Topic configuration essentials

Retention (time/size)

Retention determines how long Kafka keeps data (unless compacted).

Common settings:

- `retention.ms`
- `retention.bytes`

Example (alter topic config):

```
bin/kafka-configs.sh --bootstrap-server $BS \
  --entity-type topics --entity-name orders \
  --alter --add-config retention.ms=604800000
```

Cleanup policy

- `cleanup.policy=delete` (default for event streams)
- `cleanup.policy=compact` (keeps last value per key; great for "current state" topics)

Example:

```
bin/kafka-configs.sh --bootstrap-server $BS \
  --entity-type topics --entity-name user-profiles \
  --alter --add-config cleanup.policy=compact
```

Docker patterns

Pattern A: Run CLI inside container

```
docker exec -it <kafka_container> bash

# inside container
export BS=<bootstrap-from-container-context>

kafka-topics.sh --bootstrap-server $BS --list
```

Pattern B: Run CLI on host against mapped port

```
export BS=localhost:9092
bin/kafka-topics.sh --bootstrap-server $BS --list
```

This works only if your container is configured with correct `advertised.listeners` so clients on the host can connect.

Pros / Cons / gotchas

Pros

- Topic+partition model makes scaling straightforward.
- Replication provides durability and availability.

Cons / gotchas

- Increasing partitions can change key->partition mapping (depending on partitioner), which can affect ordering guarantees.
- High partition counts increase overhead (file handles, memory, rebalancing time).
- Deleting topics can be slow and may be disabled in some environments.

---

notes/04-cli-producers.md

04. Kafka CLI: Producers (Kafka 4.x / KRaft)

Question bank

- What does a producer do (serialize, partition, batch, send)?
- How does Kafka choose a partition when you provide **no key**?
- Why does providing a **key** matter for ordering?
- What delivery semantics do you get by default?
- What are typical causes of duplicate messages?

Prerequisites

Local binaries

```
export BS=localhost:9092
```

Create a topic if needed:

```
bin/kafka-topics.sh --bootstrap-server $BS --create --topic orders --partitio
ns 3 --replication-factor 1
```

Producing without a key

```
bin/kafka-console-producer.sh --bootstrap-server $BS --topic orders
```

Type messages and press Enter:

```
order-1-created
order-2-created
```

Notes:

- Without a key, partition selection is typically round-robin (or sticky partitioning depending on client behavior).
- Ordering is only guaranteed within each partition; without a key, records can land on different partitions.

Sending key:value messages

Kafka console producer supports parsing keys:

```
bin/kafka-console-producer.sh --bootstrap-server $BS \
  --topic orders \
  --property parse.key=true \
  --property key.separator=:
```

Now type:

```
orderId-101:OrderCreated
orderId-101:OrderConfirmed
orderId-202:OrderCreated
```

Notes:

- Same key (e.g., `orderId-101`) will consistently map to the same partition (given same partition count and partitioner behavior), preserving ordering for that key.

Docker patterns

Pattern A: CLI inside container

```
docker exec -it <kafka_container> bash
kafka-console-producer.sh --bootstrap-server <BS> --topic orders
```

Pattern B: CLI on host to mapped port

```
bin/kafka-console-producer.sh --bootstrap-server localhost:9092 --topic order
s
```

Pros / Cons / gotchas

Pros

- Fast way to validate topic connectivity and produce flow.
- Keyed messages help ensure per-entity ordering.

Cons / gotchas

- The console producer is not representative of production settings (acks, retries, idempotence).
- If you change partition count later, your key distribution can change.
- Without keys, downstream consumers will see events for the same business entity across partitions (harder to process in order).

---

notes/05-cli-consumers.md

05. Kafka CLI: Consumers (Kafka 4.x / KRaft)

Question bank

- What is a consumer group and why does it matter?
- How do offsets work? What is “committing an offset”?
- What does `--from-beginning` actually do?
- How do you consume only new messages?
- What does “in order” mean with multiple partitions?

Prerequisites

```
export BS=localhost:9092
export TOPIC=orders
```

Consume messages (default: new messages only)

```
bin/kafka-console-consumer.sh --bootstrap-server $BS --topic $TOPIC
```

Notes:

- This typically starts at the end (new messages) unless group offsets exist and `auto.offset.reset` applies.

Consume from the beginning

```
bin/kafka-console-consumer.sh --bootstrap-server $BS --topic $TOPIC --from-beginning
```

Notes:

- This is useful for debugging and demos.

Consume as part of a consumer group

```
bin/kafka-console-consumer.sh --bootstrap-server $BS --topic $TOPIC --group orders-cli
```

Notes:

- Using `--group` makes Kafka commit offsets for that group.

- Running multiple consumers with the same group will share partitions.

Consume key:value messages

To print keys:

```
bin/kafka-console-consumer.sh --bootstrap-server $BS \
  --topic $TOPIC \
  --from-beginning \
  --property print.key=true \
  --property key.separator=" : "
```

Consume “in order”

Important reality:

- Kafka guarantees order **per partition**, not across partitions.

If you need strict ordering for a business entity, produce with a key (e.g., `orderId`) so all events for that entity map to one partition.

You can also force the console consumer to read a single partition using `kafka-consumer-groups` tooling and assignments, but the console consumer is mainly for demos.

Inspect consumer groups and offsets

List groups:

```
bin/kafka-consumer-groups.sh --bootstrap-server $BS --list
```

Describe a group:

```
bin/kafka-consumer-groups.sh --bootstrap-server $BS --describe --group orders-cli
```

This shows:

- current committed offsets
- lag (how far behind)
- which consumer owns which partition

Docker patterns

Pattern A: CLI inside container

```
docker exec -it <kafka_container> bash
kafka-console-consumer.sh --bootstrap-server <BS> --topic orders --from-begin
ning
```

Pattern B: CLI on host to mapped port

```
bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic order
s --from-beginning
```

Pros / Cons / gotchas

Pros

- Quick validation of end-to-end (produce -> broker -> consume).
- Easy way to inspect lag via `kafka-consumer-groups.sh`.

Cons / gotchas

- CLI consumers/producers don't represent production behavior (error handling, retries, deserialization, transactions).
- Reading from the beginning without a group can reprocess lots of data; be careful in shared environments.

---

notes/06-spring-producer.md

06. Kafka Producer – Spring Boot Microservice

Question bank

- What's the difference between synchronous REST calls and publishing Kafka events?
- When should a controller return success if the Kafka send is async?
- How do you model an event class (schema, versioning, backward compatibility)?
- How do you choose topic name, partitions, and the message key?
- How do you handle send failures (serialization error, timeout, authorization)?
- What metadata can you log from a successful send (topic/partition/offset/timestamp)?

Minimal setup (dependencies)

Maven

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

Notes:

- You can also use `spring-boot-starter-actuator` for health endpoints and metrics.

---

application.yml (producer)

Local Kafka binaries

If your broker is on `localhost:9092`:

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSeri
alizer
      properties:
        spring.json.add.type.headers: false
```

Docker Compose in this repo

Using the compose files we created, the host can connect via `localhost:9091` (or `9092`, `9093`). Example:

```
spring:
  kafka:
    bootstrap-servers: localhost:9091
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSeri
alizer
      properties:
        spring.json.add.type.headers: false
```

Why `spring.json.add.type.headers: false`:

- It avoids adding Spring-specific type headers.
- It reduces coupling between services if consumers are not Spring.

Creating a topic (recommended: explicit)

In local/dev, create topics via CLI:

```
bin/kafka-topics.sh --bootstrap-server localhost:9091 --create \
  --topic orders.events \
  --partitions 3 \
  --replication-factor 1
```

Event class

Example event payload:

```
public record OrderCreatedEvent(
    String eventId,
    String orderId,
    java.time.Instant createdAt,
    java.math.BigDecimal amount,
    String currency
) {}
```

Best practices:

- Include `eventId` (UUID) for traceability/idempotency.
- Use ISO timestamps (`Instant`).
- Prefer adding new fields over changing/removing fields.

Producer implementation patterns

1) KafkaTemplate (recommended)

1A) Explicit @Bean configuration (ProducerFactory + KafkaTemplate)

If you prefer explicit beans (instead of relying only on `spring.kafka.*` properties), configure a producer factory and template.

```
@Configuration
public class KafkaProducerConfig {
```

```
  @Bean
  public ProducerFactory<String, OrderCreatedEvent> producerFactory(
      org.springframework.boot.autoconfigure.kafka.KafkaProperties kafkaPrope
rties
  ) {
    var props = new java.util.HashMap<String, Object>(kafkaProperties.buildPr
oducerProperties());

    props.put(org.apache.kafka.clients.producer.ProducerConfig.KEY_SERIALIZER
_CLASS_CONFIG,
        org.apache.kafka.common.serialization.StringSerializer.class);
    props.put(org.apache.kafka.clients.producer.ProducerConfig.VALUE_SERIALIZ
ER_CLASS_CONFIG,
        org.springframework.kafka.support.serializer.JsonSerializer.class);

    props.put(org.springframework.kafka.support.serializer.JsonSerializer.ADD
_TYPE_INFO_HEADERS, false);

    return new org.springframework.kafka.core.DefaultKafkaProducerFactory<>(p
rops);
  }

  @Bean
  public KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate(
      ProducerFactory<String, OrderCreatedEvent> producerFactory
  ) {
    return new KafkaTemplate<>(producerFactory);
  }
}
```

Producer service

```
@Service
public class OrderEventProducer {

  private final KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate;

  public OrderEventProducer(KafkaTemplate<String, OrderCreatedEvent> kafkaTem
plate) {
    this.kafkaTemplate = kafkaTemplate;
  }

  public CompletableFuture<SendResult<String, OrderCreatedEvent>> publishOrde
rCreated(
      String orderId,
      OrderCreatedEvent event
  ) {
    return kafkaTemplate.send("orders.events", orderId, event);
```

```
  }
}
```

Why use `orderId` as the key:

- Ensures all events for a given order go to the same partition.
- Preserves ordering for that order.

---

2) Send asynchronously and return immediately

Controller pattern

```java
@RestController
@RequestMapping("/orders")
public class OrderController {

  private final OrderEventProducer producer;

  public OrderController(OrderEventProducer producer) {
    this.producer = producer;
  }

  @PostMapping("/{orderId}")
  public ResponseEntity<Void> createOrder(@PathVariable String orderId) {
    var event = new OrderCreatedEvent(
        java.util.UUID.randomUUID().toString(),
        orderId,
        java.time.Instant.now(),
        new java.math.BigDecimal("100.00"),
        "INR"
    );

    producer.publishOrderCreated(orderId, event)
        .whenComplete((result, ex) -> {
          if (ex != null) {
            // log error; optionally trigger alert
            return;
          }

          var meta = result.getRecordMetadata();
          // log topic/partition/offset/timestamp
        });

    return ResponseEntity.accepted().build();
  }
}
```

Notes:

- Returning `202 Accepted` matches async processing.
- If the use-case requires hard guarantee that the event is in Kafka before responding, see next section.

---

3) Async send but wait synchronously (use carefully)

Sometimes you want to block until Kafka acknowledges the send (e.g., request must fail if Kafka is unavailable).

```
try {
  var sendResult = producer.publishOrderCreated(orderId, event).get(5, TimeUn
it.SECONDS);
  // ok
} catch (Exception ex) {
  // return 503 or 500
}
```

Tradeoffs:

- Increases latency
- Reintroduces availability coupling (similar to sync calls)

Handling exceptions (what can fail)

Common failure categories:

- Serialization errors (bad payload)
- Timeout / broker unavailable
- Authorization failures
- Message too large

Where to handle:

- **Controller**: translate to HTTP response code
- **Producer callback**: log metadata on success, log root cause on failure

---

Logging record metadata (why it matters)

On success, log:

- `topic`
- `partition`
- `offset`

- `timestamp`

This helps:

- prove delivery
- locate the exact record in Kafka for debugging

Pros / Cons

Pros

- Decouples producer from downstream services
- Supports fan-out (many consumers)
- Supports replay of events

Cons / pitfalls

- You must design for duplicates (at-least-once delivery is common)
- Schema/version management is required
- “Fire-and-forget” without observability can hide failures

---

notes/07-producer-acks-retries.md

07. Kafka Producer: Acknowledgements & Retries (Spring Boot)

Question bank

- What does `acks=0/1/all` mean?
- What is `min.insync.replicas` and how does it interact with `acks=all`?
- When do retries create duplicates?
- What is the difference between `request.timeout.ms` and `delivery.timeout.ms`?
- What is the difference between broker-side durability and end-to-end processing guarantee?

Producer acknowledgements

`acks=0`

- Producer does not wait for broker acknowledgement.

Pros

- Lowest latency

Cons

- Highest risk of data loss
- You usually don't want this for business events

`acks=1`

- Leader writes the record and acknowledges.

Pros

- Good latency

Cons

- If leader dies before followers replicate, data may be lost

`acks=all` (or `acks=-1`)

- Leader waits for all required ISR replicas.

Pros

- Best durability

Cons

- Higher latency
- Requires correct `min.insync.replicas` to mean anything

`min.insync.replicas`

- This is a **broker/topic-side** setting.
- With `acks=all`, the write only succeeds if at least `min.insync.replicas` replicas are in ISR.

Example consequences:

- replication factor = 3
- `min.insync.replicas=2`

Then:

- you can tolerate 1 broker down and still accept writes
- if ISR drops below 2, producers get errors (this is good—prevents silent data loss)

Retries (high level)

Retries can happen due to:

- transient network issues
- leader elections
- request timeouts

Important:

- Retries can cause duplicates unless you use idempotence (Section 08).

Timeouts

`request.timeout.ms`

- Per-request timeout waiting for broker response.

`delivery.timeout.ms`

- Upper bound on time spent trying to deliver a record (includes retries + linger).

Rule of thumb:

- `delivery.timeout.ms` should be greater than `request.timeout.ms`.

Spring Boot configuration options

Option 1: application.yml

```yaml
spring:
  kafka:
    producer:
      acks: all
      properties:
        retries: 10
        request.timeout.ms: 30000
        delivery.timeout.ms: 120000
```

Notes:

- Spring maps `spring.kafka.producer.acks` to producer config.
- Some properties must go under `producer.properties.*`.

Option 2 (B): Explicit @Bean configuration

```java
@Configuration
public class KafkaProducerReliabilityConfig {
```

```
  @Bean
  public ProducerFactory<String, OrderCreatedEvent> producerFactory(
      org.springframework.boot.autoconfigure.kafka.KafkaProperties kafkaProperties
  ) {
    var props = new java.util.HashMap<String, Object>(kafkaProperties.buildProducerProperties());

    props.put(org.apache.kafka.clients.producer.ProducerConfig.ACKS_CONFIG, "all");
    props.put(org.apache.kafka.clients.producer.ProducerConfig.RETRIES_CONFIG, 10);

    props.put(org.apache.kafka.clients.producer.ProducerConfig.REQUEST_TIMEOUT_MS_CONFIG, 30_000);
    props.put(org.apache.kafka.clients.producer.ProducerConfig.DELIVERY_TIMEOUT_MS_CONFIG, 120_000);

    return new org.springframework.kafka.core.DefaultKafkaProducerFactory<>(props);
  }
}
```

Common pitfalls

- Using `acks=all` but keeping replication factor = 1 (no benefit).
- Using `acks=all` with replication factor = 3 but leaving `min.insync.replicas=1` (weaker than expected).
- Large retries without idempotence → duplicates.

Practical recommendation (baseline)

For business events:

- `acks=all`
- replication factor >= 3 (prod)
- `min.insync.replicas=2` (prod)
- enable idempotence (next section)

---

notes/08-idempotent-producer.md

08. Idempotent Producer in Kafka (Spring Boot)

Question bank

- What problem does an idempotent producer solve?
- What are Producer ID (PID) and sequence numbers used for?
- Does idempotence guarantee exactly-once end-to-end?
- What configs are required / implied by enabling idempotence?

Why idempotent producer?

Without idempotence:

- producer retries can cause **duplicate records** (same logical message written multiple times).

With idempotence enabled:

- Kafka can detect duplicate sends from the same producer session and avoid writing duplicates to the log.

What it does NOT solve:

- consumer side duplicates (retries / rebalances)
- duplicates across different producer instances
- atomic multi-topic writes (that's transactions)

How it works (conceptually)

Kafka uses:

- **PID** (Producer ID) assigned by the broker
- **Sequence numbers** per partition

The broker can reject/ignore duplicates based on sequence numbers.

Enabling idempotence

Option 1: application.yml

```yaml
spring:
  kafka:
    producer:
      properties:
        enable.idempotence: true
```

Recommended companion settings (often implied/required):

- `acks=all`
- `retries > 0`

Option 2 (B): Explicit @Bean configuration

```java
@Configuration
public class KafkaIdempotentProducerConfig {

  @Bean
  public ProducerFactory<String, OrderCreatedEvent> producerFactory(
      org.springframework.boot.autoconfigure.kafka.KafkaProperties kafkaProperties
  ) {
    var props = new java.util.HashMap<String, Object>(kafkaProperties.buildProducerProperties());

    props.put(org.apache.kafka.clients.producer.ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);
    props.put(org.apache.kafka.clients.producer.ProducerConfig.ACKS_CONFIG, "all");
    props.put(org.apache.kafka.clients.producer.ProducerConfig.RETRIES_CONFIG, Integer.MAX_VALUE);

    return new org.springframework.kafka.core.DefaultKafkaProducerFactory<>(props);
  }
}
```

Notes:

- Using `Integer.MAX_VALUE` retries is common in examples; in real systems you still cap `delivery.timeout.ms` to avoid retrying forever.

Pros / Cons

Pros

- Greatly reduces producer-side duplicates from retries.
- Strong baseline for reliability.

Cons / pitfalls

- Does not replace consumer idempotency.
- If you rely on retries, you must still tune timeouts for your SLOs.
- Ordering is per partition; use keys correctly.

---

notes/09-spring-consumer.md

09. Kafka Consumer – Spring Boot Microservice

Question bank

- What is a consumer group and how does it scale consumption?
- What is an offset and when is it committed?
- What does `auto.offset.reset` do (earliest/latest/none)?
- What's the difference between at-most-once and at-least-once consumption?
- How do you handle failures: retry, skip, DLT?
- Why do you need idempotency in consumers?

Minimal setup (dependencies)

Maven

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

---

application.yml (consumer)

Local Kafka binaries

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
```

```
    consumer:
      group-id: orders-service
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeseriali
zer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDe
serializer
      properties:
        spring.json.trusted.packages: "*"
```

Docker Compose in this repo

```
spring:
  kafka:
    bootstrap-servers: localhost:9091
    consumer:
      group-id: orders-service
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeseriali
zer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDe
serializer
      properties:
        spring.json.trusted.packages: "*"
```

Notes:

- `group-id` identifies the consumer group.
- `auto-offset-reset: earliest` is convenient for dev; in prod it's often `latest` depending on use case.
- `spring.json.trusted.packages` should be restricted in real systems (avoid `*`).

Event class (must match producer)

```
public record OrderCreatedEvent(
    String eventId,
    String orderId,
    java.time.Instant createdAt,
    java.math.BigDecimal amount,
    String currency
) {}
```

Explicit @Bean configuration (ConsumerFactory + ListenerContainerFactory)

If you prefer explicit beans (instead of relying only on `spring.kafka.*` properties), configure a consumer factory and a listener container factory.

```
@Configuration
public class KafkaConsumerConfig {

  @Bean
  public ConsumerFactory<String, OrderCreatedEvent> consumerFactory(
      org.springframework.boot.autoconfigure.kafka.KafkaProperties kafkaPrope
rties
  ) {
    var props = new java.util.HashMap<String, Object>(kafkaProperties.buildCo
nsumerProperties());

    var valueDeserializer = new org.springframework.kafka.support.serializer.
JsonDeserializer<>(OrderCreatedEvent.class);
    valueDeserializer.addTrustedPackages("*");

    return new org.springframework.kafka.core.DefaultKafkaConsumerFactory<>(
        props,
        new org.apache.kafka.common.serialization.StringDeserializer(),
        valueDeserializer
    );
  }

  @Bean
  public ConcurrentKafkaListenerContainerFactory<String, OrderCreatedEvent> k
afkaListenerContainerFactory(
      ConsumerFactory<String, OrderCreatedEvent> consumerFactory
  ) {
    var factory = new ConcurrentKafkaListenerContainerFactory<String, OrderCr
eatedEvent>();
    factory.setConsumerFactory(consumerFactory);

    // Example: use manual acks if you want to commit only after processing s
ucceeds.
    // (You'll typically combine this with retries/DLT error handling.)
    factory.getContainerProperties().setAckMode(
        org.springframework.kafka.listener.ContainerProperties.AckMode.MANUAL
    );

    return factory;
  }
}
```

Consuming with @KafkaListener

Basic listener

```
@Service
public class OrderEventsListener {

  @KafkaListener(topics = "orders.events", groupId = "orders-service")
  public void onMessage(
      OrderCreatedEvent event,
      @Header(org.springframework.kafka.support.KafkaHeaders.RECEIVED_KEY) St
ring key,
      @Header(org.springframework.kafka.support.KafkaHeaders.RECEIVED_PARTITI
ON) int partition,
      @Header(org.springframework.kafka.support.KafkaHeaders.OFFSET) long off
set
  ) {
    // handle event
    // log key/partition/offset for traceability
  }
}
```

Notes:

- Spring will deserialize JSON into `OrderCreatedEvent` if configured.
- Ordering is per partition; if you key by `orderId` on producer, you get per-order ordering.

Consumer concurrency (scaling)

You can scale consumption by:

- increasing topic partitions
- increasing consumer instances in the same group
- setting concurrency (threads) in the listener container

Example config concept:

```
spring:
  kafka:
    listener:
      concurrency: 3
```

Rule:

- Effective parallelism is capped by partition count.

Listener container factory (when you need it)

You configure a custom `ConcurrentKafkaListenerContainerFactory` when you need:

- manual ack mode
- custom error handlers
- retry/backoff behavior
- record filtering

Example skeleton:

```
@Configuration
public class KafkaConsumerConfig {

  @Bean
  public ConcurrentKafkaListenerContainerFactory<String, OrderCreatedEvent> k
afkaListenerContainerFactory(
      ConsumerFactory<String, OrderCreatedEvent> consumerFactory
  ) {
    var factory = new ConcurrentKafkaListenerContainerFactory<String, OrderCr
eatedEvent>();
    factory.setConsumerFactory(consumerFactory);
    return factory;
  }
}
```

Commit strategies (high level)

Auto-commit

- Simple but risky: may commit offsets before your processing completes.

Manual commit / ack

- More control: commit only after processing succeeds.
- Usually used with retries/DLT patterns.

(We'll implement the detailed retry/DLT strategies in Sections 10–12.)

Pros / Cons

Pros

- Easy consumption model with `@KafkaListener`
- Consumer groups scale horizontally

Cons / pitfalls

- Deserialization errors can block a partition if not handled
- Without idempotency, retries can cause duplicate side effects
- Rebalances can interrupt processing; keep processing efficient and tune timeouts

---

notes/10-consumer-deserializer-errors.md

10. Kafka Consumer – Handle Deserializer Errors (Spring Boot)

Question bank

- What typically causes deserialization errors?
- Why can one bad message block a partition?
- What is `ErrorHandlingDeserializer` and how does it help?
- What should you do with poison-pill messages (skip, DLT, fix schema)?

Why deserialization errors are dangerous

If a consumer cannot deserialize a record:

- it may throw on poll
- the listener may fail repeatedly
- the partition may effectively become stuck (same bad record is retried)

Common causes:

- schema changes not backward compatible
- wrong serializer/deserializer pairing (string vs json)
- corrupted payloads

Approach: ErrorHandlingDeserializer

Spring Kafka provides `ErrorHandlingDeserializer` which wraps your real deserializer and:

- prevents the consumer from crashing
- attaches deserialization exception details in headers

application.yml example

```yaml
spring:
  kafka:
    consumer:
      key-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      properties:
        spring.deserializer.key.delegate.class: org.apache.kafka.common.serialization.StringDeserializer
        spring.deserializer.value.delegate.class: org.springframework.kafka.support.serializer.JsonDeserializer
        spring.json.trusted.packages: "*"
```

Option B: Explicit @Bean configuration idea

Typically you still set ErrorHandlingDeserializer via properties, but you can also build consumer props in a @Configuration.

```java
@Configuration
public class KafkaDeserializerErrorConfig {

  @Bean
  public ConsumerFactory<String, OrderCreatedEvent> consumerFactory(
      org.springframework.boot.autoconfigure.kafka.KafkaProperties kafkaProperties
  ) {
    var props = new java.util.HashMap<String, Object>(kafkaProperties.buildConsumerProperties());

    props.put(org.apache.kafka.clients.consumer.ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
        org.springframework.kafka.support.serializer.ErrorHandlingDeserializer.class);
    props.put(org.apache.kafka.clients.consumer.ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
        org.springframework.kafka.support.serializer.ErrorHandlingDeserializer.class);

    props.put(org.springframework.kafka.support.serializer.ErrorHandlingDeserializer.KEY_DESERIALIZER_CLASS,
        org.apache.kafka.common.serialization.StringDeserializer.class);
    props.put(org.springframework.kafka.support.serializer.ErrorHandlingDeserializer.VALUE_DESERIALIZER_CLASS,
        org.springframework.kafka.support.serializer.JsonDeserializer.class);
```

```
    props.put(org.springframework.kafka.support.serializer.JsonDeserializer.T
RUSTED_PACKAGES, "*");

    return new org.springframework.kafka.core.DefaultKafkaConsumerFactory<>(p
rops);
  }
}
```

What to do after you can consume safely

Once the consumer can keep running, you choose a policy:

- **Send to DLT** (recommended for poison pills)
- **Skip** (only if safe and you can audit)
- **Fix producer/schema** and replay

Next section covers DLT.

---

notes/11-consumer-dlt.md

11. Kafka Consumer – Dead Letter Topic (DLT) (Spring Boot)

Question bank

- What is a Dead Letter Topic and when should you use it?
- What metadata should be preserved when sending to DLT?
- How do `DefaultErrorHandler` and `DeadLetterPublishingRecoverer` work together?
- How do you name DLT topics (convention)?

What is a DLT?

A Dead Letter Topic is a topic where you publish records that:

- cannot be processed successfully
- should not block the main topic
- require manual investigation or offline processing

Common cases:

- deserialization errors
- validation errors
- unknown enum/value due to version mismatch

Spring Kafka building blocks

- `DefaultErrorHandler`: decides retry/backoff and when to give up
- `DeadLetterPublishingRecoverer`: publishes failed record to DLT

Recommended pattern (Option B): configure an error handler

```
@Configuration
public class KafkaDltConfig {

  @Bean
  public org.springframework.kafka.core.KafkaTemplate<Object, Object> kafkaTe
mplate(
      org.springframework.kafka.core.ProducerFactory<Object, Object> producer
Factory
  ) {
    return new org.springframework.kafka.core.KafkaTemplate<>(producerFactor
y);
  }

  @Bean
  public org.springframework.kafka.listener.DefaultErrorHandler defaultErrorH
andler(
      org.springframework.kafka.core.KafkaTemplate<Object, Object> template
  ) {
    var recoverer = new org.springframework.kafka.listener.DeadLetterPublishi
ngRecoverer(template,
        (record, ex) -> new org.apache.kafka.common.TopicPartition(record.top
ic() + ".DLT", record.partition()));

    var backoff = new org.springframework.util.backoff.FixedBackOff(1000L, 3
L);

    return new org.springframework.kafka.listener.DefaultErrorHandler(recover
er, backoff);
  }
}
```

Notes:

- This publishes to `originalTopic.DLT` on the same partition.
- Backoff here is 1s, 3 retries, then DLT.

Listener container factory wiring

In your `ConcurrentKafkaListenerContainerFactory`, set the common error handler:

```
factory.setCommonErrorHandler(defaultErrorHandler);
```

(Keep this in the same `@Configuration` where you build the factory.)

DLT best practices

- Preserve original headers + add exception details
- Add correlation id / event id (ideally already present)
- Create a separate consumer/process to inspect and reprocess DLT records

---

Pros / Cons

Pros

- Prevents poison pills from blocking the main topic
- Enables operational workflow for failures

Cons / pitfalls

- If you ignore the DLT, you're just hiding failures
- DLT volume spikes often indicate schema compatibility problems

---

notes/12-consumer-exceptions-retries.md

12. Kafka Consumer – Exceptions and Retries (Spring Boot)

Question bank

- What is retryable vs non-retryable exception?
- How do you configure backoff and max attempts?
- What happens if you retry forever?
- How do you ensure side effects are not duplicated (idempotency)?
- When should you send to DLT?

Retry strategy overview

In consumers, failures usually fall into 2 buckets:

Non-retryable (fail fast)

Examples:

- validation errors
- schema incompatibility
- business rule violation

Action:

- send to DLT (or skip with audit)

Retryable (transient)

Examples:

- temporary DB outage
- downstream service timeout
- transient network failure

Action:

- retry with backoff
- if still failing, DLT

Create retryable and non-retryable exceptions

Example:

```
public class RetryableBusinessException extends RuntimeException {
  public RetryableBusinessException(String message) { super(message); }
}

public class NonRetryableBusinessException extends RuntimeException {
  public NonRetryableBusinessException(String message) { super(message); }
}
```

Configure DefaultErrorHandler

Option B: Explicit bean config

```
@Configuration
public class KafkaRetryConfig {

  @Bean
  public org.springframework.kafka.listener.DefaultErrorHandler defaultErrorH
andler(
      org.springframework.kafka.core.KafkaTemplate<Object, Object> template
  ) {
    var recoverer = new org.springframework.kafka.listener.DeadLetterPublishi
ngRecoverer(template,
        (record, ex) -> new org.apache.kafka.common.TopicPartition(record.top
ic() + ".DLT", record.partition()));

    var backoff = new org.springframework.util.backoff.FixedBackOff(2000L, 5
L);

    var handler = new org.springframework.kafka.listener.DefaultErrorHandler
(recoverer, backoff);

    handler.addNotRetryableExceptions(
        NonRetryableBusinessException.class,
        org.springframework.kafka.support.serializer.DeserializationException.
class
    );

    return handler;
  }
}
```

Notes:

- `FixedBackOff(2000, 5)` = wait 2s, retry 5 times, then recover (DLT).
- Non-retryable exceptions go directly to recoverer.

Throwing retryable exception in listener

```
@KafkaListener(topics = "orders.events", groupId = "orders-service")
public void onMessage(OrderCreatedEvent event) {
  if (/* transient condition */) {
    throw new RetryableBusinessException("Temporary failure, please retry");
  }

  if (/* invalid data */) {
    throw new NonRetryableBusinessException("Invalid event payload");
```

```
  }

  // process
}
```

Why consumer idempotency still matters

Even with producer idempotence:

- consumer retries
- rebalances
- manual replays

can cause the same event to be processed multiple times.

Next: Section 14 covers idempotent consumers.

---

notes/13-consumer-groups.md

13. Kafka Consumer: Multiple Consumers in a Consumer Group

Question bank

- What is a consumer group and what problem does it solve?
- Why is the partition the unit of parallelism?
- What triggers a rebalance?
- What happens when you have more consumers than partitions?
- What is consumer lag and how do you monitor it?
- What are the risks of long processing times in a consumer?

Core concepts

1) Consumer group basics

A **consumer group** is a set of consumers that cooperate to read a topic.

Rules:

- Each partition is assigned to **at most one consumer** within a group.
- Different groups can read the same topic independently (fan-out).

Consequences:

- If your topic has `P` partitions, max parallelism for one group is `P` consumers.

---

2) Partition assignment and rebalancing

Partition assignment

Kafka assigns partitions to consumers in a group using an assignor strategy.

High level:

- A coordinator broker tracks group membership.
- When membership changes, it triggers a rebalance and partitions are reassigned.

Rebalancing triggers

- consumer instance starts or stops
- consumer becomes unresponsive (misses heartbeats)
- partition count changes (topic is expanded)
- group metadata changes

Why rebalances matter

During rebalance:

- consumers may pause
- partitions may move between consumers
- in-flight processing may be interrupted (depending on your design)

---

3) More consumers than partitions

If consumers > partitions:

- extra consumers are idle

That's not harmful, but it doesn't increase throughput.

---

4) Monitoring lag

Lag is roughly:

- `log end offset - committed offset`

Use CLI:

```
bin/kafka-consumer-groups.sh --bootstrap-server localhost:9091 --describe --g
roup orders-service
```

Interpretation:

- High lag can indicate slow processing, downstream slowness, or insufficient partitions.

Spring Boot notes

Scaling consumers

Scale by:

- running multiple instances of the same service with same `group.id`
- or increasing concurrency

Example:

```yaml
spring:
  kafka:
    listener:
      concurrency: 3
```

Caveat:

- Concurrency > partitions gives no benefit.

Practical lab

1) Create a topic with 3 partitions

```
bin/kafka-topics.sh --bootstrap-server localhost:9091 --create \
  --topic orders.events \
  --partitions 3 \
  --replication-factor 1
```

2) Start 2 consumers in the same group

```
bin/kafka-console-consumer.sh --bootstrap-server localhost:9091 --topic order
s.events --group g1
bin/kafka-console-consumer.sh --bootstrap-server localhost:9091 --topic order
s.events --group g1
```

3) Produce keyed messages

```
bin/kafka-console-producer.sh --bootstrap-server localhost:9091 \
  --topic orders.events --property parse.key=true --property key.separator=:

# type:
# o1:created
# o2:created
```

Observe:

- the same key consistently maps to the same partition
- each partition is consumed by only one consumer in the group

---

Pros / Cons / pitfalls

Pros

- Horizontal scaling with consumer groups
- Parallelism by partitions

Cons / pitfalls

- Rebalances can interrupt processing
- Long processing time without tuning can cause rebalances (heartbeat/session timeouts)
- Scaling requires partition planning

---

notes/14-idempotent-consumer.md

14. Idempotent Consumer in Kafka (Spring Boot)

Question bank

- Why do consumers need idempotency even if producers are idempotent?
- What is the difference between deduplication and exactly-once processing?
- Where should a unique message id live (payload vs headers)?
- What storage can you use for dedupe (DB table, Redis, compacted topic)?
- How do you avoid race conditions in “check then insert”?

Why consumer idempotency is needed

Even with idempotent producers, duplicates can still happen because:

- consumer retries
- rebalances (record may be reprocessed after restart)
- manual replay (seek to older offsets)
- multiple downstream side effects

Kafka commonly gives **at-least-once delivery**. Idempotent consumer design makes duplicates safe.

Core pattern: unique message id + processed-message store

Step 1: include a unique id

Options:

- `eventId` field in payload (recommended)
- or Kafka header like `X-Event-Id`

If you own the schema, payload id is easiest.

Step 2: store processed ids

A common approach:

- table `processed_message`
  - `event_id` (unique)
  - `processed_at`

Step 3: atomic insert

Avoid race conditions:

- perform an insert with a UNIQUE constraint
- if insert succeeds -> process
- if insert fails (duplicate) -> skip

Spring Boot + JPA example (conceptual)

Dependencies

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
```

```
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```

Entity

```
import jakarta.persistence.*;

@Entity
@Table(name = "processed_message", uniqueConstraints = {
    @UniqueConstraint(name = "uk_processed_message_event_id", columnNames = "
eventId")
})
public class ProcessedMessage {

  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  @Column(nullable = false)
  private String eventId;

  @Column(nullable = false)
  private java.time.Instant processedAt;

  protected ProcessedMessage() {}

  public ProcessedMessage(String eventId) {
    this.eventId = eventId;
    this.processedAt = java.time.Instant.now();
  }
}
```

Repository

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface ProcessedMessageRepository extends JpaRepository<ProcessedMe
ssage, Long> {
}
```

Listener logic (dedupe)

```
@Service
public class OrderEventsListener {

  private final ProcessedMessageRepository repo;

  public OrderEventsListener(ProcessedMessageRepository repo) {
```

```
    this.repo = repo;
  }

  @KafkaListener(topics = "orders.events", groupId = "orders-service")
  @org.springframework.transaction.annotation.Transactional
  public void onMessage(OrderCreatedEvent event) {
    try {
      repo.saveAndFlush(new ProcessedMessage(event.eventId()));
    } catch (org.springframework.dao.DataIntegrityViolationException dup) {
      // already processed
      return;
    }

    // perform side effect(s) exactly once from your perspective
  }
}
```

Notes:

- This pattern is simple and effective.
- Make sure DB commit semantics align with when you commit Kafka offsets (see Section 16 for coordination).

Pros / Cons / pitfalls

Pros

- Makes retries and replays safe
- Simple to reason about

Cons / pitfalls

- Requires storage and cleanup strategy
- Adds DB write per message (optimize with batching/async if needed)
- If you do side effects before storing id, you can still duplicate side effects

---

notes/15-kafka-transactions.md

15. Apache Kafka Transactions (Spring Boot)

Question bank

- What does “exactly-once semantics” mean in Kafka?
- What does `transactional.id` do?

- How do transactions differ from idempotent producer?
- What does `isolation.level=read_committed` do for consumers?
- What are common use cases: produce to multiple topics atomically, consume-transform-produce?

Concepts

1) Idempotence vs Transactions

- **Idempotent producer**: prevents duplicates caused by retries for a single partition.
- **Transactions**: allow a producer to atomically write to multiple partitions/topics and commit/abort as a unit.

Transactions are the building block for Kafka's exactly-once processing model in pipelines.

---

2) `transactional.id`

A transactional producer must have a stable identifier:

- `transactional.id`

This enables:

- tracking producer state across restarts
- fencing old instances (prevents split-brain producers)

---

3) Consumer `read_committed`

If producers use transactions, consumers have two isolation options:

- `read_uncommitted` (default): may read uncommitted/aborted records
- `read_committed`: only reads committed transaction data

Set via:

```yaml
spring:
  kafka:
    consumer:
      properties:
        isolation.level: read_committed
```

Spring Boot configuration

Producer properties

```yaml
spring:
  kafka:
    producer:
      properties:
        transactional.id: orders-tx-producer-1
```

Option B: explicit ProducerFactory

```java
@Configuration
public class KafkaTxProducerConfig {

  @Bean
  public org.springframework.kafka.core.ProducerFactory<String, OrderCreatedE
vent> producerFactory(
      org.springframework.boot.autoconfigure.kafka.KafkaProperties kafkaPrope
rties
  ) {
    var props = new java.util.HashMap<String, Object>(kafkaProperties.buildPr
oducerProperties());
    props.put(org.apache.kafka.clients.producer.ProducerConfig.TRANSACTIONAL_
ID_CONFIG, "orders-tx-producer-1");

    var pf = new org.springframework.kafka.core.DefaultKafkaProducerFactory<S
tring, OrderCreatedEvent>(props);
    pf.setTransactionIdPrefix("orders-tx-");
    return pf;
  }
}
```

Notes:

- `setTransactionIdPrefix` is a Spring-friendly approach when you run multiple instances.

Using KafkaTemplate transactions

Local Kafka-only transaction

```java
kafkaTemplate.executeInTransaction(kt -> {
  kt.send("orders.events", key, event);
  kt.send("orders.audit", key, audit);
  return true;
});
```

If an exception is thrown, the transaction is aborted.

`@Transactional` with Kafka

If you configure a Kafka transaction manager, you can use `@Transactional` so that Kafka sends participate.

Important:

- This is Kafka transaction, not database transaction (unless coordinated; see Section 16).

Pros / Cons / pitfalls

Pros

- Atomic writes across multiple topics/partitions
- Enables exactly-once processing patterns (with proper consumer settings)

Cons / pitfalls

- More configuration complexity
- Requires consumer isolation settings for correctness
- Doesn't automatically make external side effects exactly-once (DB writes, HTTP calls)

---

notes/16-kafka-db-transactions.md

16. Kafka and Database Transactions

Question bank

- Why is "Kafka + DB atomicity" difficult?
- What is the dual-write problem?
- What is the outbox pattern and why is it popular?
- How does a chained transaction manager work (and what are its limits)?
- What is the safest way to achieve consistency between DB and Kafka?

The core problem: dual writes

In many services you need to:

1) write to database
2) publish Kafka event

If you do these separately, you can end up with:

- DB write succeeded, Kafka publish failed
- Kafka publish succeeded, DB write failed

This breaks consistency.

Common approaches

1) Outbox pattern (recommended)

Idea:

- write business data + an **outbox row** in the same DB transaction
- a separate process (or Debezium CDC) publishes outbox rows to Kafka

Pros:

- strong consistency with the database transaction
- resilient to Kafka downtime

Cons:

- more moving parts (outbox publisher)

2) Best-effort 2-step (simple but weaker)

- write DB
- publish event
- if publish fails, retry later (application logic)

Pros:

- simplest

Cons:

- hard to guarantee no gaps

3) Chained transaction managers (use carefully)

Spring can coordinate multiple transaction managers, but this is not true XA two-phase commit.

This can reduce risk in some cases but is not a universal solution.

Spring Boot example: JPA Tx + Kafka Tx (conceptual)

Create JPA transaction manager

```
@Configuration
public class JpaTxConfig {

  @Bean
  public org.springframework.orm.jpa.JpaTransactionManager transactionManager
(
      jakarta.persistence.EntityManagerFactory emf
  ) {
    return new org.springframework.orm.jpa.JpaTransactionManager(emf);
  }
}
```

Kafka transaction manager

```
@Bean
public org.springframework.kafka.transaction.KafkaTransactionManager<String,
OrderCreatedEvent> kafkaTransactionManager(
    org.springframework.kafka.core.ProducerFactory<String, OrderCreatedEvent>
 producerFactory
) {
  return new org.springframework.kafka.transaction.KafkaTransactionManager<>
(producerFactory);
}
```

Coordinated transaction (chained)

Depending on Spring version, you may use a chained manager.

Even if you chain, keep in mind:

- failures can still leave partial effects

Enable logging for debugging transactions

Useful logging categories:

- `org.springframework.kafka.transaction`
- `org.springframework.transaction`
- `org.hibernate.SQL`

Practical recommendation

- For production-grade “DB state change -> event” reliability: **Outbox pattern**.
- If you keep it simple initially: implement **idempotent consumers** + retries + DLT, and accept eventual consistency.

---

notes/17-integration-testing-producer.md

17. Integration Testing – Kafka Producer (Spring Boot)

Question bank

- What is the difference between unit tests and Kafka integration tests?
- What are the main strategies: Embedded Kafka vs Testcontainers?
- How do you structure tests with Arrange / Act / Assert?
- How do you consume records in tests reliably (polling with timeout)?
- How do you verify producer properties (acks/idempotence) are applied?

Strategy options

1) Embedded Kafka

- Uses Spring Kafka test support to spin up a broker in-process (for tests).

Pros:

- fast
- simple local runs

Cons:

- less “real” than containerized Kafka

2) Testcontainers Kafka

- Runs Kafka in Docker during tests.

Pros:

- closer to production

Cons:

- slower

- requires Docker

Common test annotations

`@SpringBootTest`

Starts the application context.

`@EmbeddedKafka`

Creates an embedded broker.

Example:

```
@SpringBootTest
@org.springframework.kafka.test.context.EmbeddedKafka(partitions = 1, topics
= {"orders.events"})
class OrderProducerIT {
}
```

Arrange / Act / Assert template

Arrange

- configure topic
- prepare producer

Act

- send an event

Assert

- consume from topic and verify payload

Example: consume produced record in test

```
@SpringBootTest
@org.springframework.kafka.test.context.EmbeddedKafka(partitions = 1, topics
= {"orders.events"})
class OrderProducerIT {

  @Autowired
  private org.springframework.kafka.core.KafkaTemplate<String, OrderCreatedEv
ent> template;

  @Test
```

```
  void shouldPublishOrderCreated() {
    template.send("orders.events", "o1", new OrderCreatedEvent("e1", "o1", ja
va.time.Instant.now(),
        new java.math.BigDecimal("10.00"), "INR"));

    var consumerProps = org.springframework.kafka.test.utils.KafkaTestUtils.c
onsumerProps(
        "test-group", "true", (org.springframework.kafka.test.EmbeddedKafkaBr
oker) null);

    // In real code, create a consumer with proper broker reference and poll.
    // The exact setup is verbose; keep a helper method in tests.
  }
}
```

Note:

- In your actual repo, create a small test utility to build a consumer and poll with timeout.

Verifying producer properties

You typically verify indirectly:

- if `enable.idempotence=true`, retries don't duplicate under certain scenarios

Or directly by:

- asserting producer factory configs if you build them explicitly

---

Recommended for your notes

- Use Embedded Kafka for fast learning.
- Use Testcontainers when you want realism and already depend on Docker.

---

notes/18-saga-part-1.md

18. Saga design pattern with Apache Kafka (Part 1)

Question bank

- What is a Saga and why do we need it in microservices?
- What is the difference between choreography-based and orchestration-based saga?

- How do you model commands vs events?
- Where do you store saga state (history table / state machine)?
- How do you ensure idempotency at each step?

Saga overview

A Saga is a pattern for managing **distributed transactions** without 2PC.

Instead of atomic commit across services, you use:

- a sequence of local transactions
- events/commands to move the workflow forward

Choreography vs Orchestration

Choreography

- no central coordinator
- services react to events and emit new events

Pros:

- simple, loosely coupled

Cons:

- complex to reason about at scale
- harder to visualize global flow

Orchestration

- a saga orchestrator sends commands and tracks state

Pros:

- clear control flow
- easier monitoring

Cons:

- coordinator is a critical component

Example domain: Order -> Stock -> Payment -> Approval

Typical messages

Events:

- `OrderCreatedEvent`
- `ProductReservedEvent`
- `ProductReservationFailedEvent`
- `PaymentProcessedEvent`
- `PaymentFailedEvent`
- `OrderApprovedEvent`
- `OrderRejectedEvent`

Commands:

- `ReserveProductCommand`
- `ProcessPaymentCommand`
- `ApproveOrderCommand`
- `RejectOrderCommand`

Part 1: Happy path flow (conceptual)

1) Order Service
- creates order
- publishes `OrderCreatedEvent`

2) Stock Service
- consumes `OrderCreatedEvent`
- reserves stock
- publishes `ProductReservedEvent` OR `ProductReservationFailedEvent`

3) Payment Service
- consumes `ProductReservedEvent`
- processes payment
- publishes `PaymentProcessedEvent` OR `PaymentFailedEvent`

4) Order Service
- consumes `PaymentProcessedEvent`
- publishes `ApproveOrderCommand`
- then `OrderApprovedEvent`

Implementation notes (Kafka)

Correlation id

Add a correlation id across all messages:

- `sagaId`
- `orderId`

Idempotency

Each participant should be idempotent:

- if it receives the same command twice, it should not double-reserve stock or double-charge.

Pros / Cons

Pros

- Works well with event-driven microservices
- Improves resilience

Cons / pitfalls

- Complex failure handling
- Requires strong observability (trace ids)
- Requires idempotent handlers

---

notes/19-saga-part-2-compensation.md

19. Saga Part 2: Compensating Transaction

Question bank

- What is a compensating transaction?
- Why can’t we “rollback” across services?
- How do you model compensation commands?
- What should happen if compensation itself fails?
- How do you handle timeouts (a participant never replies)?

Compensation concept

A compensating transaction is a business action that **undoes** a previous successful step.

Examples:

- cancel stock reservation
- refund a payment
- mark order as rejected

Example: payment failed after stock reserved

Flow:

1) Stock reserved
2) Payment fails
3) Publish `CancelProductReservationCommand`
4) Stock Service releases stock and publishes `ProductReservationCancelledEvent`
5) Order Service marks order rejected

Common Kafka implementation notes

Ensure you can correlate

- use `sagaId` and `orderId` in every message

Idempotency again

Compensation handlers must also be idempotent:

- canceling the same reservation twice should be safe

Timeouts

If no response arrives:

- orchestrator can issue a timeout event
- or a scheduled job can mark saga as failed and trigger compensation

---

Pros / Cons / pitfalls

Pros

- Enables eventual consistency in distributed workflows

Cons / pitfalls

- Compensation logic is domain-specific and can be complex
- Some actions are not easily reversible
- You must handle partial failures in compensation steps too

---

notes/20-kafka-connect.md

20. Kafka Connect

Question bank

- What problems does Kafka Connect solve?
- When should you use Connect vs writing a custom consumer/producer?
- What are workers, connectors, and tasks?
- What are converters and transforms (SMTs)?
- How do you manage connector configs and deployments?

What is Kafka Connect?

Kafka Connect is a framework to:

- ingest data into Kafka from external systems (source connectors)
- push data from Kafka to external systems (sink connectors)

without writing custom integration code.

Core components

1) Workers

- runtime processes that execute connectors
- can run in:
    - standalone mode
    - distributed mode

2) Connectors

- define *what* system to connect and *how*

Examples:

- JDBC source connector
- S3 sink connector

3) Tasks

- a connector splits work into tasks
- tasks enable parallelism

Key components

Transforms (SMTs)

Single Message Transforms can:

- rename fields
- route topics
- add/drop fields

Use cases:

- adjust payload without changing producer

Converters

Converters control how data is represented:

- JSON
- Avro
- Protobuf

In enterprise environments, Avro/Protobuf + schema registry is common.

When and why to use Kafka Connect

Use Connect when:

- you need standard integration (DB, files, S3, Elasticsearch)
- you want managed scaling and retries
- you want configuration-driven integrations

Avoid Connect when:

- business logic is complex and needs custom code
- you need low-level control over processing semantics

Pros / Cons

Pros

- Fast integration without custom code
- Scales via tasks and distributed workers
- Standard operational model

Cons / pitfalls

- Requires operational setup/monitoring
- Connector behavior and offsets must be understood
- Misconfigured converters/transforms can corrupt data

---

notes/spring-kafka-annotations-reference.md

Spring Kafka annotations reference (Spring Boot + Kafka 4)

This document lists the most commonly used **Spring Kafka** annotations (from `org.springframework.kafka.*`) that you will use when building Kafka-based microservices with Spring Boot.

Notes:

- Kafka "4" here refers to the Kafka broker/client generation you run (Kafka 4.x). Your application uses the **Kafka client** version brought by Spring Boot/Spring Kafka.
- Many everyday annotations are standard Spring annotations (e.g., `@Configuration`, `@Bean`, `@Transactional`). I list them here only where they are directly relevant to Kafka patterns.

---

1) `@KafkaListener`

Package: `org.springframework.kafka.annotation.KafkaListener`

Used for: declaring a method as a Kafka message listener.

What it does:

- Creates a message-driven endpoint.
- Under the hood, Spring creates a listener container that polls Kafka and invokes your method.

- Can subscribe to one or more topics, use consumer groups, and apply container factory settings.

Key attributes:

- `topics`: topic names to subscribe
- `topicPattern`: subscribe by regex
- `groupId`: consumer group id (overrides config)
- `containerFactory`: which `ConcurrentKafkaListenerContainerFactory` bean to use
- `id`: listener id (useful for start/stop)
- `concurrency`: number of threads/containers for this listener (bounded by partition count)

Example:

```
@Service
public class OrderEventsListener {

  @KafkaListener(topics = "orders.events", groupId = "orders-service")
  public void onMessage(OrderCreatedEvent event) {
    // business logic
  }
}
```

Pros

- Very simple listener model.
- Integrates with Spring error handling, retries, and container management.

Common pitfalls

- Ordering is per partition: if you need ordering per entity, ensure producer uses a key.
- If you don't configure error handling, poison pills can cause repeated failures.

---

2) `@KafkaListeners`

Package: `org.springframework.kafka.annotation.KafkaListeners`

Used for: grouping multiple `@KafkaListener` annotations on the same method/class.

What it does:

- Container annotation that allows multiple listener definitions.

Example (two topics):

```java
@KafkaListeners({
    @KafkaListener(topics = "orders.events", groupId = "orders-service"),
    @KafkaListener(topics = "orders.audit", groupId = "orders-service")
})
public void onAnyMessage(String payload) {
}
```

---

3) `@KafkaHandler`

Package: `org.springframework.kafka.annotation.KafkaHandler`

Used for: method-level dispatch when using a **class-level** `@KafkaListener`.

What it does:

- Lets one listener class handle multiple payload types by routing to the correct handler method.

Example:

```java
@Service
@KafkaListener(topics = "orders.events", groupId = "orders-service")
public class OrdersPolymorphicListener {

  @KafkaHandler
  public void handle(OrderCreatedEvent event) {}

  @KafkaHandler
  public void handle(OrderCancelledEvent event) {}

  @KafkaHandler(isDefault = true)
  public void handleDefault(Object unknown) {}
}
```

Pitfalls

- Requires consistent serialization and type mapping strategy. If you disable type headers, you typically rely on topic-per-event-type or custom headers.

---

4) `@TopicPartition`

Package: `org.springframework.kafka.annotation.TopicPartition`

Used for: explicitly assigning specific partitions to a listener.

What it does:

- Instead of subscribing to the whole topic, you can pin a listener to particular partitions.

Example:

```
@KafkaListener(topicPartitions = {
    @TopicPartition(topic = "orders.events", partitions = {"0"})
})
public void listenPartition0(String value) {}
```

When to use

- Debugging / special processing.
- Not typical for scalable consumer-group patterns.

---

5) `@PartitionOffset`

Package: `org.springframework.kafka.annotation.PartitionOffset`

Used for: specifying starting offsets when using `@TopicPartition`.

What it does:

- Allows you to start at a given offset (or use `relativeToCurrent`).

Example:

```
@KafkaListener(topicPartitions = {
    @TopicPartition(
        topic = "orders.events",
        partitionOffsets = @PartitionOffset(partition = "0", initialOffset =
"100")
    )
})
public void replayFromOffset(String value) {}
```

---

6) `@KafkaKey`

Package: `org.springframework.kafka.annotation.KafkaKey`

Used for: marking a method parameter as the Kafka record key.

What it does:

- Useful when your listener method has multiple parameters and you want explicit key injection.

Example:

```java
@KafkaListener(topics = "orders.events", groupId = "orders-service")
public void onMessage(@KafkaKey String key, OrderCreatedEvent event) {
}
```

---

7) `@SendTo`

Package: `org.springframework.messaging.handler.annotation.SendTo`

Used for: sending a reply from a listener method to a Kafka topic.

What it does:

- Integrates request/reply messaging patterns.
- Often used together with `@KafkaListener` in request/reply flows.

Example (reply topic):

```java
@KafkaListener(topics = "orders.requests", groupId = "orders-service")
@SendTo("orders.replies")
public OrderReply handle(OrderRequest request) {
  return new OrderReply("OK");
}
```

Pitfalls

- Request/reply over Kafka is valid but adds complexity; many architectures prefer event + separate reply event.

---

8) `@EnableKafka`

Package: `org.springframework.kafka.annotation.EnableKafka`

Used for: enabling detection/processing of Kafka listener annotations.

What it does:

- Imports Kafka listener infrastructure.

Do you need it?

- In many Spring Boot apps, it may not be necessary because auto-configuration wires things.
- If you build a custom configuration or disable auto-config, adding `@EnableKafka` is common.

Example:

```
@Configuration
@EnableKafka
public class KafkaConfig {
}
```

---

9) `@EnableKafkaStreams`

Package: `org.springframework.kafka.annotation.EnableKafkaStreams`

Used for: enabling Kafka Streams infrastructure in Spring.

What it does:

- Creates and manages `KafkaStreams` instances based on your streams configuration.

Example (high level):

```
@Configuration
@EnableKafkaStreams
public class KafkaStreamsConfig {
}
```

---

10) `@KafkaListenerContainerFactory` (not an annotation)

You'll frequently see `containerFactory = "kafkaListenerContainerFactory"` referenced in `@KafkaListener`.

That factory is usually a `ConcurrentKafkaListenerContainerFactory<K, V>` bean.

---

11) Transaction-related annotations (commonly used with Kafka)

`@Transactional`

Package: `org.springframework.transaction.annotation.Transactional`

Used for: transaction boundaries in Spring.

Kafka-specific notes:

- If you configure a `KafkaTransactionManager` (or chained transaction manager), `@Transactional` can control Kafka sends done within that transaction.
- For consumer processing, you often combine manual ack, retries, and DB writes with transactional boundaries.

---

12) Producer-side helper annotations (often confused as “Kafka annotations”)

These are not from Spring Kafka specifically, but are very commonly used:

- `@Configuration` / `@Bean` (define `ProducerFactory`, `ConsumerFactory`, `KafkaTemplate`)
- `@Service` (producer/consumer service)
- `@RestController` (producer entrypoint)
- `@Header` (read Kafka headers in listener methods)

Example header usage:

```
@KafkaListener(topics = "orders.events", groupId = "orders-service")
public void onMessage(
    OrderCreatedEvent event,
    @org.springframework.messaging.handler.annotation.Header(org.springframework.kafka.support.KafkaHeaders.OFFSET) long offset
) {
}
```

Summary: “must know” annotations

- `@KafkaListener` (core consumer)
- `@KafkaHandler` (multi-type dispatch)
- `@TopicPartition` / `@PartitionOffset` (explicit assignment and replay)
- `@KafkaKey` (inject key cleanly)
- `@SendTo` (reply topic pattern)
- `@EnableKafka` (enable listener infra, if needed)
- `@EnableKafkaStreams` (Kafka Streams)

---

interview/README.md

Kafka Interview Questions (10+ Years) — Product Companies

This folder contains **senior-level Kafka interview questions** suitable for product-based companies (Amazon/Google/etc.) and service-based companies with high-scale systems.

Each section includes:

- Questions (increasing difficulty)
- What the interviewer is looking for (key points)
- Follow-ups (how they probe deeper)

Index

---

interview/00-diagrams.md

00. Kafka Diagrams (Quick Interview Reference)

Use these diagrams when you answer interview questions (topic/partition, brokers, leader-follower, consumer groups).

---

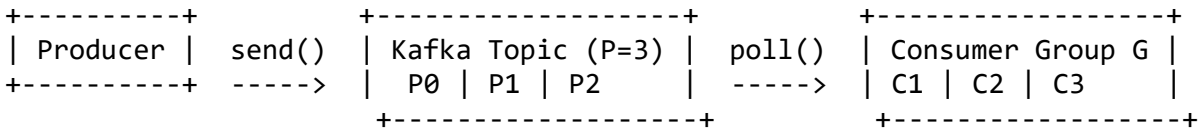
1) Topic and partitions (ordering scope)

```
Topic: orders.events

+---------------- Partition 0 ----------------+
| offset 0 | offset 1 | offset 2 | offset 3 |
+-------------------------------------------+

+---------------- Partition 1 ----------------+
| offset 0 | offset 1 | offset 2 | offset 3 |
+-------------------------------------------+

+---------------- Partition 2 ----------------+
| offset 0 | offset 1 | offset 2 | offset 3 |
+-------------------------------------------+
```

```
Ordering guarantee:
- Strong ordering ONLY within a single partition.
- If key=orderId, all events for that orderId go to the same partition => per
-order ordering.
```

---

2) Producer -> Topic -> Consumer group (high level)




```
Rules:
- In a consumer group, each partition is assigned to at most one consumer.
- Parallelism = number of partitions.
```

---

3) 3-broker cluster + partition leaders/followers (replication)

Example: topic with RF=3 and partitions=3

```
Brokers:
  [Broker-1]      [Broker-2]      [Broker-3]

Partition 0 replicas:
  Leader:   Broker-1
  Follower: Broker-2
  Follower: Broker-3

Partition 1 replicas:
  Leader:   Broker-2
  Follower: Broker-3
  Follower: Broker-1

Partition 2 replicas:
  Leader:   Broker-3
  Follower: Broker-1
  Follower: Broker-2

ISR (in-sync replicas) is the set of replicas that are fully caught up.
```

---

4) Consumer group rebalancing (assignment changes)

```
Topic has 4 partitions: P0 P1 P2 P3
```

```
Before (2 consumers):
  C1 -> P0, P1
  C2 -> P2, P3

After scaling up (3 consumers):
  C1 -> P0
  C2 -> P1
  C3 -> P2, P3

Rebalance triggers:
- consumer joins/leaves
- consumer stops heartbeating
- partition count change
```

---

5) KRaft (very high level)

```
                (metadata quorum)
+-----------------------------------------------+
| Controllers (KRaft): C1, C2, C3               |
| - cluster metadata                            |
| - partition leadership changes                |
+-----------------------------------------------+
                  ^
                  |
                  v
+-----------------------------------------------+
| Brokers: B1, B2, B3                           |
| - store partitions/log segments               |
| - serve produce/fetch                         |
+-----------------------------------------------+

In small labs, a node can be both broker and controller.
In larger setups, roles can be separated.
```

---

interview/01-system-design-architecture.md

01. System Design & Architecture (10+ Years)

1) Event-driven architecture choices

- **Question**: When do you choose Kafka over synchronous APIs (REST/gRPC) for microservice communication?
    - **Looking for**:

        - coupling (availability/latency)
        - buffering/backpressure
        - fan-out, replay
        - operational cost and observability
    - **Follow-ups**:
        - how do you handle request/reply if needed?
        - how do you ensure eventual consistency correctness?
- **Question**: Design an event-driven order pipeline (Order → Inventory → Payment → Shipping) with Kafka.
    - **Looking for**:
        - clear topic design (commands vs events)
        - keys (orderId) for ordering
        - consumer groups per service
        - retry + DLT strategy
        - idempotency for each participant
        - correlation IDs / tracing
- **Question**: How do you decide between choreography vs orchestration saga?
    - **Looking for**:
        - complexity/ownership boundaries
        - observability and control flow
        - failure handling and compensation

---

2) Topic and partition strategy

- **Question**: How do you decide partition count for a topic?
    - **Looking for**:
        - throughput scaling (partition = parallelism unit)
        - consumer parallelism requirements
        - future growth and rebalancing overhead
        - resource costs (file handles, memory)
- **Question**: How would you design topics for multi-tenant SaaS (1000+ tenants)?
    - **Looking for**:
        - topic-per-tenant vs shared topic
        - keying strategy (tenantId)
        - quotas, noisy neighbor, isolation
        - ACLs/security
- **Question**: How do you manage topic lifecycle and config drift across envs?
    - **Looking for**:
        - GitOps / IaC for topics

        - explicit configs (retention, cleanup policy)
        - automation and review process

---

3) Schema and compatibility

- **Question**: How do you version events and keep backward/forward compatibility?
    - **Looking for**:
        - schema registry concepts (even if not used)
        - additive changes vs breaking changes
        - default values, optional fields
        - topic strategy when breaking changes occur
- **Question**: How do you roll out a breaking schema change with minimal risk?
    - **Looking for**:
        - dual-publish / dual-consume strategies
        - new topic versioning (`orders.events.v2`)
        - gradual rollout and monitoring

---

4) Data correctness and business invariants

- **Question**: You have exactly-once *business requirement* (charge card once). What architecture do you propose?
    - **Looking for**:
        - distinguishing Kafka EOS vs end-to-end exactly-once
        - idempotency keys
        - database constraints
        - outbox pattern
- **Question**: Design for replay. How do you safely reprocess historical events?
    - **Looking for**:
        - consumer group strategy for replay
        - side effect isolation
        - idempotent consumer
        - backfill topics and throttling

---

interview/02-kafka-internals.md

02. Kafka Internals & Guarantees

For quick diagrams you can use in interviews, see: `interview/00-diagrams.md`.

1) Fundamentals interviewers expect

- **Question**: Explain topic, partition, segment, offset, consumer group.
  - **Looking for**:
    - ordering per partition
    - offset is position in partition
    - segments are on-disk chunks

Diagram: topic and partitions

```
---------------- Partition 0 ----------------+
 offset 0 | offset 1 | offset 2 | offset 3
-------------------------------------------+

---------------- Partition 1 ----------------+
 offset 0 | offset 1 | offset 2 | offset 3
-------------------------------------------+

Ordering is per partition.
```

- **Question**: How does Kafka store data on disk?
  - **Looking for**:
    - append-only log segments
    - index files
    - retention and cleanup

---

2) Replication and ISR

- **Question**: What is a partition leader and follower? What is ISR?
  - **Looking for**:
    - leader handles reads/writes
    - followers replicate
    - ISR = in-sync replicas, required for durability

Diagram: leader/follower replication (RF=3)

```
Brokers:  [B1]   [B2]   [B3]

Partition 0 replicas:
```

```
  Leader:   B1
  Follower: B2
  Follower: B3

Partition 1 replicas:
  Leader:   B2
  Follower: B3
  Follower: B1
```

- **Question**: What happens if the leader broker dies?
  - **Looking for**:
    - leader election among ISR
    - client metadata refresh
    - impact on producers (timeouts/retries)
- **Question**: What is the relationship between `acks=all` and `min.insync.replicas`?
  - **Looking for**:
    - write succeeds only if enough ISR
    - otherwise `NotEnoughReplicas`-type failures

---

3) KRaft basics (Kafka 4 era)

- **Question**: What is KRaft? What replaces ZooKeeper?
  - **Looking for**:
    - metadata quorum (controller quorum)
    - `controller.quorum.voters`
    - controllers manage metadata/logical cluster state
- **Question**: What is the controller's role?
  - **Looking for**:
    - partition leadership, metadata changes
    - cluster membership

---

4) Consumer internals

- **Question**: How does consumer group coordination work at a high level?
  - **Looking for**:
    - coordinator
    - heartbeats, session timeout
    - rebalancing
- **Question**: What causes rebalance storms? How do you mitigate?
  - **Looking for**:
    - slow processing, long GC, tight timeouts

        - cooperative rebalancing mention (optional)
        - tuning and stable deployments

---

5) Exactly-once semantics (EOS) vs reality

- **Question**: What does Kafka EOS guarantee?
    - **Looking for**:
        - transactional producer + read_committed
        - consume-transform-produce pipelines
        - not a magic guarantee for external DB side effects

---

interview/03-reliability-eos.md

03. Reliability: Ordering, Retries, Idempotency, EOS

1) Delivery semantics

- **Question**: Explain at-most-once vs at-least-once vs exactly-once.
    - **Looking for**:
        - where duplicates come from
        - offset commit timing
        - idempotency patterns
- **Question**: In Kafka, what is the default semantics you usually get and why?
    - **Looking for**:
        - at-least-once typical with retries

---

2) Ordering

- **Question**: How do you guarantee ordering for a business entity?
    - **Looking for**:
        - key by entityId so all events go to same partition
- **Question**: What breaks ordering?
    - **Looking for**:
        - multiple partitions
        - producing without keys
        - reprocessing assumptions

---

3) Retries and duplicates

- **Question**: When can producer retries cause duplicates?
  - **Looking for**:
    - ambiguous failures (timeout after write)
- **Question**: How does idempotent producer help?
  - **Looking for**:
    - PID + sequence numbers
- **Question**: How do you design an idempotent consumer?
  - **Looking for**:
    - eventId + DB unique constraint
    - atomic check/insert

---

4) DLT and poison pills

- **Question**: Describe a robust error-handling strategy in consumers.
  - **Looking for**:
    - classification (retryable vs non)
    - backoff, max attempts
    - DLT publishing
    - alerting and remediation workflow

---

5) Transactions

- **Question**: When do you need Kafka transactions?
  - **Looking for**:
    - atomic multi-topic writes
    - consume-transform-produce
- **Question**: Why doesn't Kafka transaction automatically solve DB+Kafka atomicity?
  - **Looking for**:
    - external side effects
    - outbox pattern

---

interview/04-operations-performance.md

04. Operations & Performance Tuning

1) Performance tuning

- **Question**: How do you tune producer throughput?
  - **Looking for**:
    - batching (linger.ms, batch.size)
    - compression
    - partitioning
    - async IO, acks tradeoffs
- **Question**: How do you tune consumer throughput?
  - **Looking for**:
    - parallelism (partitions, concurrency)
    - max.poll.records
    - avoid long processing inside poll loop

---

2) Capacity planning

- **Question**: Given X MB/s ingress and retention requirements, how do you size disk and brokers?
  - **Looking for**:
    - replication factor multiplier
    - headroom
    - compaction vs delete retention
- **Question**: How do you decide replication factor?
  - **Looking for**:
    - durability/availability goals
    - ISR behavior

---

3) Monitoring

- **Question**: What metrics do you monitor in Kafka?
  - **Looking for**:
    - consumer lag
    - under-replicated partitions
    - request latency
    - controller health

        - disk usage
- **Question**: What alerts would you configure first?
    - **Looking for**:
        - URP, ISR shrink
        - disk full
        - lag spikes

---

4) Security

- **Question**: How do you secure Kafka in production?
    - **Looking for**:
        - TLS, SASL
        - ACLs
        - secret management

---

5) Disaster scenarios

- **Question**: Broker lost, disk corrupted—what do you do?
    - **Looking for**:
        - replication recovery
        - reassignment
        - operational runbook

---

interview/05-spring-kafka.md

05. Spring Kafka / Spring Boot (Practical)

1) Core APIs

- **Question**: Explain `KafkaTemplate` vs raw Kafka producer client.
    - **Looking for**:
        - abstraction benefits
        - async send + callbacks
- **Question**: Explain `@KafkaListener` and how it works internally.
    - **Looking for**:
        - listener container polls
        - deserialization
        - error handling integration

---

2) Configuration and common issues

- **Question**: How do you configure JSON serialization safely?
  - **Looking for**:
    - JsonSerializer/JsonDeserializer
    - trusted packages
    - schema/version strategy
- **Question**: Why do deserialization errors break consumers and how do you fix?
  - **Looking for**:
    - ErrorHandlingDeserializer
    - DLT

---

3) Acks, retries, idempotence

- **Question**: How do you configure producer acks/retries in Spring?
  - **Looking for**:
    - application.yml vs explicit beans
- **Question**: How do you enable idempotent producer?
  - **Looking for**:
    - enable.idempotence
    - why it matters

---

4) Listener reliability patterns

- **Question**: How do you implement retry + backoff + DLT in Spring Kafka?
  - **Looking for**:
    - DefaultErrorHandler
    - DeadLetterPublishingRecoverer
    - classifying exceptions
- **Question**: Offsets and commits—how do you ensure you don't lose messages?
  - **Looking for**:
    - ack modes
    - manual acks vs auto

---

5) Transactions

- **Question**: How do you use Kafka transactions in Spring?
  - **Looking for**:

        - transactional.id
        - KafkaTransactionManager
        - executeInTransaction

---

interview/06-scenarios-incidents.md

06. Scenario / Debugging / Incident Questions

These are common Staff/Senior real-world prompts.

1) Lag and throughput

- **Scenario**: Consumer lag is growing continuously. What do you check first?
    - **Looking for**:
        - downstream dependency slowness
        - partition count vs consumer count
        - max.poll.records / processing time
        - rebalances
        - broker health
- **Scenario**: Producer throughput dropped by 80% after enabling `acks=all`.
    - **Looking for**:
        - ISR state, URP
        - network/disk bottleneck
        - batching/compression adjustments

---

2) Data correctness

- **Scenario**: You see duplicates in downstream DB.
    - **Looking for**:
        - confirm delivery semantics
        - consumer retry/rebalance
        - add idempotency keys and unique constraints
- **Scenario**: Events are out-of-order for the same orderId.
    - **Looking for**:
        - missing/incorrect key
        - multiple topics
        - partition changes

---

3) Connectivity failures (Docker / prod)

- **Scenario**: Client connects to broker but then fails to produce with “connection refused” to a different host.
    - **Looking for**:
        - advertised.listeners misconfig

---

4) Poison pills

- **Scenario**: One bad message causes consumer restart loop.
    - **Looking for**:
        - ErrorHandlingDeserializer
        - DLT
        - skip policy with audit

---

5) Rebalance storms

- **Scenario**: Group rebalances every few minutes.
    - **Looking for**:
        - long processing, heartbeat timeouts
        - GC pauses
        - cooperative assignor mention

---

interview/07-coding-takehome.md

07. Coding / Take-home Style Questions

1) Producer

- **Task**: Build a Spring Boot REST endpoint that publishes `OrderCreatedEvent` to Kafka with key = orderId.
    - **Evaluation**:
        - correct serialization
        - async send with callbacks
        - basic error handling
- **Task**: Log record metadata (partition/offset) and correlation id.
    - **Evaluation**:
        - observability mindset

---

2) Consumer

- **Task**: Create a consumer that writes to DB and is idempotent.
  - **Evaluation**:
    - unique constraint and proper transaction boundary
- **Task**: Implement retryable vs non-retryable exceptions and DLT.
  - **Evaluation**:
    - DefaultErrorHandler usage

---

3) Replay / backfill

- **Task**: Implement a backfill consumer group that replays from earliest and writes to a new topic.
  - **Evaluation**:
    - safe replay design
    - throttling

---

4) System design coding hybrid

- **Task**: Implement outbox publisher (DB table -> Kafka).
  - **Evaluation**:
    - correctness under failure
    - batching and retries
    - exactly-once vs at-least-once discussion

---

interview/08-leadership-tradeoffs.md

08. Leadership / Ownership / Tradeoffs (Staff+)

1) Architecture ownership

- **Question**: Describe a Kafka platform you owned end-to-end. What were the biggest lessons?
  - **Looking for**:
    - operational maturity
    - monitoring and runbooks
    - incident handling
- **Question**: How do you drive adoption of event-driven architecture across teams?
  - **Looking for**:

        - standards (naming, schemas)
        - governance without blocking
        - self-service tooling

---

2) Handling incidents

- **Question**: Describe a major Kafka incident and how you resolved it.
    - **Looking for**:
        - hypotheses and debugging
        - communication
        - mitigation and long-term fixes

---

3) Tradeoffs and decision making

- **Question**: Topic-per-event vs shared topic: how do you decide?
    - **Looking for**:
        - schema evolution, routing, consumer complexity
- **Question**: Outbox vs direct publish: when do you mandate outbox?
    - **Looking for**:
        - criticality, compliance
- **Question**: How do you ensure privacy/PII compliance with Kafka?
    - **Looking for**:
        - encryption, access control, retention, data deletion strategy

---

4) Mentoring and standards

- **Question**: What code review checklist do you use for Kafka producers/consumers?
    - **Looking for**:
        - keys/ordering
        - retries/timeouts
        - DLT and observability
        - idempotency
        - schema compatibility

Kafka Udemy Notes

Kafka CLI Cheat Sheet (macOS)

Assumptions:
- You are in Kafka home (folder that contains `bin/`).
- Commands use the Kafka shell scripts (`*.sh`).
- You have a broker reachable on `localhost:9092`.

0) Quick setup

Set a reusable bootstrap variable:

```bash
export BS=localhost:9092
# Multi-broker example:
# export BS=localhost:9092,localhost:9094
```

---

1) Start/Stop Kafka (KRaft)

Start brokers (example):

```bash
./bin/kafka-server-start.sh config/server-1.properties
./bin/kafka-server-start.sh config/server-2.properties
./bin/kafka-server-start.sh config/server-3.properties
```

Stop brokers (best-effort):

```bash
./bin/kafka-server-stop.sh
```

---

2) Topics

List topics

```bash
./bin/kafka-topics.sh --bootstrap-server $BS --list
```

Describe a topic (partitions, leader, replicas)

```bash
./bin/kafka-topics.sh --bootstrap-server $BS --describe --topic product-created-events-topic
```

Create a topic

Example: 3 partitions, replication factor 1

```bash
./bin/kafka-topics.sh --bootstrap-server $BS \
--create --topic product-created-events-topic \
--partitions 3 --replication-factor 1
```

Create a DLT topic (match partitions)

If your main topic has 3 partitions, create the DLT with 3 partitions too:

```bash
./bin/kafka-topics.sh --bootstrap-server $BS \
--create --topic product-created-events-topic-dlt \
--partitions 3 --replication-factor 1
```

Alter partitions (only increase)

```bash
./bin/kafka-topics.sh --bootstrap-server $BS \
```

--alter --topic product-created-events-topic-dlt --partitions 3
```

### Delete a topic

```bash
./bin/kafka-topics.sh --bootstrap-server $BS --delete --topic product-created-events-topic
```

Notes:
- Topic deletion can be controlled by broker config `delete.topic.enable=true`.

---

3) Produce messages

Produce values

```bash
./bin/kafka-console-producer.sh --bootstrap-server $BS --topic product-created-events-topic
```

Produce with keys (`key:value`)

```bash
./bin/kafka-console-producer.sh --bootstrap-server $BS \
--topic product-created-events-topic \
--property parse.key=true \
--property key.separator=:
```

Example input:

```text
user1:{"productId":"1","title":"iPhone","price":1500,"quantity":1}
```

```

4) Consume messages

Consume from latest

```bash
./bin/kafka-console-consumer.sh --bootstrap-server $BS --topic product-created-events-topic
```

Consume from beginning

```bash
./bin/kafka-console-consumer.sh --bootstrap-server $BS \
--topic product-created-events-topic --from-beginning
```

Print key/partition/offset

```bash
./bin/kafka-console-consumer.sh --bootstrap-server $BS \
--topic product-created-events-topic --from-beginning \
--property print.key=true \
--property print.partition=true \
--property print.offset=true \
--property print.value=true
```

Consume a DLT

```bash
./bin/kafka-console-consumer.sh --bootstrap-server $BS \
--topic product-created-events-topic-dlt --from-beginning \
--property print.key=true --property print.value=true
```

---

5) Consumer groups

List groups

```bash
./bin/kafka-consumer-groups.sh --bootstrap-server $BS --list
```

Describe a group (lag, offsets)

```bash
./bin/kafka-consumer-groups.sh --bootstrap-server $BS \
--describe --group product-created-events
```

Reset offsets (reprocess)

Dry run:

```bash
./bin/kafka-consumer-groups.sh --bootstrap-server $BS \
--group product-created-events \
--topic product-created-events-topic \
--reset-offsets --to-earliest --dry-run
```

Execute:

```bash
./bin/kafka-consumer-groups.sh --bootstrap-server $BS \
--group product-created-events \
--topic product-created-events-topic \
--reset-offsets --to-earliest --execute
```

---

6) Topic configuration (retention, etc.)

Describe topic configs

```bash
./bin/kafka-configs.sh --bootstrap-server $BS \
```

```
--entity-type topics --entity-name product-created-events-topic \
--describe
```

Set retention (example: 1 hour)

```bash
./bin/kafka-configs.sh --bootstrap-server $BS \
--entity-type topics --entity-name product-created-events-topic \
--alter --add-config retention.ms=3600000
```

Remove an override

```bash
./bin/kafka-configs.sh --bootstrap-server $BS \
--entity-type topics --entity-name product-created-events-topic \
--alter --delete-config retention.ms
```

---

7) Useful debug commands

Check broker API versions

```bash
./bin/kafka-broker-api-versions.sh --bootstrap-server $BS
```

Check KRaft quorum status

```bash
./bin/kafka-metadata-quorum.sh --bootstrap-server $BS describe --status
```

---

8) Common scenarios

Scenario A: Create main topic + DLT correctly

```bash
./bin/kafka-topics.sh --bootstrap-server $BS --create \
--topic product-created-events-topic --partitions 3 --replication-factor 1

./bin/kafka-topics.sh --bootstrap-server $BS --create \
--topic product-created-events-topic-dlt --partitions 3 --replication-factor 1
```

Scenario B: DLT exists but has fewer partitions

```bash
./bin/kafka-topics.sh --bootstrap-server $BS \
--alter --topic product-created-events-topic-dlt --partitions 3
```

Scenario C: Delete unwanted topics

```bash
./bin/kafka-topics.sh --bootstrap-server $BS --delete --topic product-created-events-topic-dlt-2
./bin/kafka-topics.sh --bootstrap-server $BS --delete --topic product-created-events-topic-dlt-3
```

Kafka Command Reference?

Section 2: Apache Kafka Broker

Kafka (KRaft Mode) Setup & Commands – Formatted Documentation

1. Generate Cluster UUID

bin/kafka-storage.sh random-uuid

Example output:

an-adF1MQXebx1sNK7I2Mw

- **What it does:**

Generate a **unique Cluster ID** for your Kafka cluster.

- Kafka needs a permanent ID to identify a cluster.
- This ID is written in Kafka's metadata storage.
- Acts like a *fingerprint* for your Kafka installation.
- Generate once and reuse all brokers in the same cluster.

2. Format Kafka Storage (Single Node)

```
bin/kafka-storage.sh format \
  --config config/server.properties \
  --cluster-id i5obklXcSdKLvoyyYnp6-Q \
```

--standalone

◇ What it does:

Initializes Kafka's metadata storage in **KRaft mode**.

This command:

- Creates internal Kafka directories
- Stores the cluster ID
- Initializes controller metadata
- Prepares Kafka to run without ZooKeeper
- Marks this node as both **Broker + Controller**

Options Explained:

- --config config/server.properties
  → Uses the Kafka configuration file
- --cluster-id ...
  → Permanently attaches this node to the Kafka cluster
- --standalone
  → Single-node setup (broker + controller together)

⚠ Important:

- Run **only once**
- Running again deletes Kafka metadata
- Equivalent to *formatting a hard disk before installing an OS*

3. Start Kafka Server

bin/kafka-server-start.sh config/server.properties

◇ What it does:

Starts the Kafka server process.

It:

- Launches Kafka in KRaft mode
- Starts:
    - **Broker** → handles messages
    - **Controller** → manages metadata, partitions, leader election
- Opens ports:
    - 9092 → Client communication
    - 9093 → Controller communication

Once started:

- Kafka is live
- Producers/Consumers can connect
- Spring Boot apps can publish & consume events

Running Multiple Kafka Brokers (3-Node Cluster)

Step 1: Create Configuration Files

Copy server.properties three times:

config/server-1.properties
config/server-2.properties
config/server-3.properties

Step 2: Modify Each Server Configuration

◇ server-1.properties

node.id=1

# Broker & Controller ports
listeners=PLAINTEXT://:9092,CONTROLLER://:9093

controller.quorum.bootstrap.servers=localhost:9093,localhost:9095,localhost:9097

advertised.listeners=PLAINTEXT://localhost:9092,CONTROLLER://localhost:9093

log.dirs=/tmp/server-1/kraft-combined-logs

- **server-2.properties**

```
node.id=2

listeners=PLAINTEXT://:9094,CONTROLLER://:9095

controller.quorum.bootstrap.servers=localhost:9093,localhost:9095,localhost:9097

advertised.listeners=PLAINTEXT://localhost:9094,CONTROLLER://localhost:9095

log.dirs=/tmp/server-2/kraft-combined-logs
```

- **server-3.properties**

```
node.id=3

listeners=PLAINTEXT://:9096,CONTROLLER://:9097

controller.quorum.bootstrap.servers=localhost:9093,localhost:9095,localhost:9097

advertised.listeners=PLAINTEXT://localhost:9096,CONTROLLER://localhost:9097

log.dirs=/tmp/server-3/kraft-combined-logs
```

Step 3: Format Storage for Each Broker

Use the **same Cluster ID** for all brokers.

a) Format Server 1

```
./bin/kafka-storage.sh format \
  -t an-adF1MQXebx1sNK7I2Mw \
  -c config/server-1.properties \
  --standalone
```

Output:

Formatting dynamic metadata voter directory /tmp/server-1/kraft-combined-logs with metadata.version 4.1-IV1.

b) Format Server 2

```
./bin/kafka-storage.sh format \
  -t an-adF1MQXebx1sNK7I2Mw \
  -c config/server-2.properties \
  --standalone
```

Output:

Formatting dynamic metadata voter directory /tmp/server-2/kraft-combined-logs with metadata.version 4.1-IV1.

c) Format Server 3

```
./bin/kafka-storage.sh format \
  -t an-adF1MQXebx1sNK7I2Mw \
  -c config/server-3.properties \
  --standalone
```

Output:

Formatting dynamic metadata voter directory /tmp/server-3/kraft-combined-logs with metadata.version 4.1-IV1.

⚠ Run these **only once per broker**.

Step 5: Start All Kafka Servers

Open 3 terminals:

./bin/kafka-server-start.sh config/server-1.properties

./bin/kafka-server-start.sh config/server-2.properties

./bin/kafka-server-start.sh config/server-3.properties

Now you have:

- 3 Kafka brokers
- 3 Controllers
- Fully working Kafka KRaft cluster

Step 7: Stop Kafka Servers

./bin/kafka-server-stop.sh

(Repeat in all terminals or kill each process)

Interview Ready Summary

In Kafka 4.x, ZooKeeper is removed and Kafka runs in KRaft mode.
We first generate a cluster UUID, then format Kafka storage to initialize metadata and controller state.
Each broker must be formatted once with the same cluster ID.

Finally, we start each Kafka server using its own configuration file to form a multi-node Kafka cluster.

Section 3: Apache Kafka CLI: Topics

Creating a new Kafka Topic?

Step 1: Generate Cluster UUID
./ bin/kafka-storage.sh random-uuid

Step 2: Format Kafka Storage for all 3 server
./bin/kafka-storage.sh format --config config/server-1.properties --cluster-id an-adF1MQXebx1sNK7I2Mw --standalone ./bin/kafka-storage.sh format --config config/server-2.properties --cluster-id an-adF1MQXebx1sNK7I2Mw --standalone

```
./bin/kafka-storage.sh format --config config/server-3.properties --cluster-id an-adF1MQXebx1sNK7I2Mw --standalone
```

Step 3: Start all 3 Kafka servers.

```
./bin/kafka-server-start.sh config/server-1.properties

./bin/kafka-server-start.sh config/server-2.properties

./bin/kafka-server-start.sh config/server-3.properties
```

Partitions can be no. Of consumer.

Replication parameter specifies how many copies of each partition are stored across different brokers or different servers in my Kafka cluster.

And this is a way to achieve reliability or fault tolerance in Kafka Cluster because it allows data to be available even if some brokers fail or become unreachable.

For example, if you set replication factor to three, then each partition will have three replicas, one of which will be leader and other two will be followers.

But the number you specify here cannot be greater than the number of brokers that you have in your cluster.

So if you start at one server only, then you cannot have replication factor greater than one.

But if you start three servers in a cluster, then you can specify replication factor three, and each broker will have a copy of a partition.

Section 6: Kafka Producer – Spring Boot Microservice

Introduction to Kafka Producer?

Apache Kafka brokers accept messages or events from Kafka producers.

Kafka producer is a spring boot microservice that publishes events to Kafka Broker.

So the primary role of Kafka Producer is to publish messages or events to one or more Kafka topics.

In a Spring boot application, this is typically achieved using spring for Apache Kafka library, which simplifies the process of integrating Kafka functionality into your spring application.

Before sending a message, Kafka producer serializes it into a byte array.

So another responsibility of Kafka Producer is to serialize messages to binary format that can be transmitted over the network to Kafka brokers.

Another responsibility of Kafka producer is to specify the name of Kafka topic, where these messages should be sent.

In this particular use case, I call Kafka topic, the product created event topic, and it will store product created events.

When sending a new message to Kafka topic, we will also specify the partition where we want to store it, although this parameter is optional and if we do not specify topic partition, then Kafka Producer will make this decision for us.

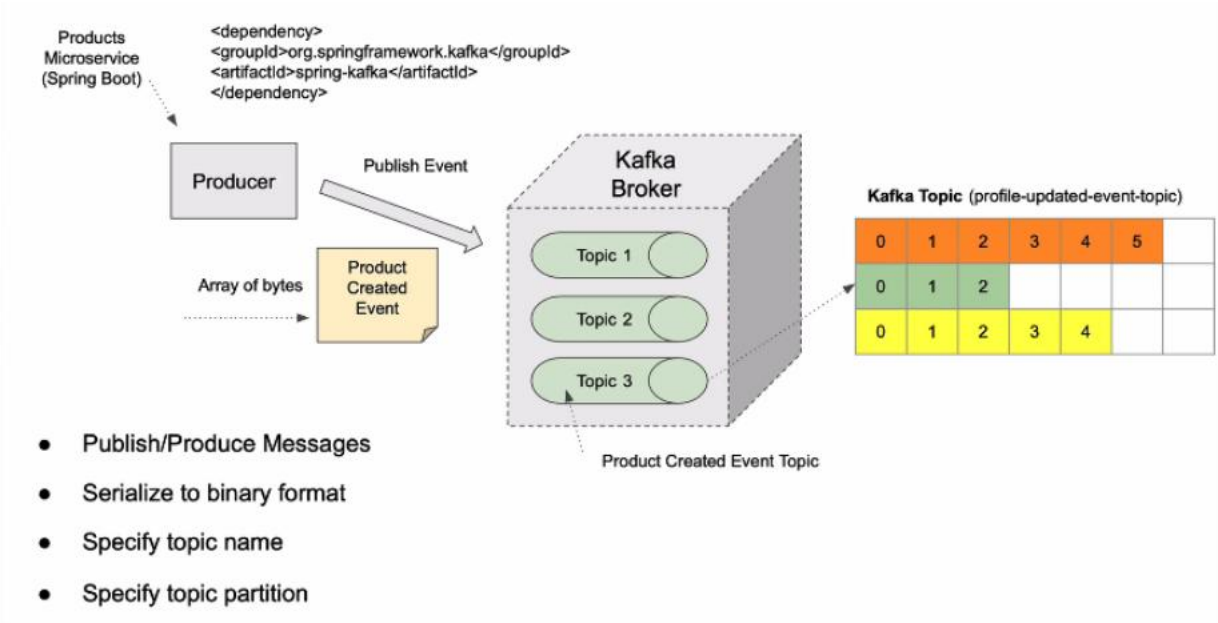
And if neither partition nor key is provided, then Kafka producer will distribute messages across partitions in a round robin fashion, ensuring a balanced load.

If a partition is not specified but a message key is provided, then Kafka will hash the key and use the result to determine the partition, and this ensures that all messages with the same key go to the same partition.

So another responsibility of Kafka producer is to choose which partition to write event data to.

Responsibility of Kafka Producer:

- Publish/Produce Messages
- Serialize to binary format
- Specify topic name
- Specify topic partitions



Kafka Producer – Introduction to synchronous communication style?

Let's assume that we have a mobile application that sends a Http request to spring boot microservice.

And let's assume that this spring boot microservice is called orders microservice.

This microservice application, it exposes API endpoints that allow to place new order, view existing orders, or maybe cancel or track order status as well.

So a mobile application can send a Http request to create a new order, for example.

And in this particular case, the mobile application will most likely use synchronous communication style with orders microservice and it will wait for a response.

It will wait for a response because it does want to know for sure if order was successfully created or not.

So synchronous communication style is when sender sends a request and then waits for a response before proceeding.

In the following lectures we will create and we will configure Spring Boot microservice to work as Kafka producer.

And we will use a special class from Spring Framework that is called Kafka Template.

This class encapsulates Kafka Producer and provides a very user friendly way to interact with Kafka from Spring Boot application.

So our orders microservice will create a new object.

It will be called Order Created Event, and it will use Kafka Producer to send a message to Kafka Broker.

If orders microservice wants to know that the order created event was successfully stored in Kafka Broker,

it will use Kafka Producer to send Kafka message using synchronous communication style, which means that it will wait for acknowledgement from Kafka Broker to confirm

that the message was successfully received and stored before it proceeds with sending next message or performing other operations.

Because Kafka Producer is waiting to receive a response, it is blocked or it is on hold until the response from Kafka Broker is received.

Once Kafka Broker successfully stores Kafka message in a topic, it will send the response to Kafka Producer that everything is okay and that the message has been stored.

And once Kafka producer receives this acknowledgement that the message is stored successfully, or this microservice will respond back to mobile application with Http response.

So in this diagram, I used synchronous communication style between mobile application and spring boot microservice, and between Kafka Producer and Kafka Broker.

Using synchronous communication style between Kafka Producer and Kafka broker is considered to be more reliable because it ensures that message was successfully stored before moving on to the next task.

But because producer is waiting, it can make your system just a little bit slower.

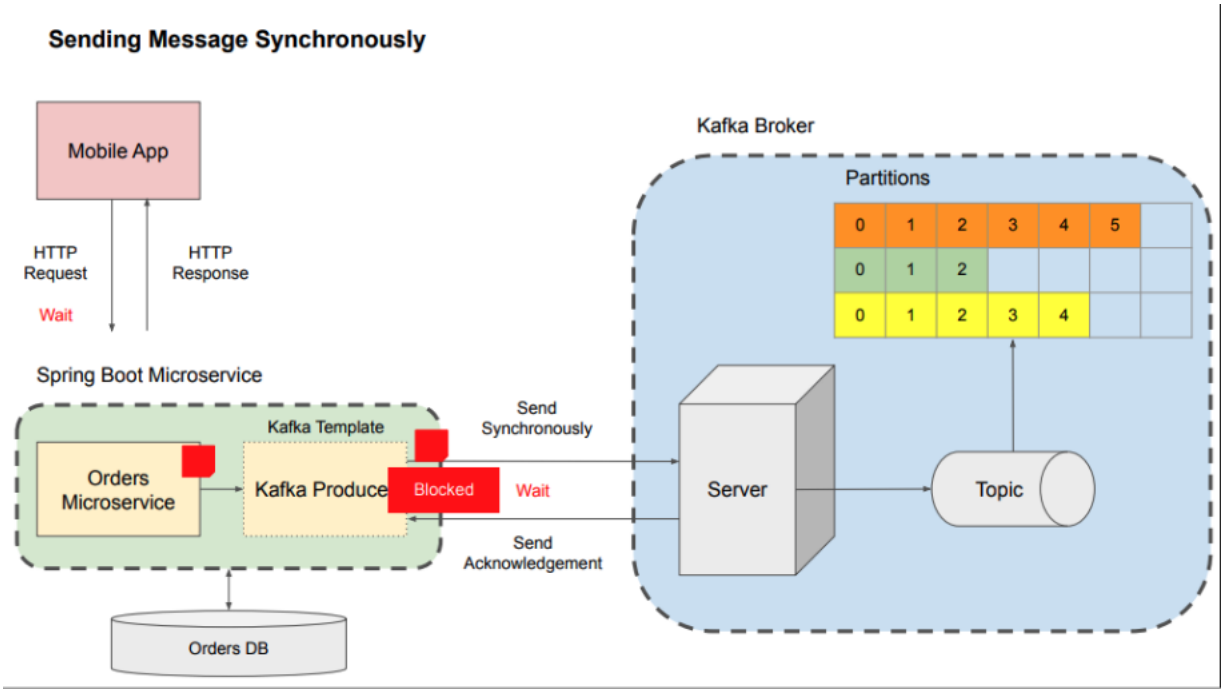
In those cases where high throughput and low latency is required, you might want to consider a different communication style that is called asynchronous communication.

When using a synchronous communication style, Kafka producer sends a message to Kafka Broker and synchronously, but it is not put on hold after and it is not blocked.

While waiting for response from Kafka Broker, Kafka producer sends a message to Kafka Broker and it continues its execution without being blocked or put on hold.

It can still receive a response from Kafka Broker, but it will receive it at a later point.

And that is the main difference between synchronous and asynchronous communication styles.



Kafka Producer – A use case for asynchronous communication style?

Let's assume that we have a mobile application that displays user login page.

Before user can proceed, they will need to enter their username and password to authenticate.

And once user enters their username and password and taps on login button, mobile application will send a Http request to a login API endpoint.

In this case, mobile application will use synchronous communication style and will wait for Spring boot microservice to respond.

Let's assume that user authentication is handled by authentication microservice.

Authentication Microservice will check its database to see if user with provided username and password exists, and if authentication is successful, it will create a new **UserLoggedEvent** event.

It will then use Kafka Producer to publish this event to Kafka Broker.

Now the purpose of this **UserLoggedEvent** event is only informational and it is needed for analytics purposes only.

Even if this event fails and does not get stored in Kafka topic, we don't want our user to wait or receive error message.

So in this case, we make Kafka producer send message asynchronously.

If we send message asynchronously, Kafka producer will not be blocked, it will not be put on hold and will continue its execution right away.

And because user authentication is successful, our microservice will return a successful Http response.

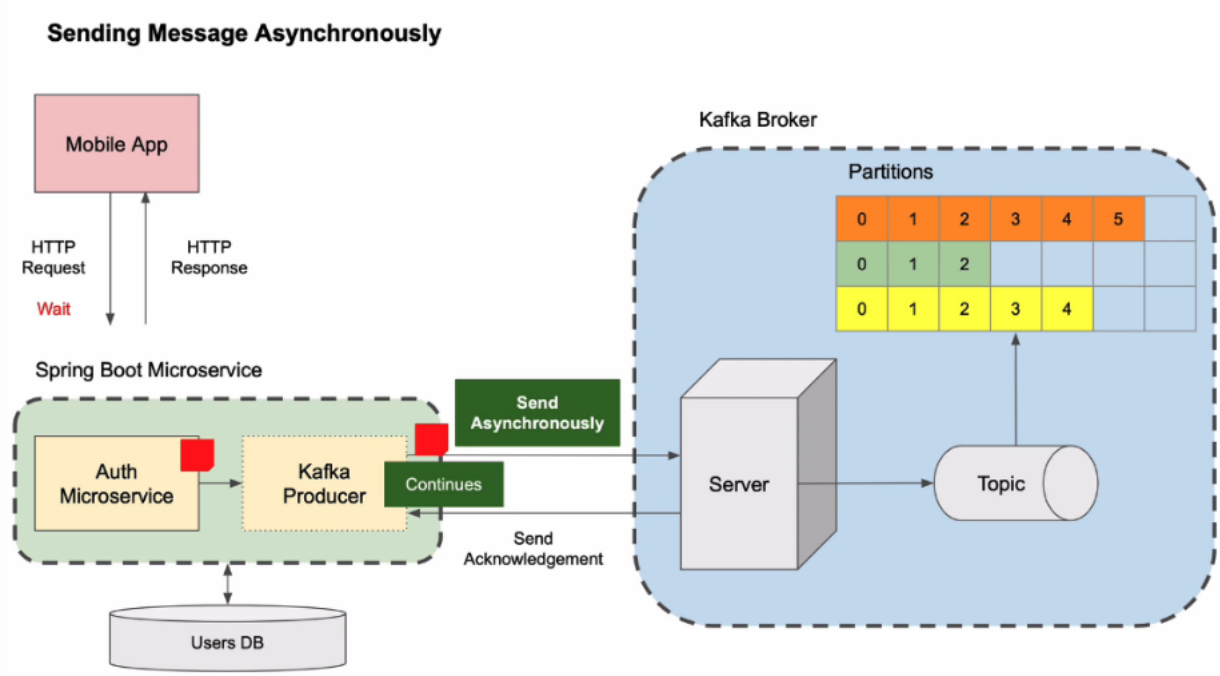
So the user of mobile application will log in and will start using it without waiting for Kafka Broker to acknowledge that it has received an event.

Meanwhile, because **UserLoggedEvent** event was published, Kafka Broker will receive it and it will store it in Kafka topic.

But even if this operation fails and Kafka Broker does not store this event successfully, it will not affect user of our mobile application.

Now, even though Kafka producer does send message asynchronously if needed, we can still make our microservice receive and process response from Kafka Broker.

Once Kafka Broker sends its acknowledgment, we can handle it and process it with asynchronous callback function.



Section 7: Kafka Producer: Acknowledgement and Retries

Kafka Producer Acknowledgement?

When a Kafka **producer** sends a record, it is sent to a **Kafka broker**. The broker writes the record to a **topic**, and within that topic the record is stored in a specific **partition**.

Replication, Leader, and Followers

If your Kafka cluster uses a **replication factor > 1**, each partition is replicated across multiple brokers:

- One broker is elected as the **leader** for that partition.
- Other brokers host replicas as **followers** (replicas).
- The producer writes to the **leader**. The leader then replicates the record to follower replicas.

Why acknowledgements matter

After the leader receives and writes the record, Kafka sends an **acknowledgement (ack)** back to the producer based on the producer's configuration.

If the leader **acknowledges too early** (before followers replicate) and then crashes permanently, the record may be **lost**. To improve durability, you can configure the producer to wait for acknowledgements from more replicas.

Producer acks **settings (durability vs latency)**

acks=1 **(Leader only)**

- The producer waits for an acknowledgement from the **leader broker only**.
- **Pros:** good latency / throughput
- **Cons:** record may be lost if the leader fails before followers replicate

This is commonly the default and is a practical balance between speed and safety.

acks=all **(All in-sync replicas)**

- The producer waits for acknowledgements from **all in-sync replicas (ISR)**.
- **Pros:** strongest durability; data is not lost as long as at least one ISR remains alive
- **Cons:** higher latency; slower writes because the producer waits for multiple brokers

Spring Kafka example property:

- spring.kafka.producer.acks=all

acks=0 **(No acknowledgements)**

- The producer does **not** wait for any acknowledgement.
- **Pros:** fastest possible producer performance
- **Cons:** no guarantee the broker received or stored the record (possible silent loss)

Typical use case: high-volume, low-criticality data like frequent GPS location updates where occasional loss is acceptable.

Does acks=all **get slower as you add more brokers?**

Yes and no.

- **No**, because the producer does **not** wait for every broker in the cluster.
- It waits only for the **in-sync replicas (ISR)** for the partition being written to.

The number of replicas involved depends on the topic's **replication factor**, which is set when the topic is created.

Controlling how many replicas must acknowledge: min.insync.replicas

Kafka also lets you configure a minimum number of replicas that must be in-sync to accept writes durably:

- min.insync.replicas=2 (example)

This means:

- Even if a partition has (say) 5 replicas, Kafka can be configured so that only **2 in-sync replicas** must successfully store the record for the write to be considered valid (when using acks=all).

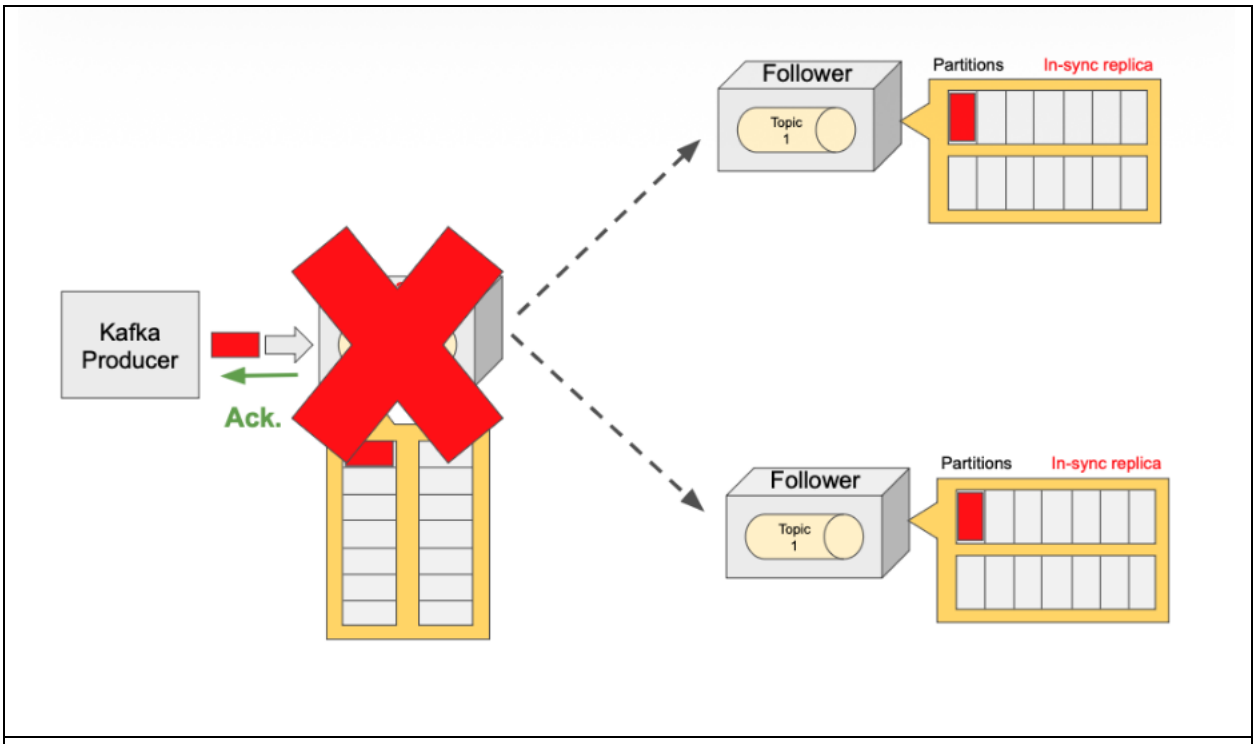
Effect:

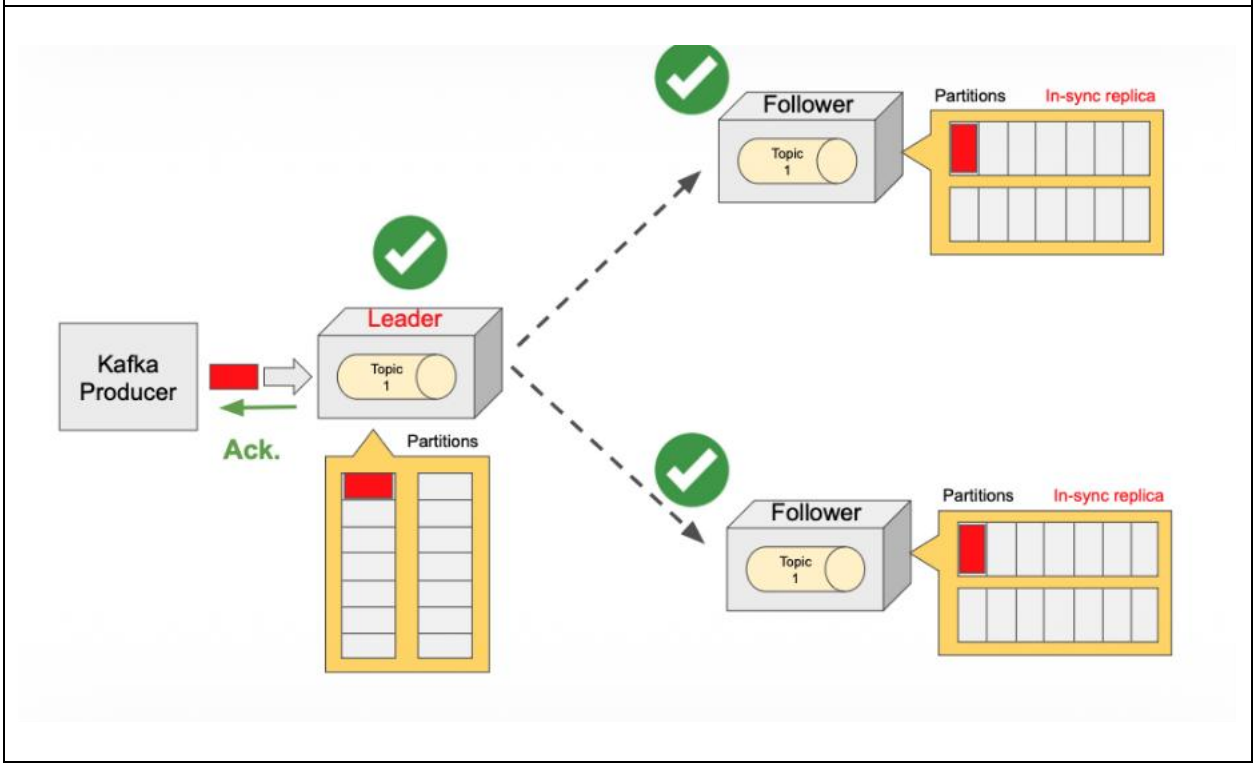
- **Faster** than waiting for all replicas
- Still **reliable**, because at least the configured number of brokers have persisted the record

Key takeaway

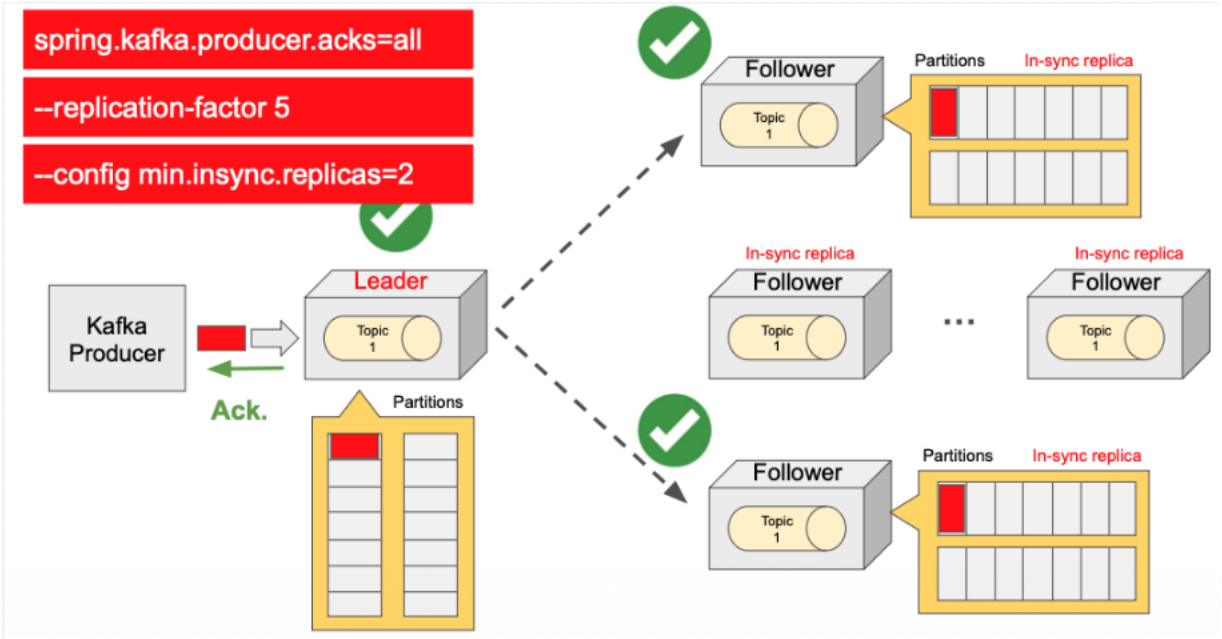
Use producer acknowledgement settings (acks) **together with** replication and ISR-related settings (like min.insync.replicas) to tune the trade-off between:

- **Latency/throughput**
- **Durability and data-loss risk**

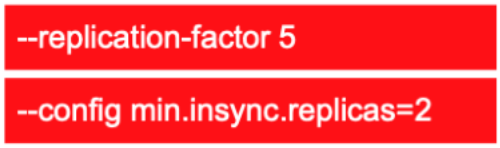

Follower
Partitions
In-sync replica
Topic 1
Kafka Producer
Ack.
Follower
Partitions
In-sync replica
Topic 1


Follower
Partitions
In-sync replica
Topic 1
Leader
Kafka Producer
Topic 1
Ack.
Partitions
Follower
Partitions
In-sync replica
Topic 1

- **spring.kafka.producer.acks=all**
  Waits for an acknowledgement from all brokers.
- **spring.kafka.producer.acks=1**
  Waits for an acknowledgement from a leader broker only.
- **spring.kafka.producer.acks=0**
  Does not wait for an acknowledgement.



- **spring.kafka.producer.acks=all**
  Waits for an acknowledgement from all brokers.



Kafka Producer Retries?

Context: acks=all + min.insync.replicas

If you configure strong durability, for example:

- Producer acks=all (wait for all **in-sync replicas**)
- Broker/topic min.insync.replicas=3

...it means the broker will only accept the write if **at least 3 replicas are in-sync** (the leader + 2 followers). The producer will only get a successful acknowledgement once the leader and required in-sync replicas have persisted the record.

What if a follower goes down?

If one follower goes down and the cluster no longer has enough in-sync replicas to satisfy min.insync.replicas=3, the write cannot be acknowledged successfully.

In that case, the producer typically receives a **retriable error** (temporary failure), and Kafka's default behavior is to **retry** sending until:

- the record is successfully delivered, or
- the **delivery timeout** is reached (default is **2 minutes**)

Producer responses you may see

When a producer sends a record, one of the following may occur:

1) No response expected (acks=0)

If acks=0, the producer does not wait for acknowledgements. It sends and continues without knowing whether the broker stored the record.

- **Pros:** fastest
- **Cons:** no delivery guarantees

2) Success acknowledgement

With acks=1 or acks=all, the broker responds with a success acknowledgement if the write meets the acknowledgement requirements.

3) Error response

If the broker cannot complete the request, it responds with an error. Errors fall into two broad categories:

Non-retriable (permanent) errors

Retrying won't help. Example:

- **Record too large** (exceeds max.request.size / broker limits)

Result: producer fails the send and returns/raises an exception.

Retriable (temporary) errors

Retrying might succeed later. Examples:

- transient network issues
- leader temporarily unavailable (leader election)
- not enough in-sync replicas (e.g., ISR shrank below min.insync.replicas)

Result: producer automatically retries (you typically don't need custom logic).

Retry controls (key producer configs)

Kafka automatically decides whether an error is retriable. You control *how long / how often* it retries using producer settings.

1) retries

Controls **how many retry attempts** the producer will make.

- Default is very large (effectively "retry a lot") but bounded by delivery.timeout.ms.
- If you set retries=10, the producer will try up to 10 retries (subject to timeout).

2) retry.backoff.ms

Controls the **wait time between retries**.

- Default: 100ms
- Example: 1000ms means wait 1 second between retries.

Example behavior

If:

- retries=10
- retry.backoff.ms=1000

Then on a retriable error the producer will retry **up to 10 times**, waiting **~1 second** between attempts (unless delivery.timeout.ms is hit first).

Preferred control: delivery.timeout.ms

Kafka documentation generally recommends controlling retries via **total time spent trying**, rather than a fixed retry count.

delivery.timeout.ms

Defines the **maximum total time** allowed for the entire send operation, including:

- time waiting to batch (linger.ms)
- time waiting for broker responses (request.timeout.ms)
- time spent retrying

Default: **120000 ms (2 minutes)**

If delivery.timeout.ms is exceeded, the producer fails the send with a timeout error.

Related timing configs (must be consistent)

linger.ms

Maximum time the producer waits to **batch** records before sending them.

- Default: 0 (send ASAP)
- Increasing linger.ms improves batching and can improve throughput (and compression effectiveness), at the cost of latency.

request.timeout.ms

Maximum time to wait for a response from the broker for a **single request**.

- Default: 30000 ms (30 seconds)

Important constraint

If you change delivery.timeout.ms, it must be >= the effective time budget implied by:

- linger.ms
- request.timeout.ms (and retry behavior)

In practice, ensure delivery.timeout.ms is large enough that requests + retries can actually occur.

Spring Boot / Spring Kafka configuration examples

Example A: Reliable writes with bounded retries

```
properties
# Strong durability
spring.kafka.producer.acks=all
# Retry controls
spring.kafka.producer.properties.retries=10
spring.kafka.producer.properties.retry.backoff.ms=1000
# Total time budget for delivery (2 minutes here)
spring.kafka.producer.properties.delivery.timeout.ms=120000
# Single-request wait time (30s default)
spring.kafka.producer.properties.request.timeout.ms=30000
# Optional batching (0 default; increase for throughput)
spring.kafka.producer.properties.linger.ms=0
```

Example B: Prefer "time-based" delivery control (recommended pattern)

```
properties
spring.kafka.producer.acks=all
# Let retries be high, but cap total delivery time
spring.kafka.producer.properties.retries=2147483647
spring.kafka.producer.properties.delivery.timeout.ms=60000
spring.kafka.producer.properties.request.timeout.ms=30000
spring.kafka.producer.properties.retry.backoff.ms=250
```

This approach emphasizes: "Try as needed, but don't exceed 60 seconds total."

Example C: High-throughput, low criticality (acks=0)

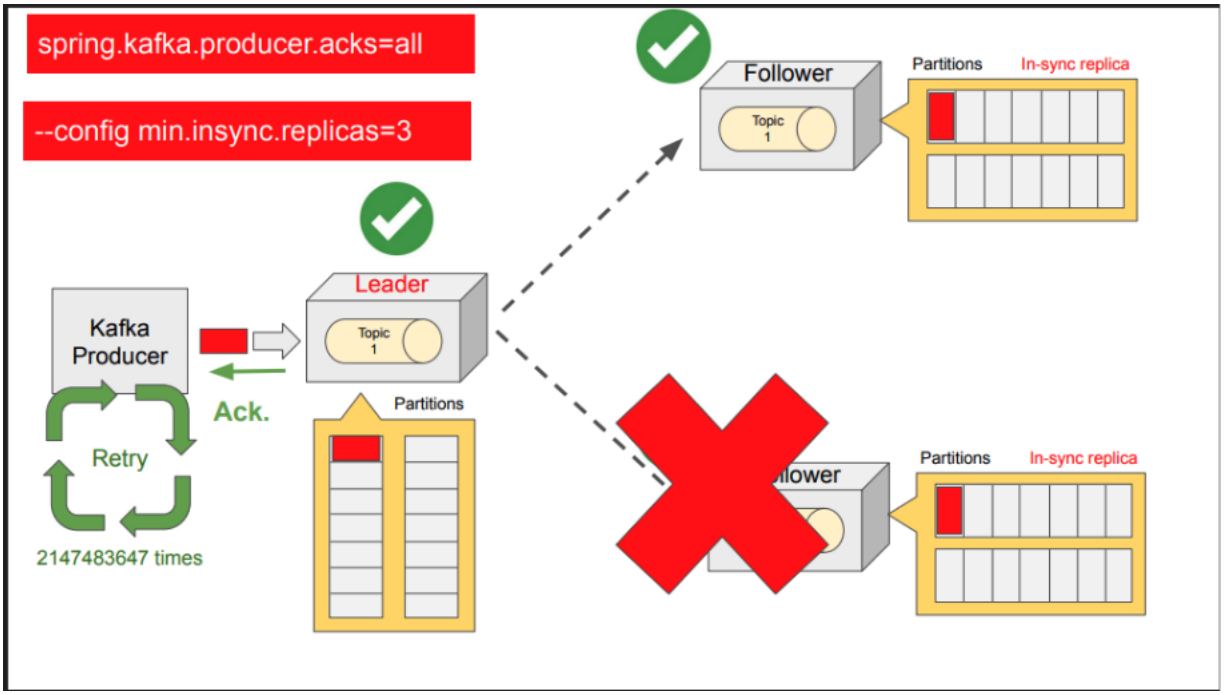
```
properties
spring.kafka.producer.acks=0
spring.kafka.producer.properties.linger.ms=20
```

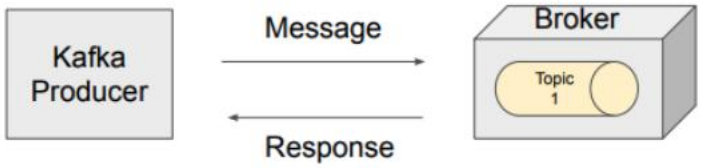
Fastest producer behavior, but you accept potential message loss.

Key takeaway

Use these properties together to define producer behavior under transient failures:

- **Durability requirement:** acks (often all for critical data)
- **Retry pacing:** retry.backoff.ms
- **Retry limit:** retries (optional; often less important than time-based control)
- **Overall time budget:** delivery.timeout.ms (most important)
- **Batching vs latency:** linger.ms
- **Per-request timeout:** request.timeout.ms





- **No response** - if the producer is configured with acks=0
- Acknowledgement (ACK) of Successful Storage
- **Non-Retryable Error** - A permanent problem that is unlikely to be resolved by retries.
- **Retryable Error** - A temporary problem that can be resolved by retrying the send operation.

spring.kafka.producer.**retries**=10

How many times Kafka Producer will try to send a message before marking it as failed. Default value is 2147483647.

spring.kafka.producer.properties.**retry.backoff.ms**=1000

How long the producer will wait before attempting to retry a failed request. Default value is 100 ms.

spring.kafka.producer.properties.**delivery.timeout.ms**=120000

The maximum time Producer can spend trying to deliver the message. Default value is 120000 ms (2 minutes).

delivery.timeout.ms >= linger.ms + request.timeout.ms

spring.kafka.producer.properties.**linger.ms**=0

The maximum time in milliseconds that the producer will wait and buffer data before sending a batch of messages. The default value is 0.

spring.kafka.producer.properties.**request.timeout.ms**=30000

The maximum time to wait for a response from the broker after sending a request. The default value is 30000 ms.

Section 8: Idempotent Producer in Java

Idempotent Kafka Producer Introduction?

Why idempotence is needed

Kafka producers and brokers communicate over the network, so failures can happen even in the "happy path".

Normal (happy path):

1. The producer sends a record to the broker.
2. The broker writes the record to the target topic/partition.
3. The broker sends an acknowledgement (ack) back to the producer.

Failure scenario (duplicates without idempotence):

1. The producer sends a record.
2. The broker successfully writes it.
3. The broker sends an ack, but a network issue prevents the ack from reaching the producer.
4. The producer assumes the send failed and retries (if retries are enabled).
5. The broker receives the same record again and writes it again.

Result: **duplicate records** appear in the topic.

Why duplicates are a problem

In some systems duplicates are tolerable, but in others they can cause serious issues:

- **Banking:** a customer could be charged twice.
- **E-commerce:** an order could be shipped twice.

What an idempotent producer does

An **idempotent producer** prevents duplicates caused by retries.

Definition (practical): An idempotent producer can resend the same record multiple times, but the broker will **store it only once** and **preserve ordering** (within the scope of Kafka's idempotence guarantees).

How it behaves with retries enabled

With idempotence enabled:

1. Producer sends a record.
2. Broker writes it and attempts to send an ack.
3. If the ack is lost and the producer retries:
    a. The broker recognizes the retry as a duplicate send.
    b. The broker does **not** write the record again.
    c. The broker simply acknowledges it.

Result: **no duplicates** in the topic even when retries occur due to transient failures.

How to enable idempotence

Kafka producer property

The key setting is:

- enable.idempotence=true

Spring Boot / Spring Kafka (application.properties)

properties
spring.kafka.producer.properties.enable.idempotence=true

Native Kafka producer config (generic)

properties
enable.idempotence=true

Important note about defaults

In newer Kafka versions, idempotence is commonly **enabled by default**.
Even so, it's recommended to **explicitly set** enable.idempotence=true to avoid accidental disabling due to conflicting configuration.

Required supporting settings (to keep idempotence valid)

If you want idempotence to work reliably, these settings must not conflict:

1) acks=all

The producer should wait for acknowledgements from all in-sync replicas.

properties
acks=all

2) retries > 0

Retries must be enabled (a positive value). This is part of the safe delivery model for transient failures.

properties
retries=10

3) max.in.flight.requests.per.connection <= 5

This limits how many produce requests can be in-flight simultaneously without waiting for acknowledgements.

- Keeping it **≤ 5** helps the producer preserve ordering during retries/failures and avoid inconsistent ordering behavior.

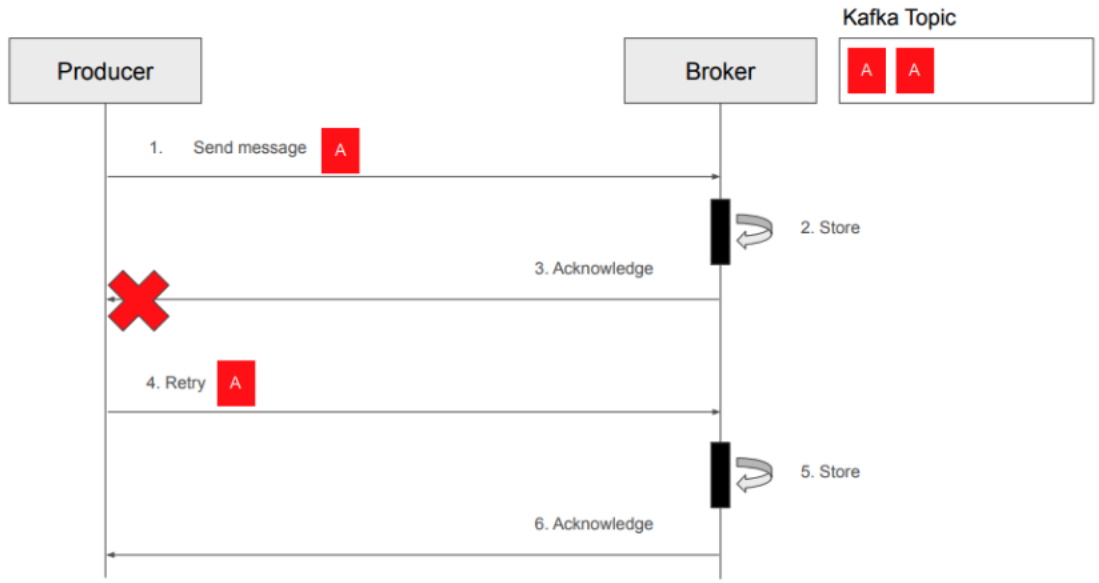
properties
max.in.flight.requests.per.connection=5

What happens if configs conflict?

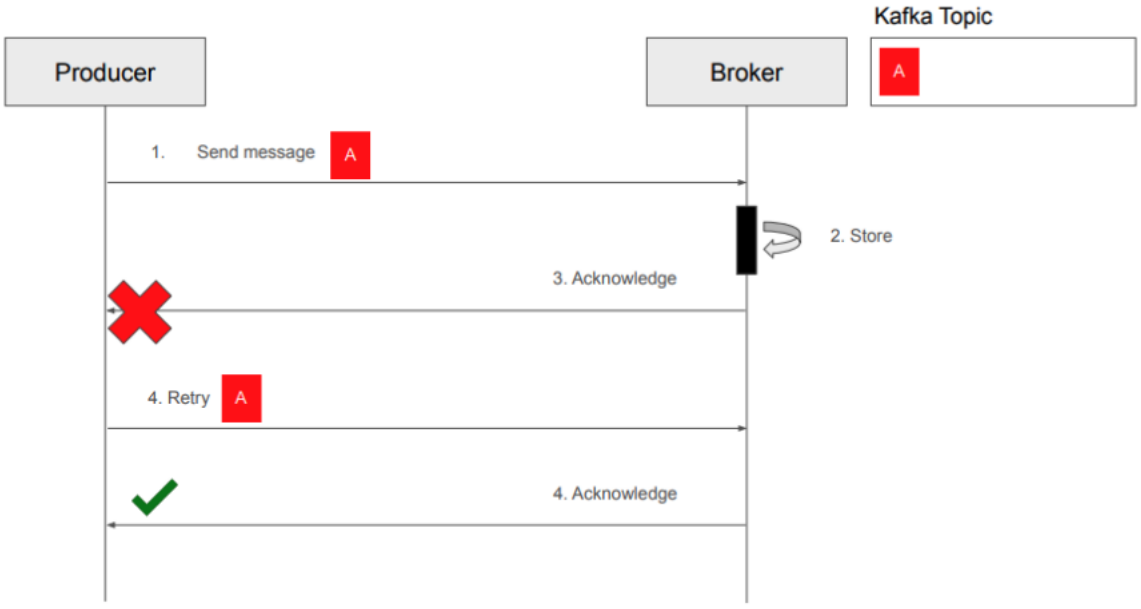
- If idempotence is **not explicitly enabled**, conflicting settings may silently disable the default idempotence behavior.
- If you **explicitly set** enable.idempotence=true but configure incompatible values, Kafka will throw a **configuration exception** at startup (fail fast), which is usually what you want.

Example: Recommended Spring Kafka configuration

properties
spring.kafka.producer.acks=all
spring.kafka.producer.properties.enable.idempotence=true
spring.kafka.producer.properties.retries=10
spring.kafka.producer.properties.max.in.flight.requests.per.connection=5



An idempotent producer avoids duplicate messages in the log in the presence of failures and retries.



enable.idempotence = true

application.properties file

```
spring.kafka.producer.properties.enable.idempotence=true
```

@Bean

```
props.put (ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);
```

acks=all

retries=2147483647

max.in.flight.requests.per.connection=5

application.properties file

```
spring.kafka.producer.properties.acks=all
spring.kafka.producer.properties.retries=2147483647
spring.kafka.producer.properties.max.in.flight.requests.per.connection=5
spring.kafka.producer.properties.enable.idempotence=true
```

ConfigException

Section 9: Kafka Consumer – Spring Boot Microservice

Introduction to Kafka Consumer?

Products microservice as a Kafka **producer**. It publishes events (messages) to a Kafka topic.

To read these events, you will create a new **Spring Boot microservice** that acts as a Kafka **consumer**. This consumer service will be called the **email notification microservice**, and it will continuously consume new messages from the Kafka topic.

Kafka consumer does not delete messages

When a Kafka consumer reads a message from a topic, Kafka **does not delete** that message. The record remains in the topic and can still be read again (depending on consumer offsets and retention settings).

This also means you can run **multiple independent consumers**, and each consumer can receive its **own copy** of the messages (based on how they are grouped/configured).

Topic partitions and message storage

Consider a topic named **Product Created Event Topic** with **3 partitions**.

- Messages in Kafka are stored inside **partitions**
- Each partition is an ordered log of records
- Each record has an **offset** (0, 1, 2, …) within its partition

In the diagram representation:

- Partition 0 contains its own sequence of messages
- Partition 1 contains its own sequence of messages
- Partition 2 contains its own sequence of messages

How consumers read from partitions

A consumer service polls Kafka for new records at a regular interval (this polling behavior is configurable via consumer properties).

Parallel reads across partitions

Kafka reads from different partitions **in parallel**.

- **No ordering guarantee across partitions**
- **Ordering is guaranteed within a single partition**

Example (Partition 1):

- Offset 0 is always read before offset 1
- Offset 1 is always read before offset 2
- And so on

But Kafka does not guarantee that Partition 1 messages are processed before Partition 2 messages.

Scaling consumers with partitions (consumer group concept)

If you run only **one consumer instance**, it can read from **all 3 partitions**.

If you run **three instances** of the same consumer microservice (as part of the same logical group), Kafka can assign:

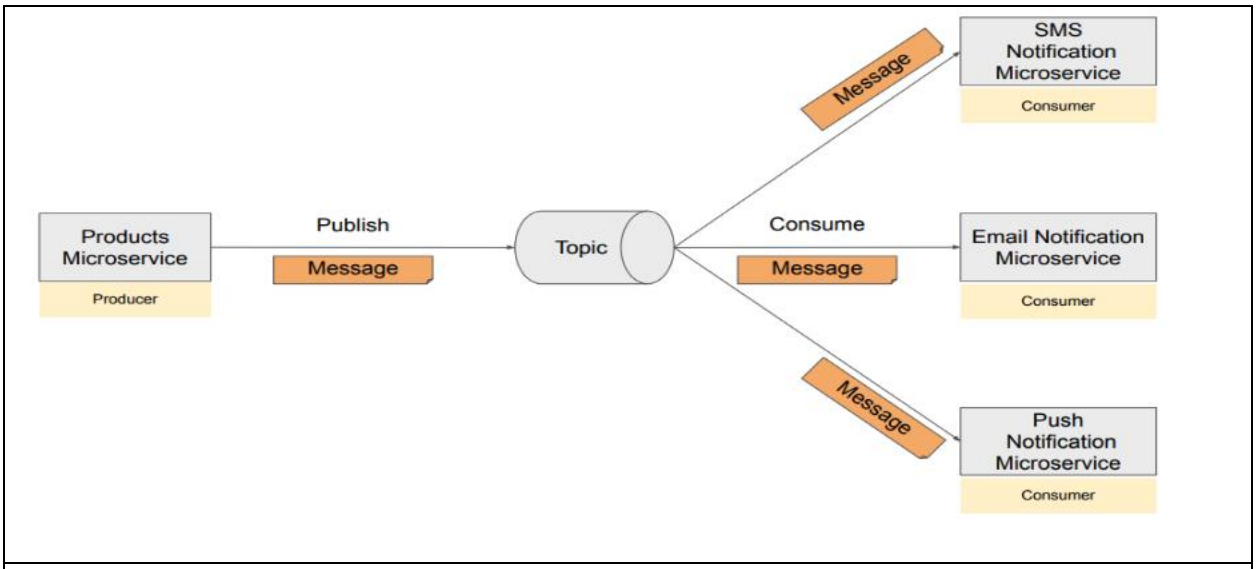
- Consumer 1 -> Partition 0
- Consumer 2 -> Partition 1
- Consumer 3 -> Partition 2

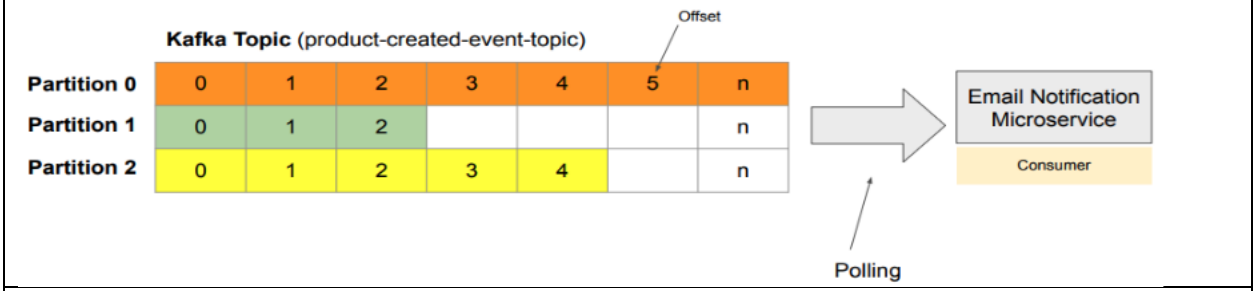
This lets you scale processing throughput because work is distributed across consumer instances. (You'll cover the detailed mechanics later in the lessons.)

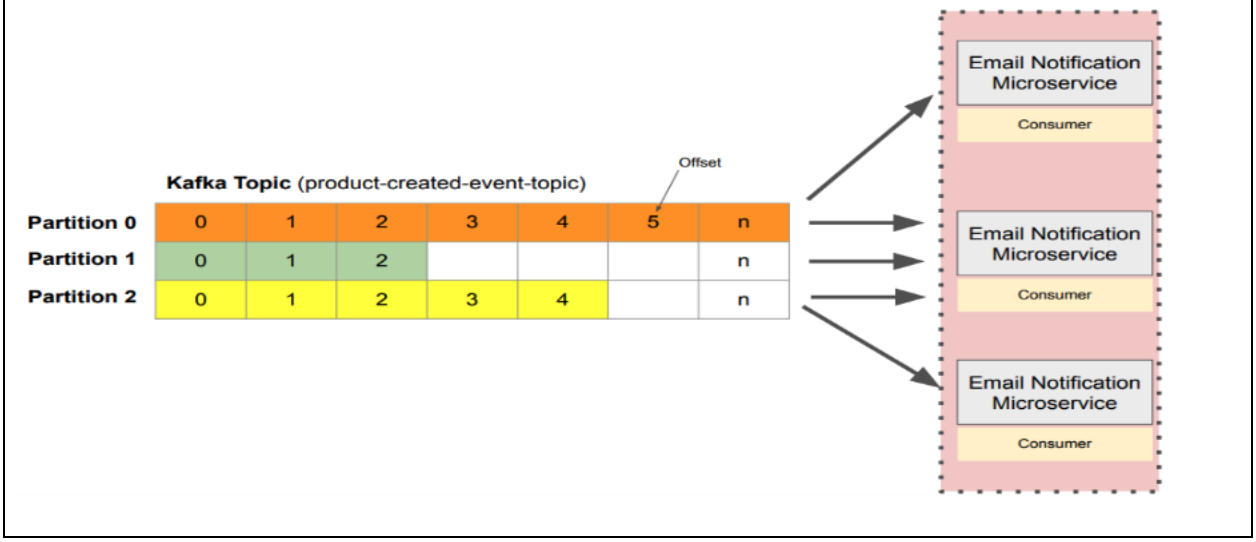
Message retention (how long messages stay)

Because Kafka doesn't delete records when consumed, messages remain in the topic until Kafka removes them based on **retention policy**.

- Default retention is **168 hours (7 days)**
- This can be changed using Kafka topic configuration properties

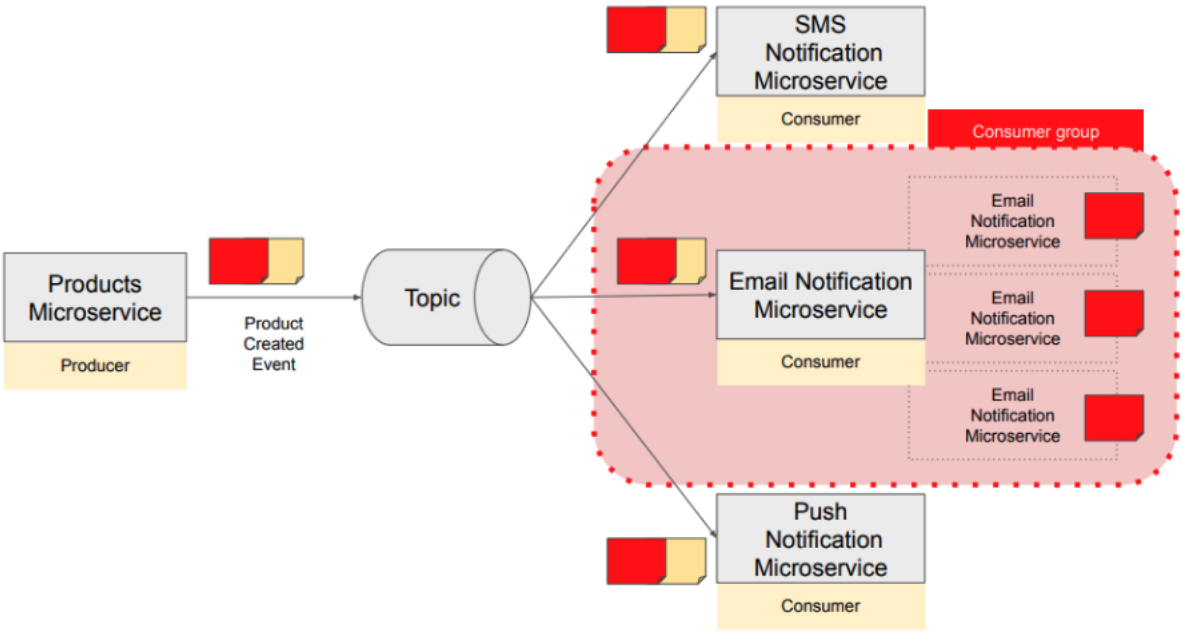

Products Microservice
Producer
Publish
Message
Topic
Consume
Message
Message
Message
SMS Notification Microservice
Consumer
Email Notification Microservice
Consumer
Push Notification Microservice
Consumer


Kafka Topic (product-created-event-topic)
Offset
Partition 0
Partition 1
Partition 2
0 1 2 3 4 5 n
0 1 2 n
0 1 2 3 4 n
Email Notification Microservice
Consumer
Polling


Kafka Topic (product-created-event-topic)
Offset
Partition 0
Partition 1
Partition 2
0 1 2 3 4 5 n
0 1 2 n
0 1 2 3 4 n
Email Notification Microservice
Consumer
Email Notification Microservice
Consumer
Email Notification Microservice
Consumer

Section 13: Kafka Consumer – Multiple Consumers in a Consumer Group

Introduction to Kafka Consumer Groups?



Introduction to Kafka Consumer Groups — Documentation Version

Scenario: one producer, multiple consumers

In this architecture:

- A **products microservice** acts as a **Kafka producer**.
- It publishes a **Product Created Event** to a **Kafka topic**.
- Multiple microservices listen to that topic as **Kafka consumers**.

When the producer publishes a new event, Kafka stores it in the topic. As soon as the event is available, consumers that subscribe to the topic can read it.

Two different consumption patterns

1) Different microservices: each one should receive a copy

If you have different consumer services such as:

- Email Notification Service
- SMS Notification Service
- Push Notification Service

…then **each service needs its own copy** of the Product Created Event.
So all three services will consume the same event (each for its own purpose).

This is a common pattern where multiple independent services react to the same business event.

The scaling problem: multiple instances of the same service

Now consider the **email notification microservice**.

If you scale it horizontally and run **4 instances** of the email service, you **do not** want all 4 instances to process the exact same message—because that would send 4 emails for the same event.

Instead, you want:

- The event to be processed **only once**
- By **any one** of the email-service instances

What is a Kafka consumer group?

A **consumer group** is Kafka's mechanism for scaling a consumer application safely.

When multiple instances of the **same microservice** share the same `group.id`:

- Kafka treats them as members of **one consumer group**
- For each message/partition, Kafka ensures only **one group member** processes the data
- Work is distributed across the group members

This allows you to scale processing throughput without duplicating work.

Why consumer groups help with performance

Consumer groups are especially useful when:

- The topic contains **lots of messages**
- One consumer instance cannot keep up

- You want faster processing by running **multiple instances** of the same consumer service

In this model:

- You can start multiple instances of the email notification microservice
- Group them into one consumer group
- Each instance processes a portion of the work

Even with multiple instances running, **each new message is processed once**, by exactly one instance in that consumer group.

Rebalancing and partition Assignment in Apache Kafka?

Kafka Consumer Rebalancing & Partition Assignment (Apache Kafka)

Scenario setup

Assume you have a Kafka topic named `product-created-events` with **3 partitions**. Messages published to this topic are distributed across these partitions.

You also have an **email-notification microservice** that acts as a **Kafka consumer** (part of a consumer group).

How partition assignment works

Single consumer instance

If you run **one** instance of the email-notification service in a consumer group, Kafka assigns that single consumer **all 3 partitions**.
That consumer reads from all partitions (in parallel) and processes messages.

Scaling out: adding more consumer instances

When load increases, you can scale by starting additional instances of the same consumer microservice (same `group.id`).

Example: start a **second** consumer instance.

- Kafka detects there are now **2 consumers** in the group.
- Kafka **reassigns** the partitions across the two consumers.
- After assignment:
    - One consumer might read **1 partition**
    - The other consumer might read **2 partitions**

Kafka tries to distribute partitions so work is shared.

What is rebalancing?

Rebalancing is the automatic process where Kafka **reassigns partitions** among consumers in the same consumer group.

Rebalancing happens whenever:

- A **new consumer joins** the group
- An **existing consumer leaves** the group (shutdown, crash, network issue)

Important behavior during rebalancing

- Rebalancing is usually quick.
- But during rebalancing, consumers **pause consumption briefly** while assignments are updated.

After the rebalance completes, all active consumers resume consuming from their newly assigned partitions.

Key rule: consumers cannot exceed partitions (within one group)

A critical constraint:

- **One partition can be consumed by only one consumer in a group at a time.**
- Therefore, you **cannot** have more active consumers than partitions for a given topic (within the same consumer group).

Examples:

- If a topic has **1 partition**, only **1 consumer** in the group can be active.
- If a topic has **3 partitions**, you can scale up to **3 consumer instances**.
- If you start a **4th consumer**, it will **sit idle** (no partition available to assign).

Group coordinator and heartbeats

When a consumer joins a group, Kafka assigns one broker as the **group coordinator** for that consumer group.

Consumers regularly send **heartbeat** messages to the group coordinator to indicate they are alive and able to consume.

- Default heartbeat interval is about **3 seconds** (configurable).

What happens when heartbeats stop?

If a consumer stops (crash, shutdown, network partition), it stops sending heartbeats.

When the broker detects missing heartbeats:

- It removes that consumer from the group
- It triggers a **rebalance**
- It reassigns the partitions among the remaining consumers

Kafka ensures again that:

- Each partition is assigned to **exactly one** consumer in the group.

Summary

- **Partition assignment** determines which consumer reads which partitions.
- **Rebalancing** automatically redistributes partitions when consumers join/leave.
- **Max parallelism** in a consumer group is limited by the **number of topic partitions**.
- **Heartbeats** to the group coordinator control membership; missing heartbeats trigger rebalancing.

Assigning Microservices to a consumer group in Kafka?

Assigning a Microservice to a Kafka Consumer Group — Documentation Version

Goal

To scale a Kafka consumer microservice (like an email-notification service) without processing the same message multiple times, you assign all instances of that microservice to the same **consumer group** using group.id.

When multiple instances share the same group.id, Kafka ensures each message is processed **once** by **one** instance in the group.

3 ways to set group.id (Spring Kafka)

1) Using KafkaListener annotation

You can set the consumer group directly on the listener:

```
java
@KafkaListener(topics = "product-created-events-topic", groupId = "product-created-events")
public void handle(ProductCreatedEvent event) { ... }
```

Use when:

- You want the group assignment close to the listener logic
- You have different listeners that intentionally use different groups

2) Using ConsumerFactory configuration (Java config)

You can set the group id in your Kafka consumer configuration class:

```
java
config.put(ConsumerConfig.GROUP_ID_CONFIG, "product-created-events");
```

Often this value is read from config using Environment (or

Value) rather than hardcoded.

Use when:

- You prefer central control of consumer settings
- You want one place to manage consumer configs (deserializers, retries, etc.)

3) Using application.properties / application.yml

Spring Boot supports configuring the group id via properties:

```
properties
spring.kafka.consumer.group-id=product-created-events
```

(or in YAML)

Use when:

- You want environment-based configuration (dev/test/prod)
- You want to avoid hardcoding group ids in code

Recommended approach (best practice)

Pick **one primary approach** to avoid confusion. Most teams prefer:

- application.properties / application.yml for spring.kafka.consumer.group-id

...and keep Java config focused on advanced customization (deserializers, error handling, DLT, etc.).

Using a custom property name (optional pattern)

You *can* define your own custom property key (for clarity), for example:

properties
```
consumer.group.id=product-created-events
```

Then read it inside ConsumerFactory:

java
```
config.put(
ConsumerConfig.GROUP_ID_CONFIG,
environment.getProperty("consumer.group.id")
);
```

This makes it obvious the property is **custom**, not a Spring Boot standard key.

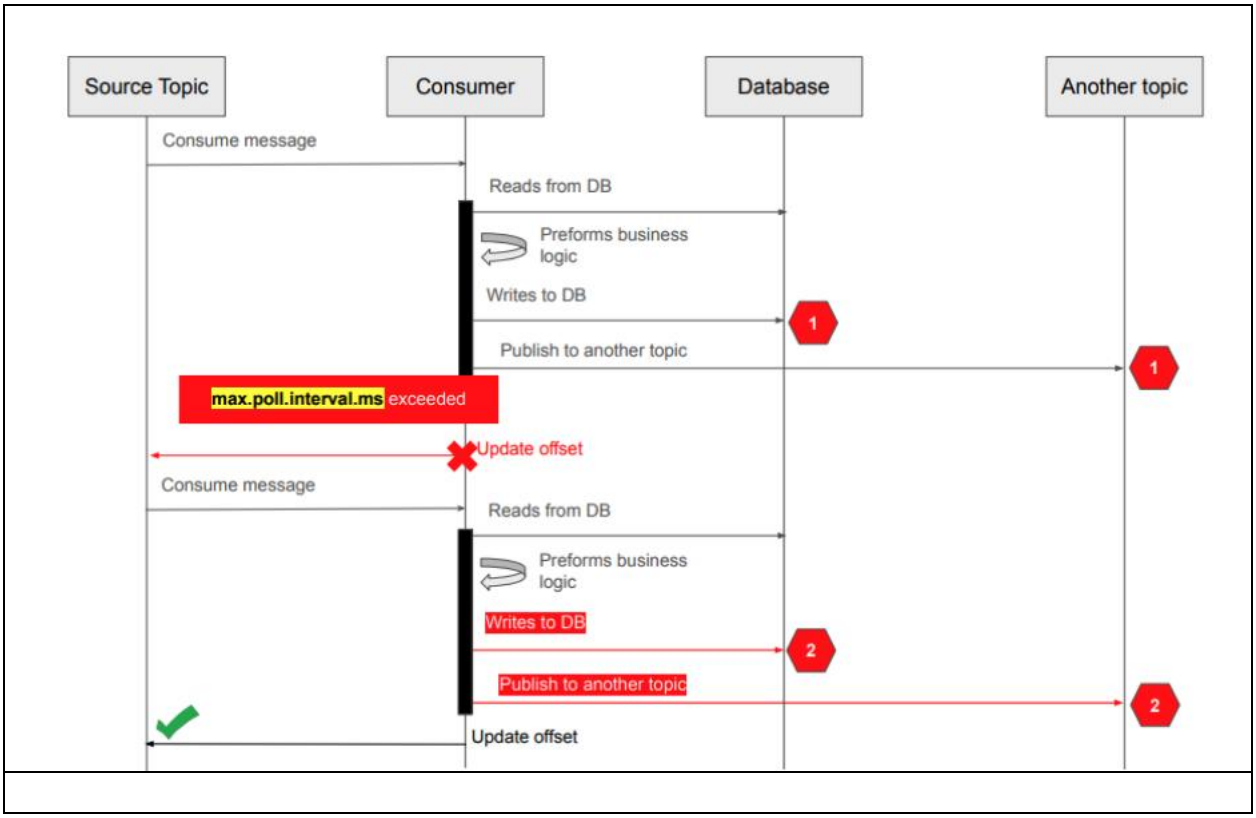
Note: If you use a custom property name, Spring Boot will not automatically apply it—you must read and apply it yourself (as shown above).

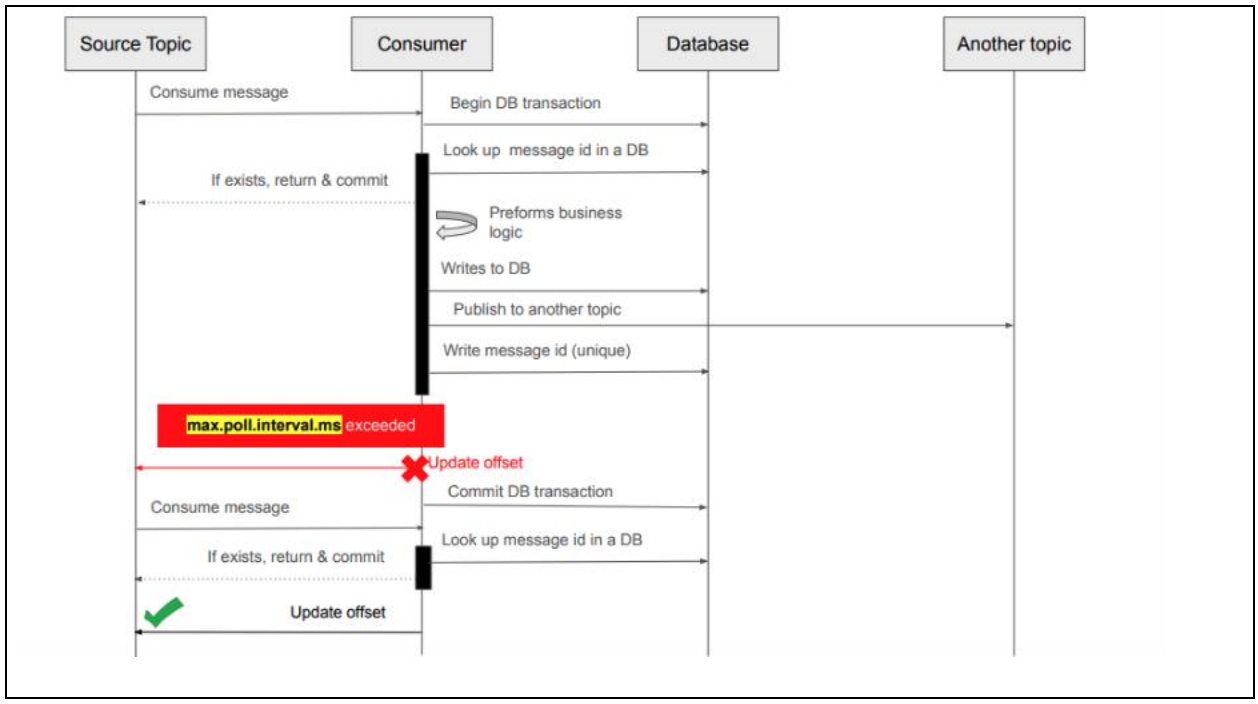
Summary

- group.id determines which consumer group the microservice belongs to.
- You can set it via:
  - @KafkaListener(groupId=...)
  - ConsumerFactory Java configuration
  - spring.kafka.consumer.group-id in application config
- Prefer a **single consistent approach** across the project to keep configuration predictable.

Section 14: Idempotent Consumer in Kafka?

Idempotent Consumer – Introduction?





Idempotent Kafka Consumer — Documentation Version

What is an idempotent consumer?

An **idempotent consumer** is a consumer that can receive and process the **same message multiple times** without causing any side effects or data inconsistencies.

- Even if Kafka delivers a message again (due to retries, failures, or rebalancing),
- The consumer ensures the business outcome happens **only once**.

This is a key technique for building **effectively "exactly-once" processing behavior** at the application level.

Why idempotent consumers are needed (duplicate processing scenarios)

Scenario A: Long processing time + max.poll.interval.ms exceeded

A common cause of duplicate consumption is when the consumer takes too long to process a record.

Flow:

1. Consumer polls messages from a topic and starts processing one message.
2. The message triggers long business logic:
    a. read from database
    b. compute

    c. write to database
    d. publish an event to another topic
3. Processing exceeds `max.poll.interval.ms` (maximum time allowed between polls).
4. Broker assumes the consumer is unhealthy, removes it from the group, and triggers a **rebalance**.
5. The message offset is **not committed**, so Kafka considers the record **not successfully consumed**.
6. Kafka delivers the same message again to another instance (or the same instance after rejoining).

Risk: the consumer may perform the business action twice:

- two database records written
- two downstream events produced
- potentially two payments made (critical issue)

Scenario B: Producer duplicates (non-idempotent producer)

If the producer is not idempotent, it may publish duplicate messages. Even if the consumer is stable, it can still receive duplicates and must handle them safely.

Key idea: idempotence is only part of the solution

An idempotent consumer helps a lot, but by itself it does not eliminate every edge case. For high reliability, you typically combine:

- **Idempotent producer**
- **Idempotent consumer**
- **Transactions** (to group DB + Kafka operations so they succeed/fail together)

Implementation approach: Deduplicate using a unique Message ID

Core strategy

Every Kafka message should have a **unique ID** (e.g., in message headers).

When a consumer receives a message, it should:

1. Read the message ID.
2. Check a database table to see if that ID was processed before.
3. If already processed → **skip**
4. If new → process business logic, then record the message ID as processed.

Recommended processing flow (with transactions)

1) Start transaction

Execute the consumer handler inside a database transaction (and optionally Kafka transaction depending on your design).

2) Check deduplication store first

- Look up `message_id` in a database table (e.g., `processed_messages`).

3) If already processed

- Return immediately.
- Commit transaction.
- Commit offset (so Kafka won't redeliver).

4) If not processed

- Execute business logic:
  - DB reads/writes
  - publish event to another topic (optional)
- Insert `message_id` into `processed_messages`
- Commit transaction
- Commit offset

Why this works during rebalancing / re-delivery

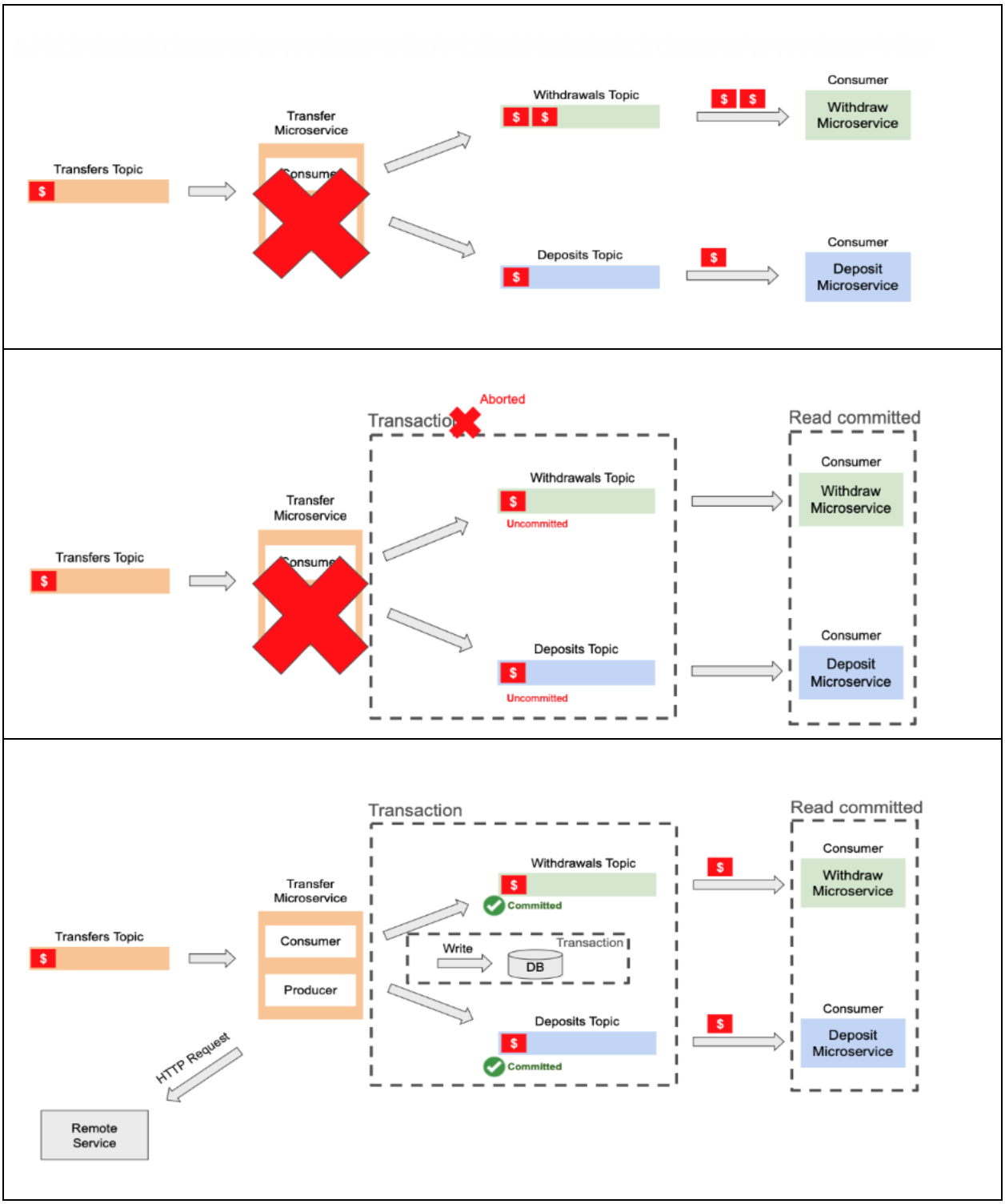
If the consumer is kicked out of the group due to `max.poll.interval.ms`, Kafka will redeliver the record.

But:

- Your original transaction might still have completed successfully (DB write already committed).
- So when the message comes again, the consumer checks the database, sees the message ID already processed, and skips it.
- The consumer then commits the offset so it won't be delivered again.

Section 15: Apache Kafka Transactions

Introduction to Transactions in Apache Kafka?


Transfers Topic
Transfer Microservice
Withdrawals Topic
Deposits Topic
Consumer
Withdraw Microservice
Consumer
Deposit Microservice
Aborted
Transaction
Read committed
Uncommitted
Uncommitted
Committed
Committed
Write
DB
Transaction
Producer
HTTP Request
Remote Service

Kafka Transactions — Documentation Version

Two key goals of Kafka transactions

Kafka transactions are designed to support two important guarantees:

1) All-or-nothing (atomic) behavior

Either **all Kafka operations inside a transaction succeed**, or **none of them take effect**.

- If any operation fails, the transaction is **aborted**
- Messages produced during the transaction are **not committed** and are not visible to consumers configured to read only committed data

2) Exactly-once semantics (EOS)

Kafka aims for "delivered once and only once" behavior using a combination of:

- **Idempotent producer**: prevents duplicate sends caused by retries
- **Transactions**: prevent consumers from reading messages that are part of an incomplete transaction

The common Kafka pattern: Consume → Process → Produce

A typical Kafka microservice often follows this workflow:

1. **Consume** a message from an input topic
2. **Process** it (business logic, DB interactions, validation, etc.)
3. **Produce** one or more messages to output topics for other services

Example scenario (money transfer workflow)

Imagine a topic called `transfer`, consumed by a **Transfer microservice**.

When a new message arrives in the `transfer` topic, the Transfer microservice processes it and may produce events to downstream topics such as:

- `withdrawal` (handled by a Withdrawal microservice)
- `deposits` (handled by a Deposits microservice)

What can go wrong without transactions?

Without transactions, this failure can occur:

1. Transfer microservice consumes a transfer request
2. It produces a message to the **withdrawal** topic
3. The service crashes before completing the rest of the workflow
4. Kafka re-delivers the original transfer message (since it was not fully "completed" from Kafka's perspective)

5. The Transfer microservice processes it again and produces **another withdrawal message**
6. Later it produces **only one deposit message**

Result:

- Withdrawal happens **twice**
- Deposit happens **once**
- This is a serious inconsistency (especially for payment-like use cases)

How transactions solve this problem

When transactions are enabled, the service performs multiple Kafka writes as a **single atomic unit**.

Transactional behavior (high-level)

1. Transfer microservice starts processing a message
2. It begins a Kafka transaction
3. It produces a message to withdrawal
4. It produces a message to deposits
5. If everything succeeds → it **commits** the transaction
6. If it crashes / an exception occurs → it **aborts** the transaction

Visibility rules: committed vs uncommitted

When a producer writes messages within a transaction:

- Those messages are initially marked as **uncommitted**
- Consumers configured with `isolation.level=read_committed` will **not see** them
- Only after the transaction commits do those messages become **visible**

So:

- If the service crashes mid-processing, the produced messages remain **uncommitted** and hidden
- Downstream services won't process partial output

This prevents "withdraw twice, deposit once" outcomes caused by partial progress.

What transactions guarantee (and what they don't)

What Kafka transactions guarantee

- **Atomic writes to multiple Kafka topics**
- **All-or-nothing** publishing of a set of records
- Downstream consumers can be protected from partial results using read_committed

What Kafka transactions do *not* automatically cover

Kafka transactions do not automatically include external side effects like:

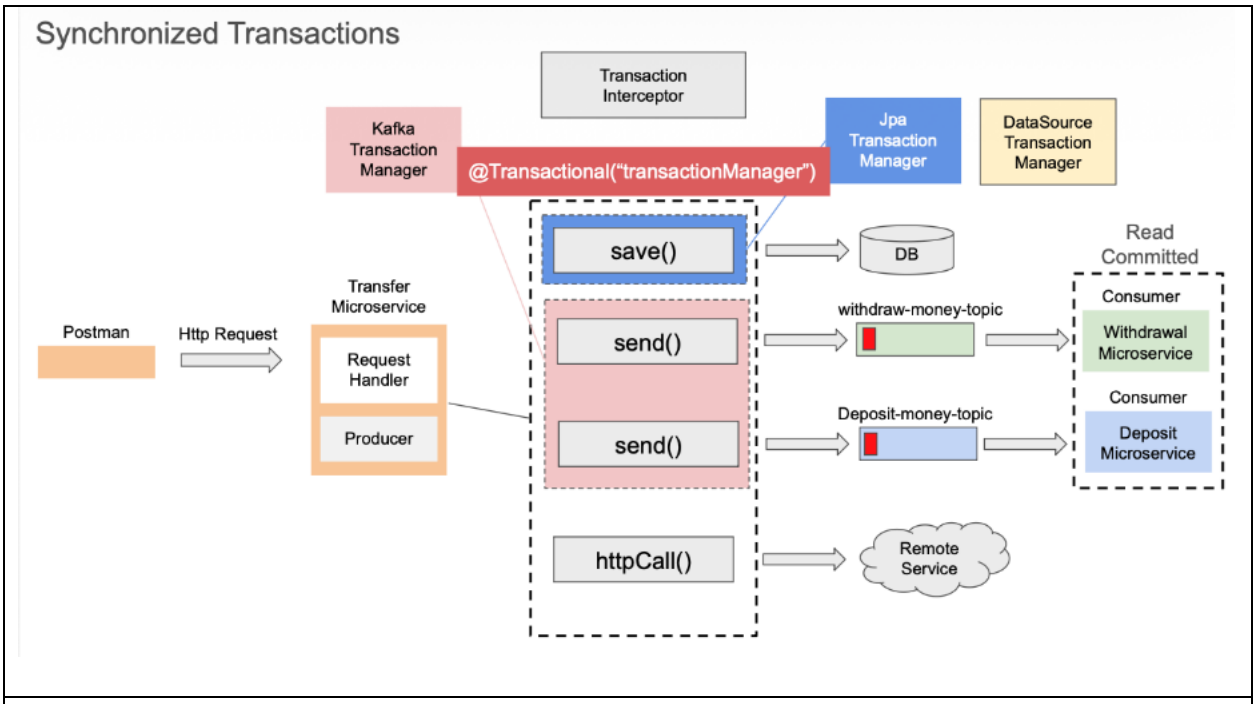
- **HTTP requests** to other microservices
  - If an HTTP call succeeds and then the Kafka transaction aborts, the HTTP call is **not rolled back**
  - Handling this requires **compensating transactions** (undo logic across services)
- **Database transactions**
  - Kafka does not manage DB transactions by itself
  - However, frameworks like **Spring** can integrate Kafka + DB transactions so they can be coordinated (so DB changes and Kafka sends can be rolled back together depending on configuration)

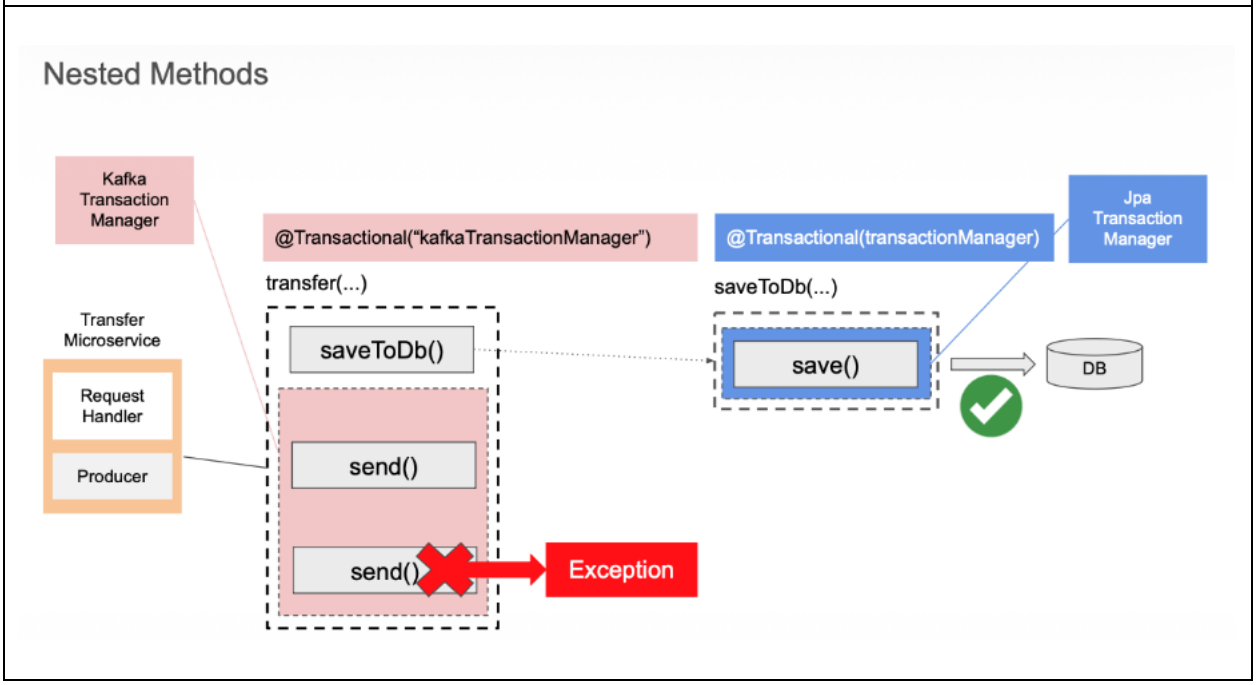
Summary

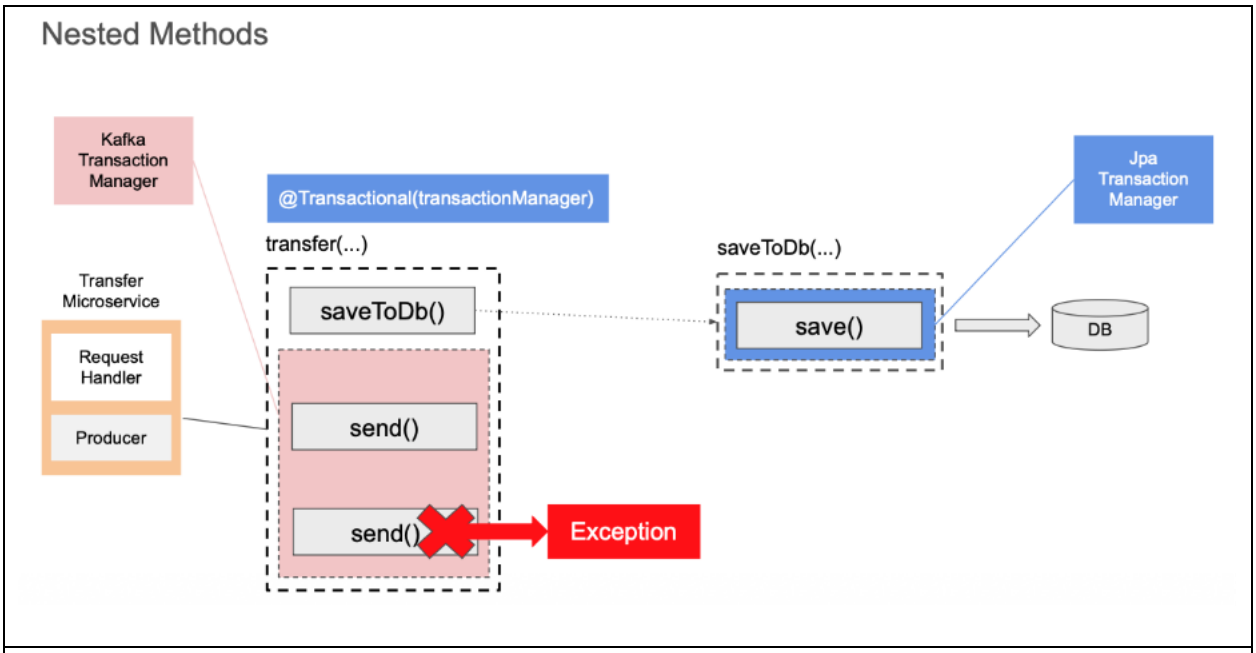
- Kafka transactions enable **atomic, all-or-nothing** behavior for producing messages across one or more topics.
- Combined with an **idempotent producer** and consumers configured for `read_committed`, they help implement **exactly-once processing patterns**.
- Transactions solve partial publish problems but don't automatically roll back external calls (HTTP) and need integration for DB consistency.

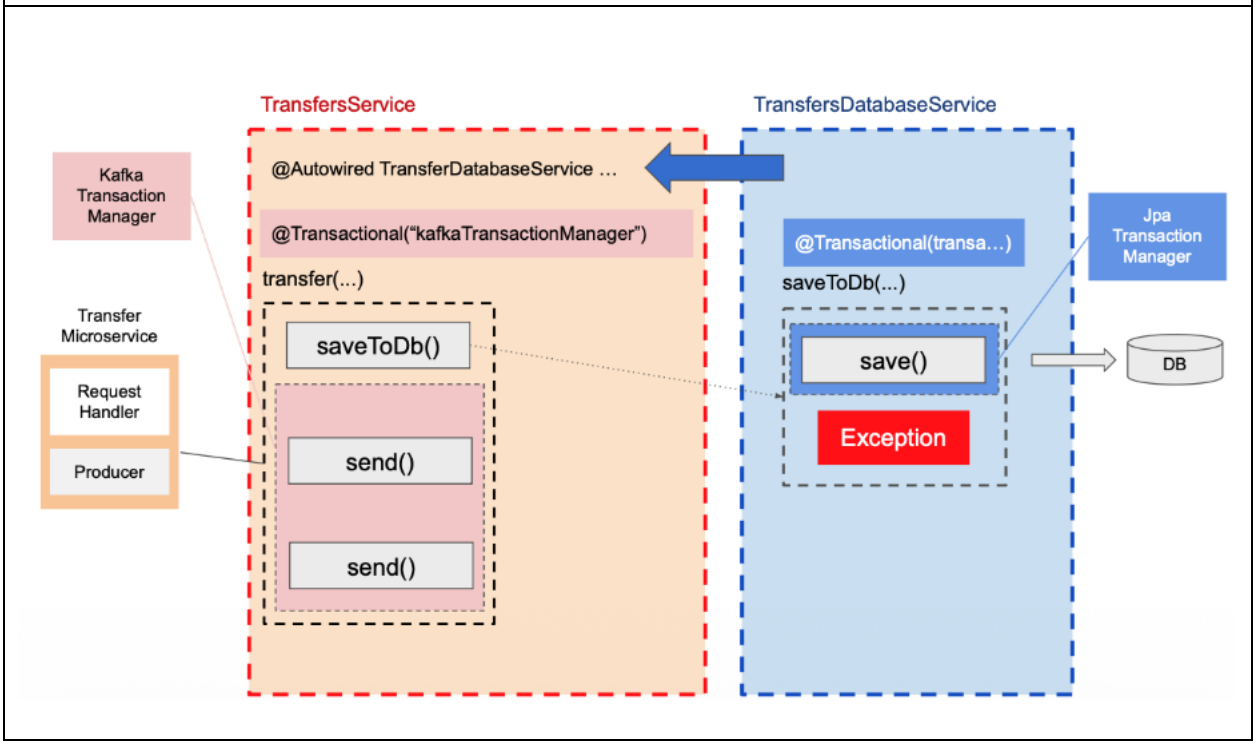
Section 16: Apache Kafka and Database Transaction

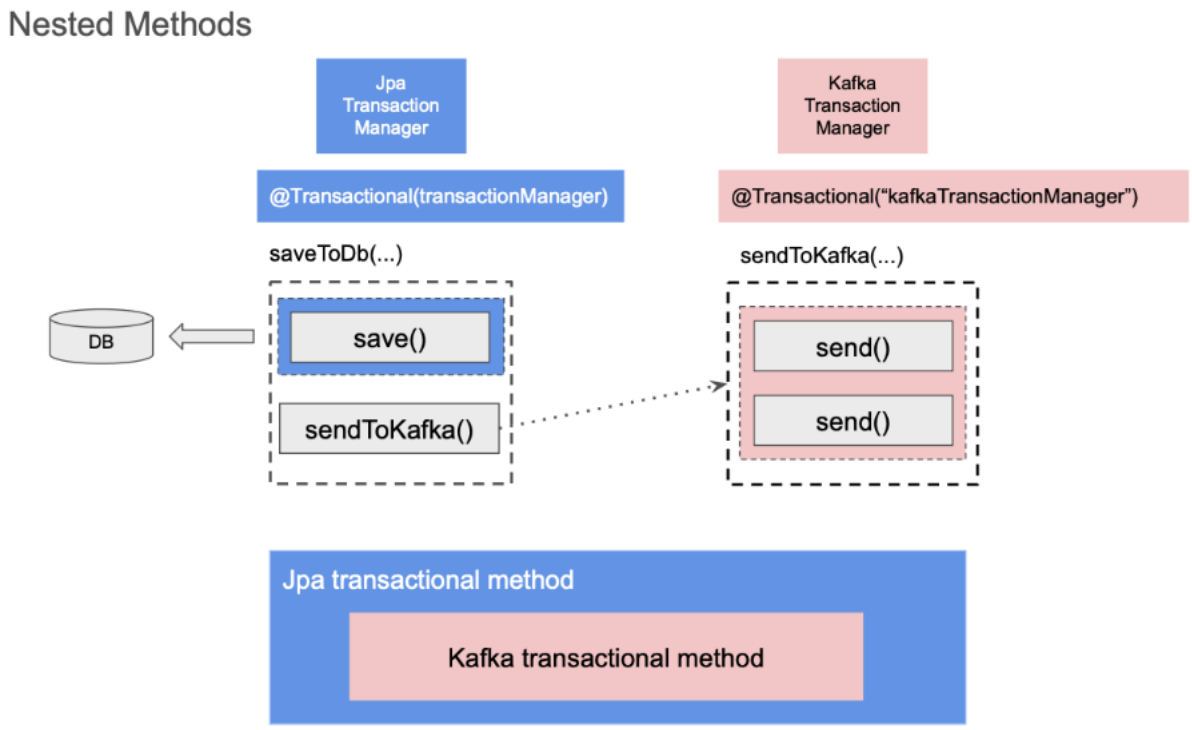
Kafka & Database Transactions – Introduction?


Synchronized Transactions
Transaction Interceptor
Kafka Transaction Manager
@Transactional("transactionManager")
Jpa Transaction Manager
DataSource Transaction Manager
save()
DB
Read Committed
Transfer Microservice
Postman
Http Request
Request Handler
Producer
send()
withdraw-money-topic
Consumer
Withdrawal Microservice
send()
Deposit-money-topic
Consumer
Deposit Microservice
httpCall()
Remote Service


Nested Methods
Kafka Transaction Manager
@Transactional("kafkaTransactionManager")
@Transactional(transactionManager)
Jpa Transaction Manager
transfer(...)
saveToDb(...)
Transfer Microservice
saveToDb()
save()
DB
Request Handler
Producer
send()
send()
Exception


Nested Methods
Kafka Transaction Manager
@Transactional(transactionManager)
transfer(...)
saveToDb(...)
Jpa Transaction Manager
Transfer Microservice
saveToDb()
save()
DB
Request Handler
send()
Producer
send()
Exception


TransfersService
TransfersDatabaseService
Kafka Transaction Manager
@Autowired TransferDatabaseService ...
@Transactional("kafkaTransactionManager")
@Transactional(transa...)
Jpa Transaction Manager
transfer(...)
saveToDb(...)
Transfer Microservice
saveToDb()
save()
DB
Request Handler
send()
Exception
Producer
send()



Kafka + Database Transactions (Spring) — Documentation Version

Goal

In many real systems, a single request triggers **both**:

- **Database work** (e.g., saving transfer state, audit logs)
- **Kafka work** (publishing events such as "withdraw requested" and "deposit requested")

The goal is to ensure **consistent outcomes**:

- If something fails, you typically want **both Kafka sends and DB updates to roll back**
- If everything succeeds, you want **DB + Kafka** changes to be committed in a predictable order

Reference flow: Transfer microservice

Components

- **Transfer microservice**

    - Receives an HTTP request (Postman/cURL)
    - Runs the business logic
    - Produces events to Kafka
    - Persists transfer details into a database table (in this section)
- **Kafka topics**
    - withdraw-money-topic
    - deposit-money-topic
- **Consumer microservices**
    - Withdrawal microservice consumes withdraw topic
    - Deposit microservice consumes deposit topic
    - Both consumers are configured to read **committed only** (read_committed) so they do not process uncommitted/aborted transactional records

Kafka transactions (producer-side)

In the transfer service, Kafka transactions are enabled so that multiple sends are grouped into one atomic unit.

What it means

When the method that sends messages is transactional:

- Sending to withdraw-money-topic
- Sending to deposit-money-topic

...both happen inside **one Kafka transaction**, typically managed by a **Kafka Transaction Manager**.

Outcome

- If the Kafka transaction commits:
    - Both records become **committed** and visible to read_committed consumers
- If the Kafka transaction aborts:
    - Records are not committed and should not be processed by read_committed consumers

Adding database operations to the same flow

In the next step, the same service method also performs database work (e.g., repository.save(...)).

Now the method performs **Kafka operations + DB operations** together.

This introduces a key design question:

"Do Kafka operations and DB operations participate in one transaction or two?"

That depends on **which transaction manager** starts the transaction for the method.

Two transaction managers involved

In this section, you effectively have:

- **Kafka Transaction Manager**
  - Manages Kafka producer transactions
- **Database transaction manager** (e.g., JpaTransactionManager / DataSourceTransactionManager)
  - Manages database transactions

How Spring ties this together (TransactionInterceptor)

Spring uses a component often referred to conceptually as the **transaction interceptor**:

- When a method annotated with Transactional is invoked, the interceptor runs
- It starts a transaction **before** the method execution
- It commits or rolls back **after** the method finishes (depending on success/failure)

Why Kafka can join a DB transaction

When the interceptor starts a **database transaction**, the KafkaTemplate can **synchronize** its own transactional behavior with that ongoing transaction.

So even though Kafka and DB have different transaction managers, Spring can coordinate them within the same annotated method.

Commit order (important detail)

When both Kafka sends and DB updates participate in the same Spring-managed transaction:

- **Database commits first**
- **Kafka commits second**

So the overall effect is:

1. DB write is finalized
2. Kafka records are committed (and become visible to consumers)

Common pitfall: splitting logic across methods incorrectly

You may refactor like this:

- **Outer method**: Kafka sends
- **Inner method**: database save

Scenario A — Outer method uses Kafka transaction manager

If the **outer method** is annotated with

Transactional using **KafkaTransactionManager**, and the inner DB save is executed via a JPA repository:

- Kafka transaction is managed by Kafka TM
- Repository save runs under a **separate DB transaction** (managed by JPA TM)

Result: two separate transactions.

Consequence

If an exception happens in Kafka flow:

- Kafka transaction rolls back
- DB transaction may still **commit successfully**

In most business workflows, this is **not desired** (you don't want the DB showing "transfer saved" if the events were not successfully committed).

Recommended approach: make the DB transaction the "outer" transaction

To ensure consistent rollback across Kafka + DB operations:

Use Transactional with the database transaction manager on the outer method

When the outer method is driven by the DB transaction manager:

- DB transaction is started by Spring
- KafkaTemplate synchronizes Kafka operations with that transaction
- If an exception occurs, Spring rolls back the overall transaction, causing:
  - DB rollback
  - Kafka transaction abort

This is the strategy that gives the most "all-or-nothing" behavior for DB + Kafka within the same service.

Key takeaways

- **Kafka transactions** guarantee atomicity across Kafka sends, but do not automatically include DB work unless coordinated.
- If Kafka-managed transaction is the outermost scope, DB operations may run in a separate transaction and still commit.
- To have Kafka + DB behave like a single unit, the safest pattern is:
  - **Outer Transactional managed by DB transaction manager**
  - KafkaTemplate participates via transaction synchronization
- Commit order in coordinated flows is typically:
  - **DB commit first, then Kafka commit**

Section 19: Saga Design Pattern and apache Kafka

Orchestration Based Saga?

Orchestration-Based Saga (Formatted Transcription)

What it is

In an **orchestration-based saga**, one microservice (often the **Order** service) contains a **Saga class**. This Saga class acts as an **orchestrator/manager** that:

- **Consumes events**
- **Decides the next step**
- **Publishes commands** to other microservices
- Continues until the transaction is completed or rolled back

Happy Path (Success Flow)

Step 1: Create Order

- **Order Service** receives an HTTP request to create a new order.
- It creates the order in its database.
- It publishes an event: OrderCreatedEvent

Step 2: Reserve Product

- **Saga** consumes OrderCreatedEvent
- Saga publishes command: ReserveProductCommand
- **Product Service** consumes ReserveProductCommand
- Product Service reserves stock and publishes: ProductReservedEvent

Step 3: Process Payment

- **Saga** consumes ProductReservedEvent
- Saga publishes command: ProcessPaymentCommand

- **Payment Service** consumes ProcessPaymentCommand
- If payment is successful, it publishes: PaymentProcessedEvent (or PaymentProcessEvent, depending on naming)

Step 4: Continue to Shipment (Optional extension)

At this point the order can be marked approved and shipment can begin. Depending on your design:

- you can start a **new saga**, or
- continue with **more steps in the same saga**

Example continuation:

- Saga consumes PaymentProcessedEvent
- Saga publishes: CreateShipmentTicketCommand
- **Shipment Service** consumes it and publishes: ShipmentTicketCreatedEvent
- Saga consumes ShipmentTicketCreatedEvent
- Saga publishes: ApproveOrderCommand
- **Order Service** (handler class) consumes ApproveOrderCommand, updates DB to APPROVED
- Order Service publishes: OrderApprovedEvent

Failure Path (Compensation / Rollback Flow)

Scenario: Payment fails

Assume:

- Payment Service cannot verify payment details or charge the user.
- Since payment failed:
  - we **cannot ship**
  - we **cannot keep product reserved**
  - we must **rollback** previous changes

Key idea: Compensations happen in reverse order

A compensation is an action that **undoes** a previous successful step.

Compensation Flow Example

1) Payment failure event is published

- Payment Service publishes an event like:
  - CardAuthorizationFailedEvent (your transcription says "CART Authorization Failed event", likely intended "Card")

2) Cancel product reservation

- Saga consumes CardAuthorizationFailedEvent
- Saga publishes command: CancelProductReservationCommand
- Product Service consumes it

- Product Service marks product back to **AVAILABLE**
- Product Service publishes: ProductReservationCancelledEvent

3) Reject the order

- Saga consumes ProductReservationCancelledEvent
- Saga publishes command: RejectOrderCommand
- Order Service consumes it
- Order status changes from **OPEN/CREATED** to **REJECTED**
- Order Service publishes: OrderRejectedEvent (or equivalent)

Summary Rules to Remember

- **Saga orchestrates the workflow**
- Saga does not "do the work"; it:
  - publishes commands
  - consumes events
  - decides what's next
- On failure, Saga triggers **compensating transactions**
- Compensations should generally be executed in **reverse order** of the successful steps

Reserve Product in Stock?

Reserve Product in Stock (Formatted Transcription)

Overview

This flow starts when a client places an order and continues through the **product reservation** step, ending when the saga triggers the **payment** step.

Step-by-Step Flow

1) Client places an order (Orders Service)

- A client application sends an **HTTP POST** request to create/place a new order.
- The request is handled by the **Orders Controller**.
- The controller passes the request data to an **Orders Service** class.
- The Orders Service saves the order details into the **orders database table**.

2) Orders Service publishes OrderCreatedEvent

- After the order is successfully stored, the Orders Service publishes an event:
  - OrderCreatedEvent
- This event is sent to the **orders events topic**.

This is the point where the **saga flow begins**.

3) Saga consumes OrderCreatedEvent **and publishes** ReserveProductCommand

- The **Orders Saga** class consumes OrderCreatedEvent.
- The saga publishes a command (commands are treated like "instructions" to other services):
  - ReserveProductCommand
- This command is published to the **products commands topic**.

4) Product Service reserves stock and publishes ProductReservedEvent

- The **Products Service** is subscribed to the products commands topic.
- It consumes ReserveProductCommand.
- It reserves the product in stock (updates inventory/state).
- After reserving the product, it publishes:
  - ProductReservedEvent
- This event is published to the **products events topic**.

5) Saga consumes ProductReservedEvent **and moves to payment**

- The **Orders Saga** consumes ProductReservedEvent.
- The saga begins the next step: processing payment.
- It publishes:
  - ProcessPaymentCommand

Summary (One-line)

HTTP POST → **save order** → OrderCreatedEvent → **saga** → ReserveProductCommand → **product reserves stock** → ProductReservedEvent → **saga** → ProcessPaymentCommand**.**

Section 20: Saga: Part 2: Compensating Transaction?

Introduction to Compensating Transactions in Saga (Formatted Transcription)

What a Saga is

A **Saga** is an event-handling component that manages a **sequence of local transactions** across multiple microservices.

In earlier lessons, the saga handled a simple flow:

- it consumed an event
- then published a command
- which triggered the next local transaction in the saga

Why compensating transactions are needed

If **one of the local transactions fails**, the saga must run a series of **compensating transactions**.

Goal of compensating transactions

Compensating transactions **undo changes** made by the earlier (successful) steps in the saga.

Example:

- If `ProcessPaymentCommand` fails,
- the saga must undo the preceding modifying transaction:
    - `ReserveProductCommand` (because stock was reserved and must be released)

Core rule: compensations happen in reverse order

This is a key point in the Saga pattern:

- **Modifying transactions happen forward**
- **Compensating transactions happen backward (reverse order)**

So you undo:

- the **last** successful modification first,
- then move backward step-by-step until the **first** modification is undone.

Important note: only compensate steps that changed state

You **do not** need compensating transactions for saga steps that did **not** modify your system.

Example:

- If a saga step is only a **read operation**, there is nothing to undo.

Good practice: store history of events/status changes

It's a best practice to store a history of what happened during the saga.

In the example system (Orders, Products, Payments):

- each microservice has its **own database**
- in the Orders microservice, we also store an **Order History** table

How the history table works

- The **Orders table** stores the current state (e.g., CREATED, APPROVED, REJECTED)
- The **Order History table** stores an append-only log of status changes with timestamps

When compensation happens:

- the existing row in the **Orders table** is updated to the new status
- but the **History table is NOT updated or modified**
- instead, a **new history row is inserted**

Why this is useful

- **Auditing**
- **Debugging**
- **Compliance**
- Similar concept to **event sourcing**, where you keep all past events to reconstruct state at any time.

Key takeaway

When executing compensating transactions:

- **do not delete**
- **do not edit past history records**
- **only append new history records**

- Section 1: Introduction to Apache Kafka
    - Introduction
    - What is Microservice
    - Microservice vs Monolithic application
    - Microservice communication
    - Event driven architecture with Apache Kafka
    - Apache Kafka for Microservices
    - Messages and Events in Apache Kafka
    - Kafka Topic and Partitions
    - Ordering of events in Apache Kafka
- Section 2: Apache Kafka Brokers
    - What is Apache Kafka Broker?
    - Apache Kafka Broker: Leader and Follower Roles. Leadership balance
    - Multiple Kafka broker: configuration files
    - Multiple Kafka Broker: Storage folders
    - Starting multiple Kafka broker with KRaft
    - Stopping Apache Kafka Brokder
- Section 3: Kafka CLI: Topics
    - Introduction to Kafka Topic CLI
    - Creating a new Kafak Topic
    - List and describe Kafka Topic
    - Deleting Kafka Topic
- Section 4: Kafka CLI: Producers
    - Introduction to Kafka Producer CLI
    - Producing Kafka Message without a Key
    - Sending Kafka Message as a Key:Value Pair
- Section 5: Kafka CLI: Consumers
    - Introduction to Kafka Consumer CLI
    - Consuming messages from Kafka topic from the beginning
    - Consuming new Kafka messages only
    - Consuming Key value pari messages from Kafka topic
    - Consuming Kafka message in order
- Section 6: Kafka Producer – Spring boot Microservice
    - Introduction to Kafka Producer
    - Kafka Producer – Introduction to synchronous communication style
    - Kafka Producer – A use case for asynchronous communication style
    - Kafka Producer configuration properties/yaml
    - Creating Kafka Topic
    - Creating an Event class
    - Kafka Producer: Send Message Asynchronously

    - Kafka Asynchronous Send: Message Synchronously
    - Kafka Producer: Handle Exception in Controller
    - Logging Record Metadata information
- Section 7: Kafka Producer: Acknowledgements & Retries
    - Kafka Producer Acknowledgement: Introduction
    - Kafka Producer Retries: Introduction
    - Configure Producer Acknowledgements in Spring Boot Microservices
    - The min.insync.replicas configuration
    - Kafka Producer Retries
    - Kafka Producer Delivery & Request Timeout
    - Kafka Producer Spring Boot Configuration
- Section 8: Idempotent Producer in Kafka
    - Introduction to Idempotent Kafka Producer
    - Enable Kafka Producer Idempotence in application.properties
    - Enable Kafka Producer Idempotence in Spring bean
- Section 9: Kafka Consumer – Spring Boot Microservice
    - Introduction to Kafka Consumer
    - Kafka Consumer Configuration Properties
    - Kafka Consumer with @KafkaEventListener and @KafkaHandler annotation
    - Kafka Consumer spring bean configuration
    - Kafka Listener Container Factory
- Section 10: Kafka Consumer – Handle Deserializer Errors
    - Introduction to Error Handling in Kafka Consumer
    - Causing the deserialization problem
    - Kafka Consumer ErrorHandlingDeserializer
- Section 11: Kafka Consumer – Deal Letter Topic (DLT)
    - Introduction to Dead Letter Topic (DLT) in Kafka
    - Handle errors: The DefaultErrorHandler and DeadLetterPublishingRecoverer classes
    - Create and Configure KafkaTemplate object
- Section 12: Kafka Consumer – Exceptions and Retries
    - Introduction to Exception Handling in Kafka consumer and retries
    - Creating retryable and not retryable exceptions in Kafka
    - Configure DefaultErrorHandler with a list of not retryable exceptions
    - Register RetryableException and define wait time interval
    - Throwing a RetryableException

    - Overview of a destination microservice
- Section 13: Kafka Consumer: Multiple Consumers in a Consumer Group
    - Introduction to Kafka Consumer Group
    - Rebalancing and partition assignment in Kafka
    - Assigning Microservice to a cosumer group in Kafka
    - Multiple consumers consuming messages from Kafka Topic

- Section 14: Idempotent Consumer in Kafka
    - Idempotent Consumer – Introduction
    - Include a unique id into message header
    - Reading a unique id from message header
    - Adding database-related dependencies
    - Configure database connection details
    - Creating JPA Entity
    - Create JPA Repository
    - Storing a unique message id in a database table
    - Check if Kafka message was processed earlier
- Section 15. Apache Kafka Transactions
    - Introduction to Transactions in Kafka
    - Enable Kafka Transactions in application properties
    - Enable Kafka Transaction in the @Bean method
    - Kafka and @Transactional annotaion
    - Rollback transaction for speicific exception
    - Reading committed messages in Kafka Consumer
    - Kafka local transaction with KafkaTemplate
- Section 16: Kafka and DAtabase Transactions
    - Kafka & Database Transaction – Introduction
    - Creating Jpa Transaction Manager
    - Synchronized Transaction: Saving to Database
    - Enable logging for Kafka and Jpa Transaction Manager
- Section 17: Integration Testing – Kafka Producer
    - Test class Annotation
    - Test method structure – Arrange, Act & Assert
    - Implementing the Arrange and Act Sections

    - Kafka Consumer Configuration in Test class
    - The setUp() and tearDown() methods
    - Verify the Kafka Producer Configuration Properties
    - Test Method for Idempotent Kafka Producer
- Section 18: Saga design pattern with Apache Kafka
    - Choreography Based Saga
    - Orchestration Based Saga
    - Reserve product in Stock – Introduction
    - Publish OrderCreatedEvent
    - Handle OrderCreatedEvent
    - Send ReserveProductCommand
    - Save order status in History database table
    - Handle ReserveProdcutCommand
    - Reserve Product in Stock
    - Publish the ProductReserveEvent
    - Publish ProductReservationFailedEvent
    - Handle the ProductReserveEvent
    - Handle the ProcessPaymentCommand
    - Handle Process Payment Command
    - Publish Payment Processed and Payment Failed
    - Handle PaymentProceesedEvent
    - Handle the ApproveOrderCommand
    - Handle the OrderApprovedEvent
- Section 19: Saga Part 2: Compensating Transaction
    - Introduction to Compensating Transaction in Saga
    - Handle PaymentFailedEvent
    - Publish CancelProductReservationCommand
    - Handle PaymentFailedEvent
    - Publish CancelProductReservation Command
    - Handle CancelProductReservationCommand
    - Publish ProductReservationCancelled Event
    - Handle ProductReservationCancelledEvent
    - Handle the RejectOrderCommand
- Section 20: Kafka Connect
    - Introduction to Kafka Connect
    - When and why to use Kafka Connect?

- Kafka Connect Core Components: workers, connectors, and Tasks
- Kafka Connect Key Components: Transforms, Converters